Lecture 7:

Memory management

CS211 - Operating systems

Spring 2026

Projects

- Possible topics
  - Implement a terminal to run programs
    - Run the code from a file and handle exceptions
  - Implement a system monitor (CPU cores, memory, processes)
  - Implement a file browser system
    - Command line is ok
  - Implement a web server to illustrate thread-based vs event-based programming
- Each group (3-5 students)
  - Only use standard low-level libraries of C/C++, Java, etc
  - Do not use the code of other public projects
  - Do not use the code generated by AI tools

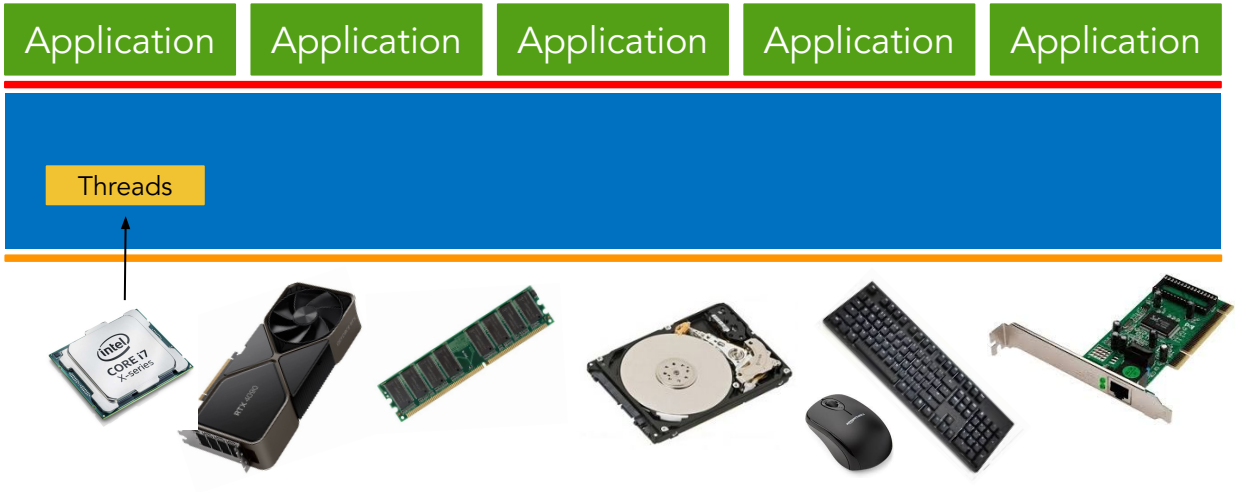
Recall: Why do we need operating systems?

Recall: Why do we need operating systems?

OS interface

Hardware interface

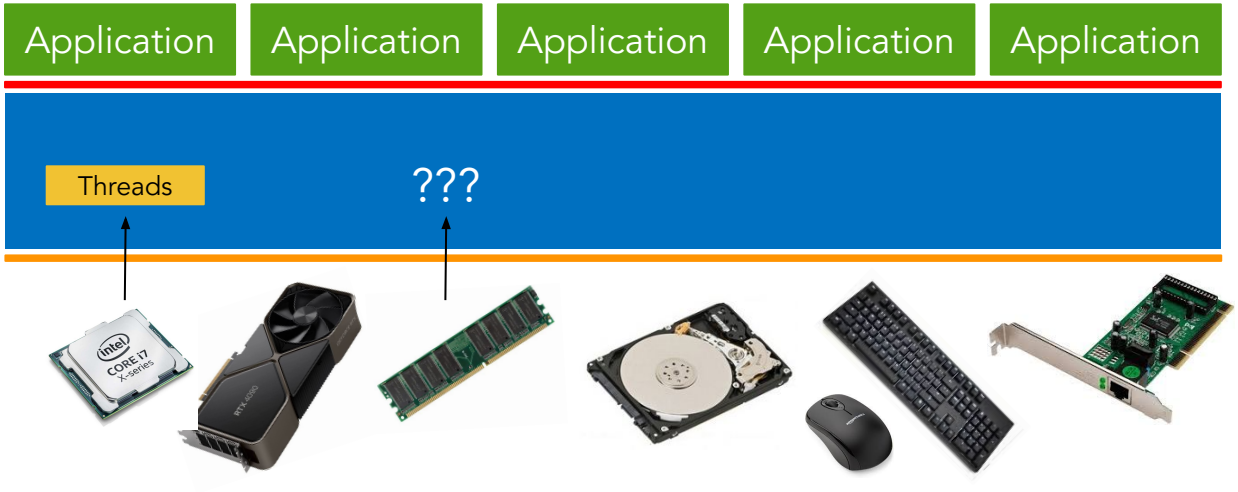
Recall: Why do we need operating systems?



OS interface

Hardware interface

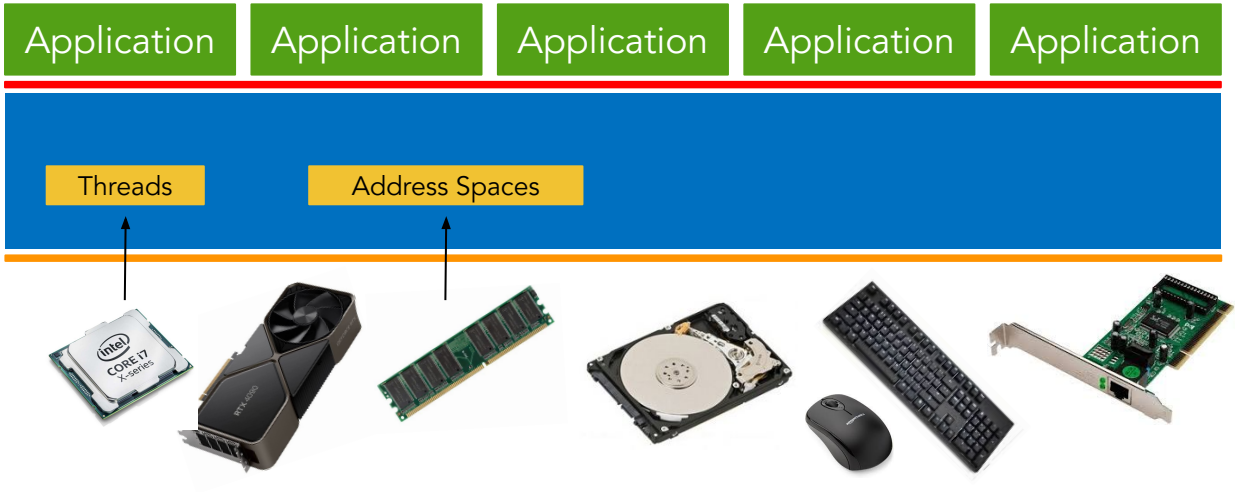
Recall: Why do we need operating systems?



OS interface

Hardware interface

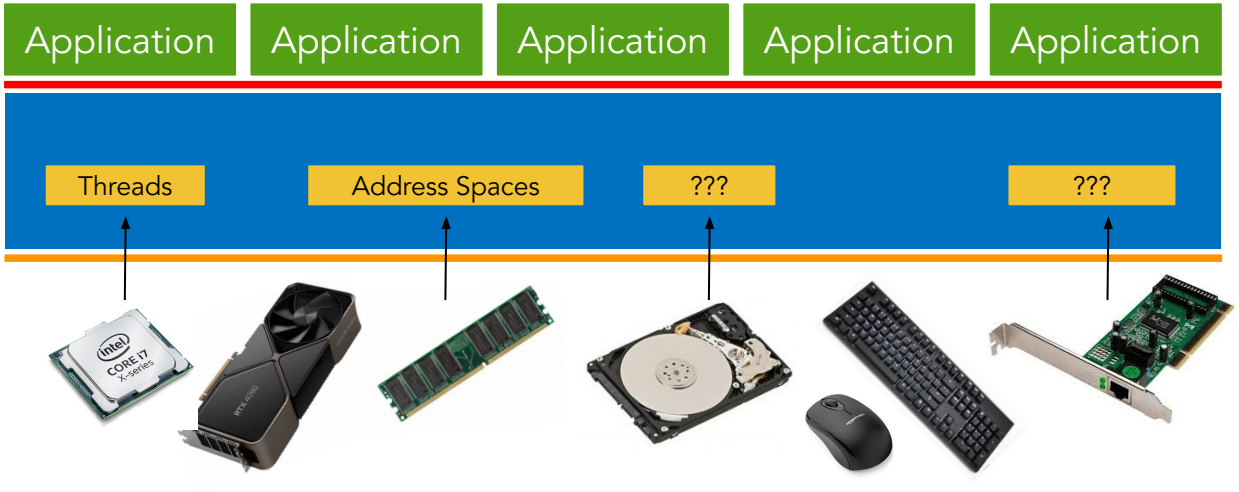
Recall: Why do we need operating systems?



OS interface

Hardware interface

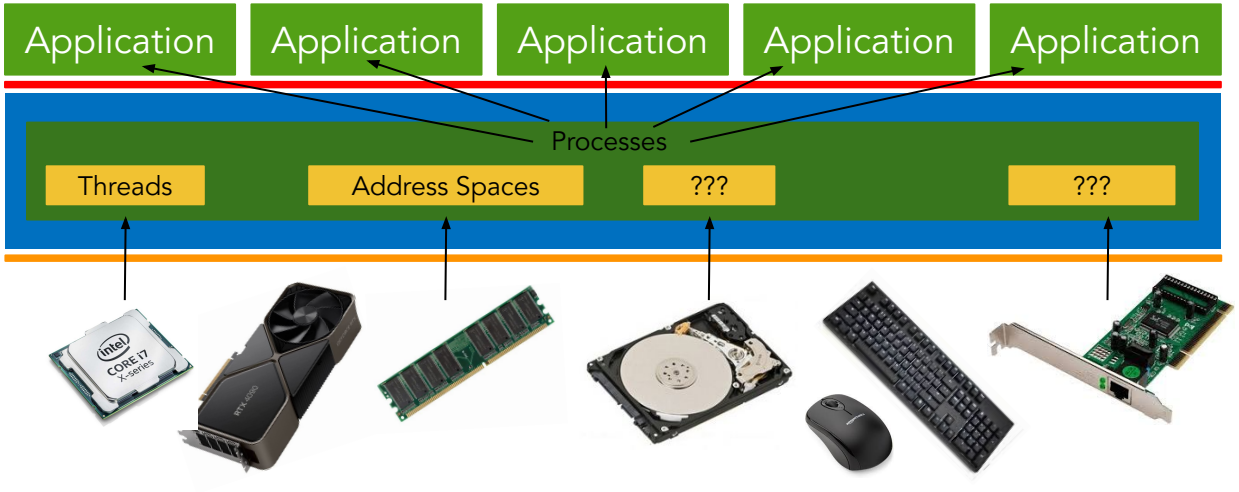
Recall: Why do we need operating systems?



OS interface

Hardware interface

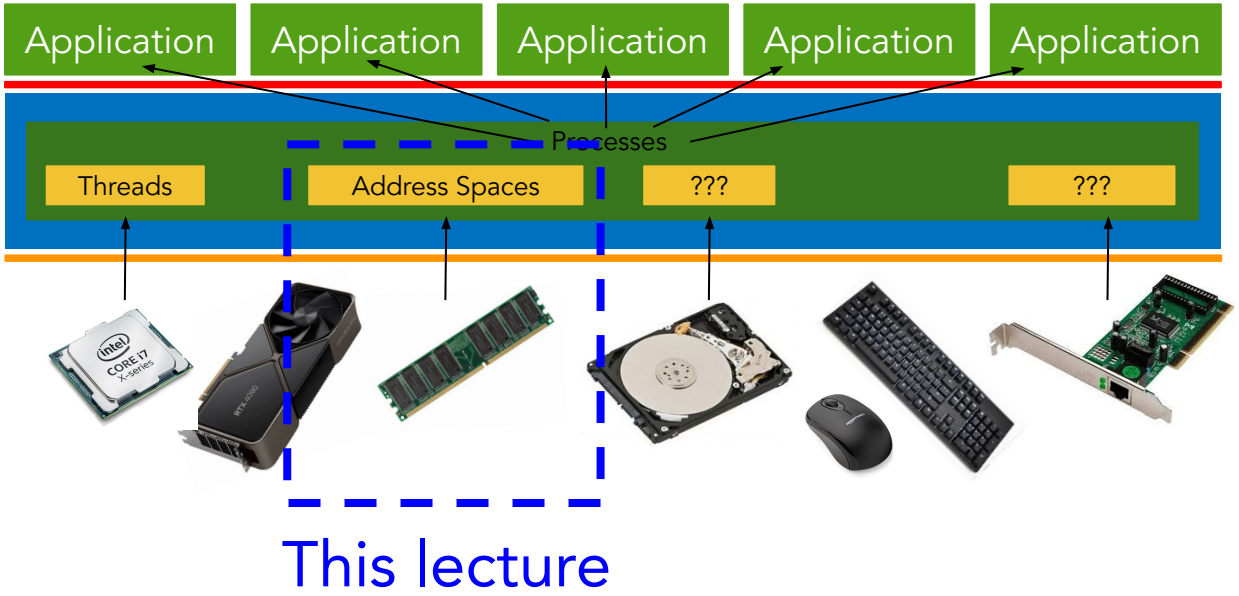
Recall: Why do we need operating systems?



OS interface

Hardware interface

Recall: Why do we need operating systems?



Objectives of memory management

- Isolation
  - Don’t want a process to have access to the memory of other processes
- Sharing
  - Want option to overlap when desired (for efficiency and communication)
- Virtualization
  - Want to create the illusion of more resources than exist in underlying physical system
- Utilization
  - Want the best use of this limited resource
    - Expandable memory
    - Relative memory addresses
    - No fragmentation

Objectives of memory management

- Isolation
  - Don't want a process to have access to the memory of other processes
- Sharing
  - Want option to overlap when desired (for efficiency and communication)
- Virtualization
  - Want to create the illusion of more resources than exist in underlying physical system
- Utilization
  - Want the best use of this limited resource
    - Expandable memory
    - Relative memory addresses
    - No fragmentation
      - What is it?

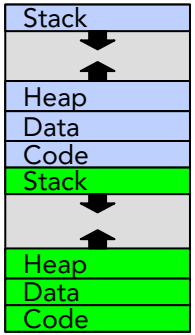
Fragmentation

Fragmentation

- Internal fragmentation: Space inside allocated address space may not be fully used

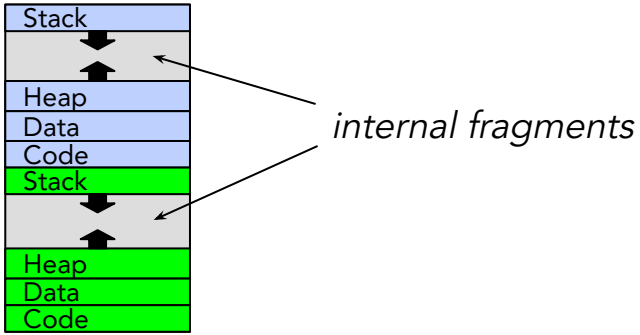
Fragmentation

- Internal fragmentation: Space inside allocated address space may not be fully used



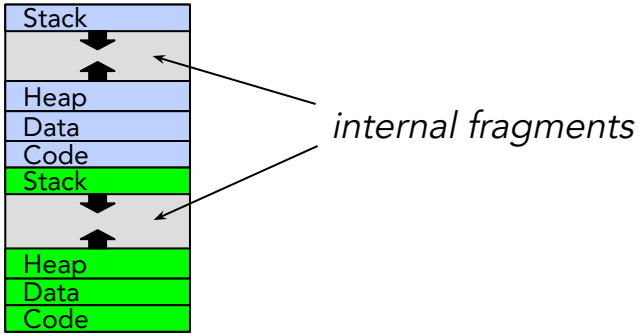
Fragmentation

- Internal fragmentation: Space inside allocated address space may not be fully used



Fragmentation

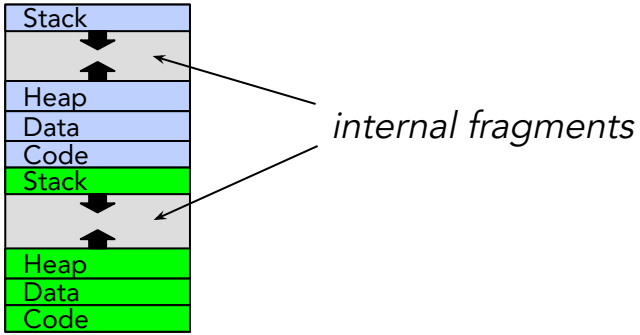
- Internal fragmentation: Space inside allocated address space may not be fully used



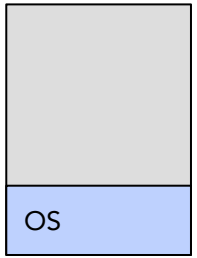
- External fragmentation: Free chunks between allocated regions

Fragmentation

- Internal fragmentation: Space inside allocated address space may not be fully used

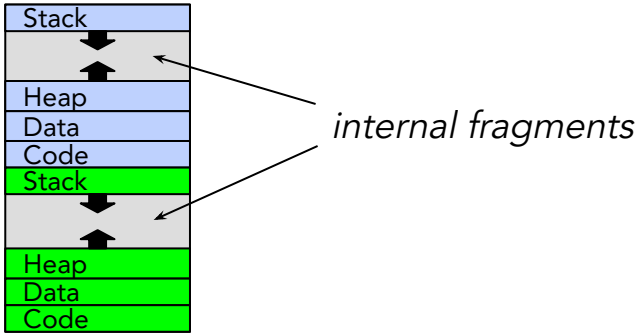


- External fragmentation: Free chunks between allocated regions

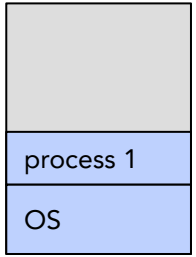


Fragmentation

- Internal fragmentation: Space inside allocated address space may not be fully used

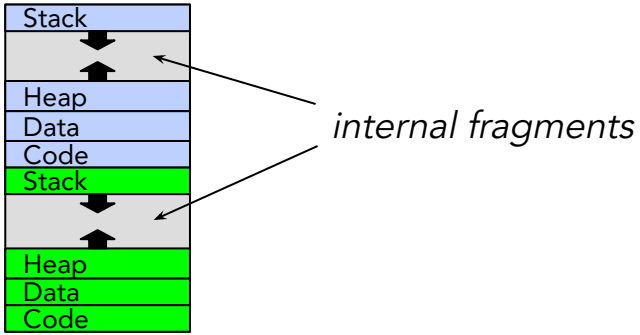


- External fragmentation: Free chunks between allocated regions

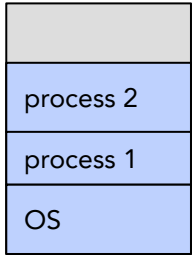


Fragmentation

- Internal fragmentation: Space inside allocated address space may not be fully used

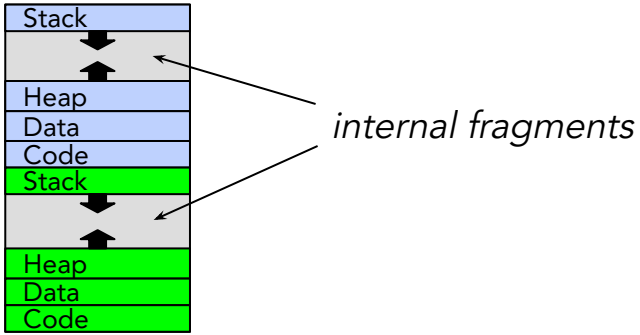


- External fragmentation: Free chunks between allocated regions

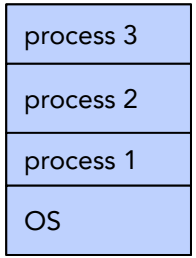


Fragmentation

- Internal fragmentation: Space inside allocated address space may not be fully used

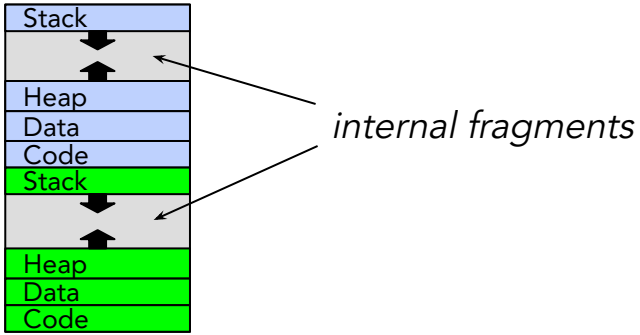


- External fragmentation: Free chunks between allocated regions

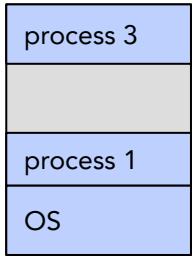


Fragmentation

- Internal fragmentation: Space inside allocated address space may not be fully used

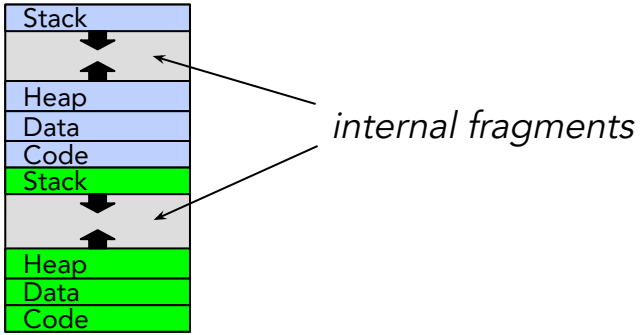


- External fragmentation: Free chunks between allocated regions

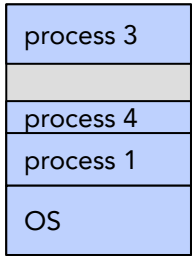


Fragmentation

- Internal fragmentation: Space inside allocated address space may not be fully used

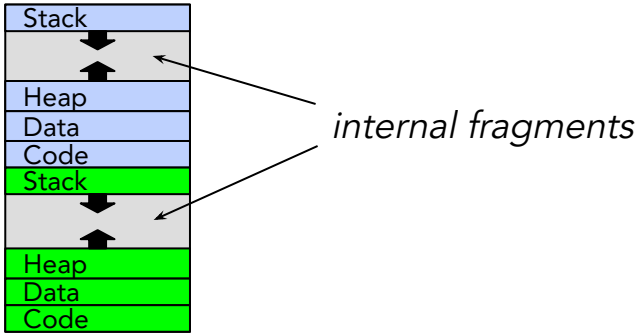


- External fragmentation: Free chunks between allocated regions

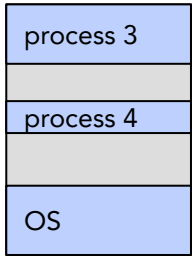


Fragmentation

- Internal fragmentation: Space inside allocated address space may not be fully used

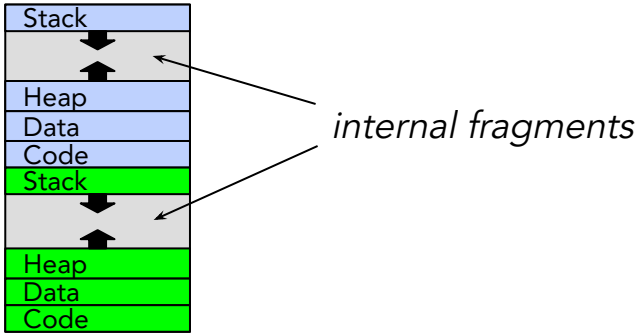


- External fragmentation: Free chunks between allocated regions

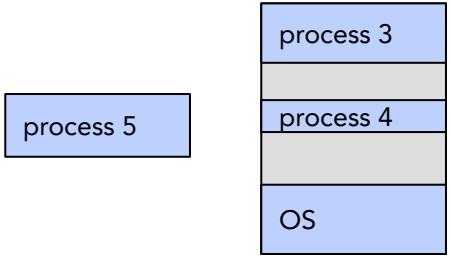


Fragmentation

- Internal fragmentation: Space inside allocated address space may not be fully used

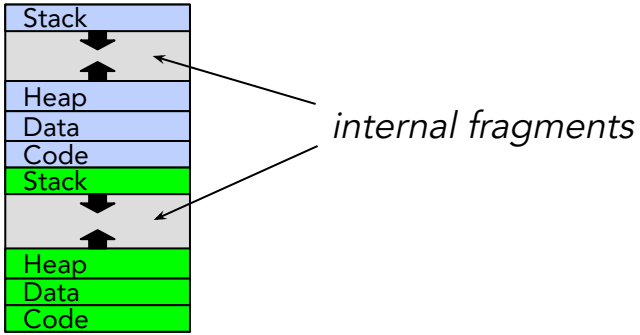


- External fragmentation: Free chunks between allocated regions

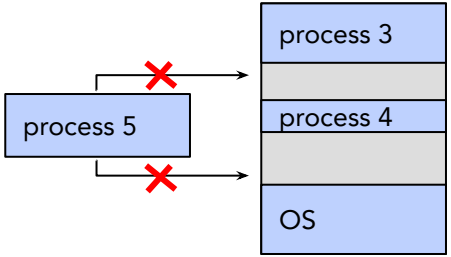


Fragmentation

- Internal fragmentation: Space inside allocated address space may not be fully used

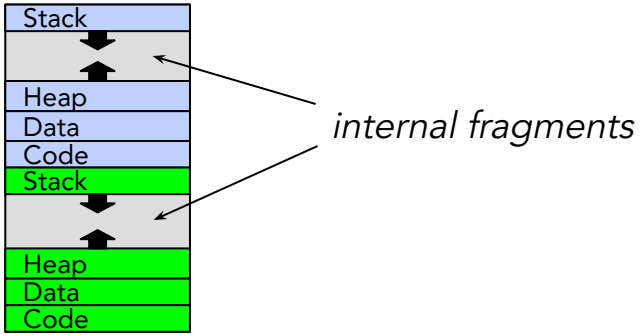


- External fragmentation: Free chunks between allocated regions

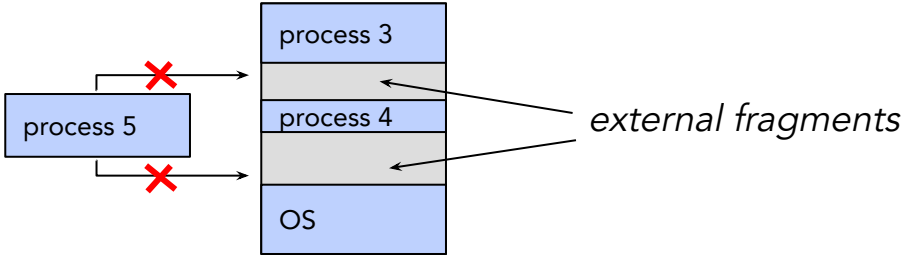


Fragmentation

- Internal fragmentation: Space inside allocated address space may not be fully used

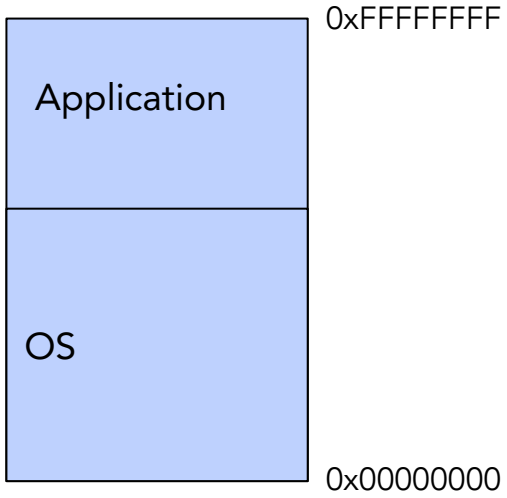


- External fragmentation: Free chunks between allocated regions

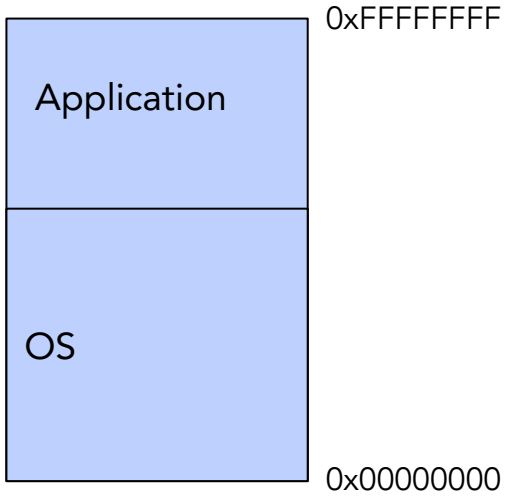


Version 1.0 (“stone age”): Uniprogramming

Version 1.0 (“stone age”): Uniprogramming

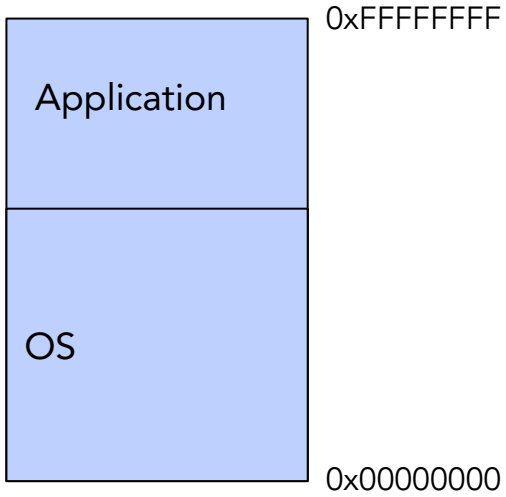


Version 1.0 (“stone age”): Uniprogramming



- Application
  - always loaded into the same place in physical memory
  - can access any physical address

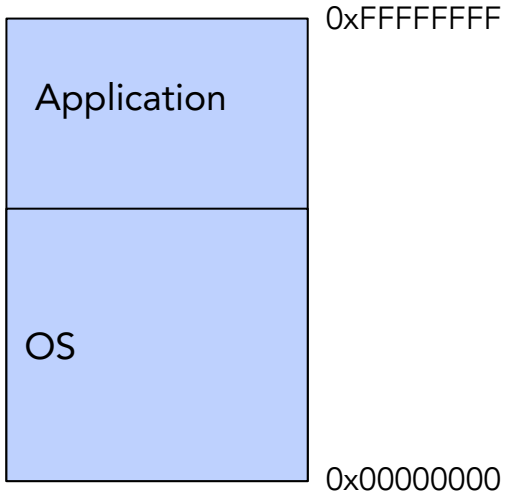
Version 1.0 (“stone age”): Uniprogramming



Isolation	*?*
Sharing	*?*
Virtualization	*?*
Expandable	*?*
Relative addresses	*?*
No internal fragmentation	*?*
No external fragmentation	*?*

- Application
  - always loaded into the same place in physical memory
  - can access any physical address

Version 1.0 (“stone age”): Uniprogramming

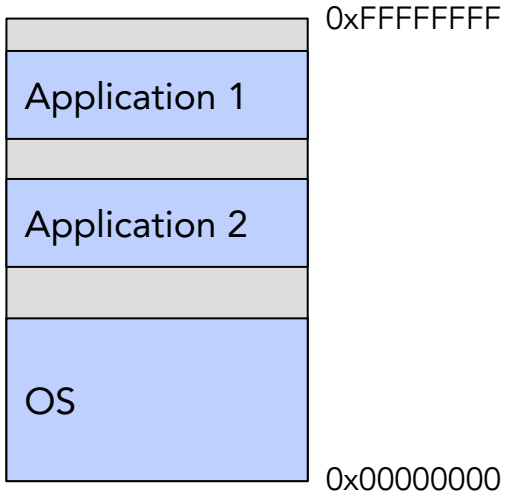


Isolation	✗
Sharing	*N/A*
Virtualization	✗
Expandable	✗
Relative addresses	✗
No internal fragmentation	✗
No external fragmentation	*N/A*

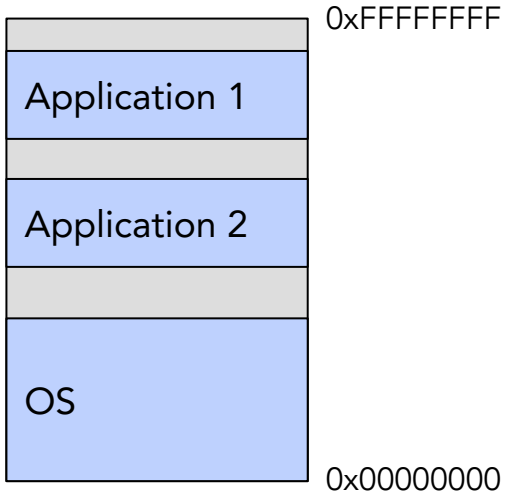
- Application
  - always loaded into the same place in physical memory
  - can access any physical address

Version 1.1 (“iron age”): Multi-programming

Version 1.1 (“iron age”): Multi-programming

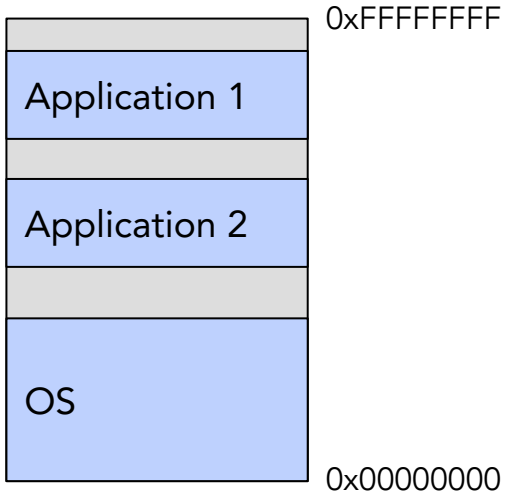


Version 1.1 (“iron age”): Multi-programming



- Loader/linker adjusts addresses when program loaded into memory

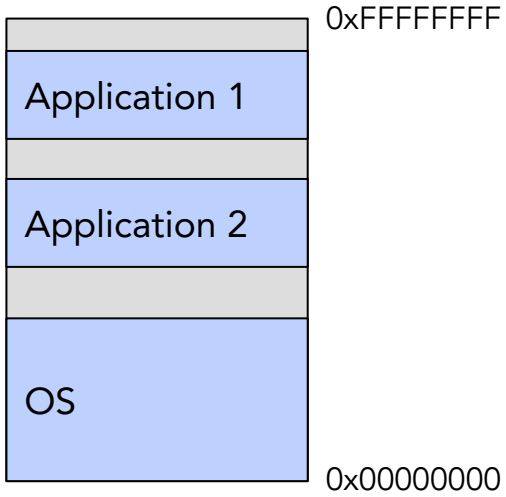
Version 1.1 (“iron age”): Multi-programming



Isolation	*?*
Sharing	*?*
Virtualization	*?*
Expandable	*?*
Relative addresses	*?*
No internal fragmentation	*?*
No external fragmentation	*?*

- Loader/linker adjusts addresses when program loaded into memory

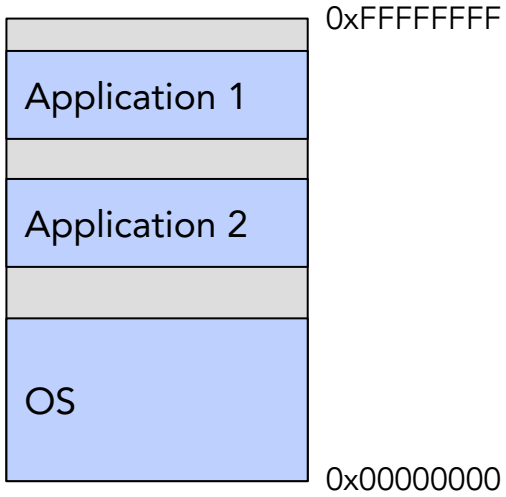
Version 1.1 (“iron age”): Multi-programming



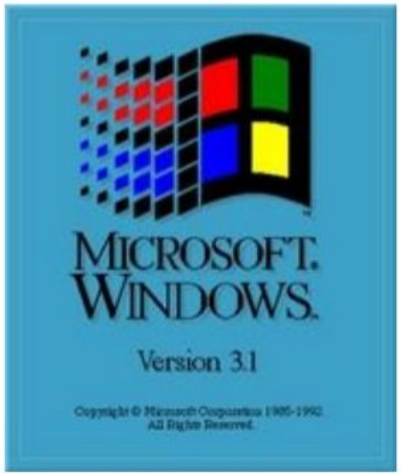
Isolation	✗
Sharing	✔
Virtualization	✗
Expandable	✗
Relative addresses	✗
No internal fragmentation	✗
No external fragmentation	✗

- Loader/linker adjusts addresses when program loaded into memory

Version 1.1 (“iron age”): Multi-programming

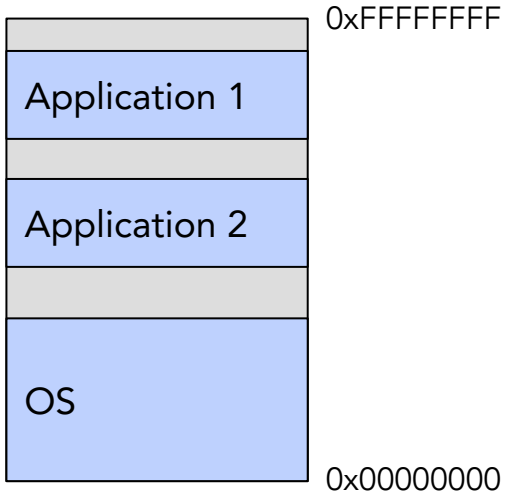


Isolation	✗
Sharing	✔
Virtualization	✗
Expandable	✗
Relative addresses	✗
No internal fragmentation	✗
No external fragmentation	✗

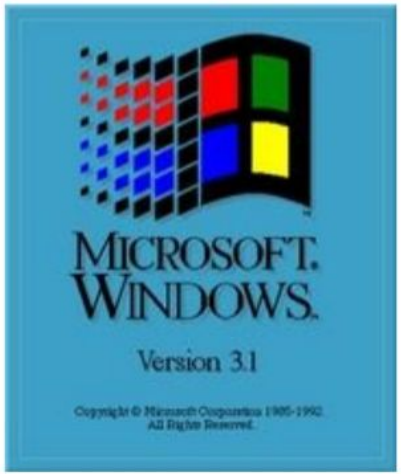


➢ Loader/linker adjusts addresses when program loaded into memory

Version 1.1 (“iron age”): Multi-programming



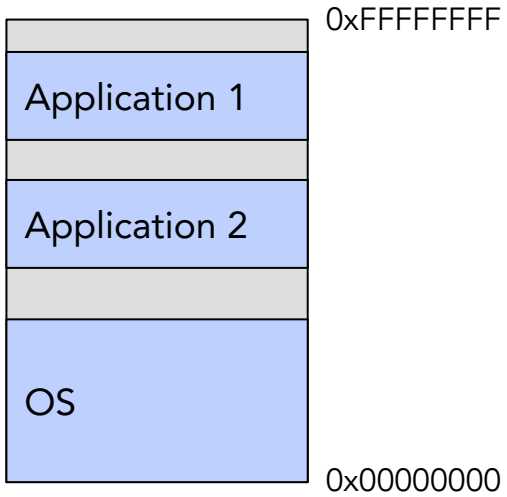
Isolation	✗
Sharing	✔
Virtualization	✗
Expandable	✗
Relative addresses	✗
No internal fragmentation	✗
No external fragmentation	✗



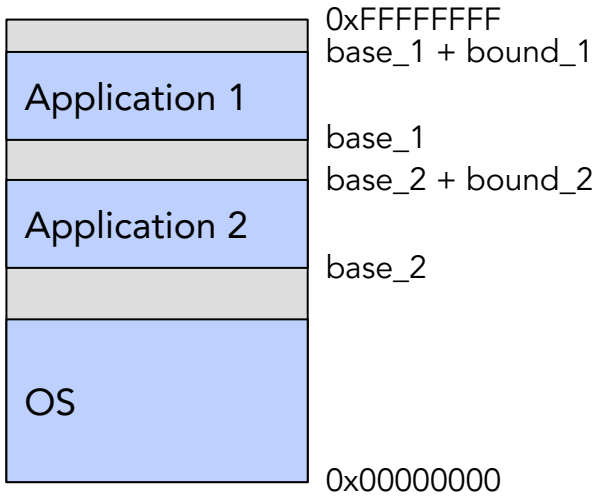
➢ Loader/linker adjusts addresses when program loaded into memory

Version 2.0: Base and bound

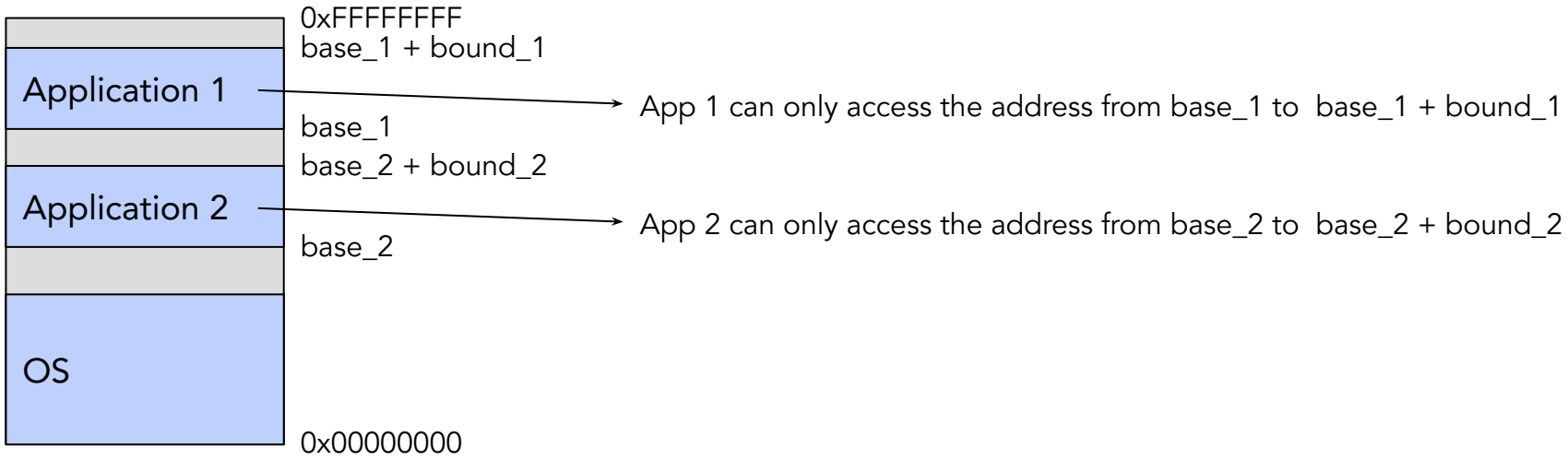
Version 2.0: Base and bound



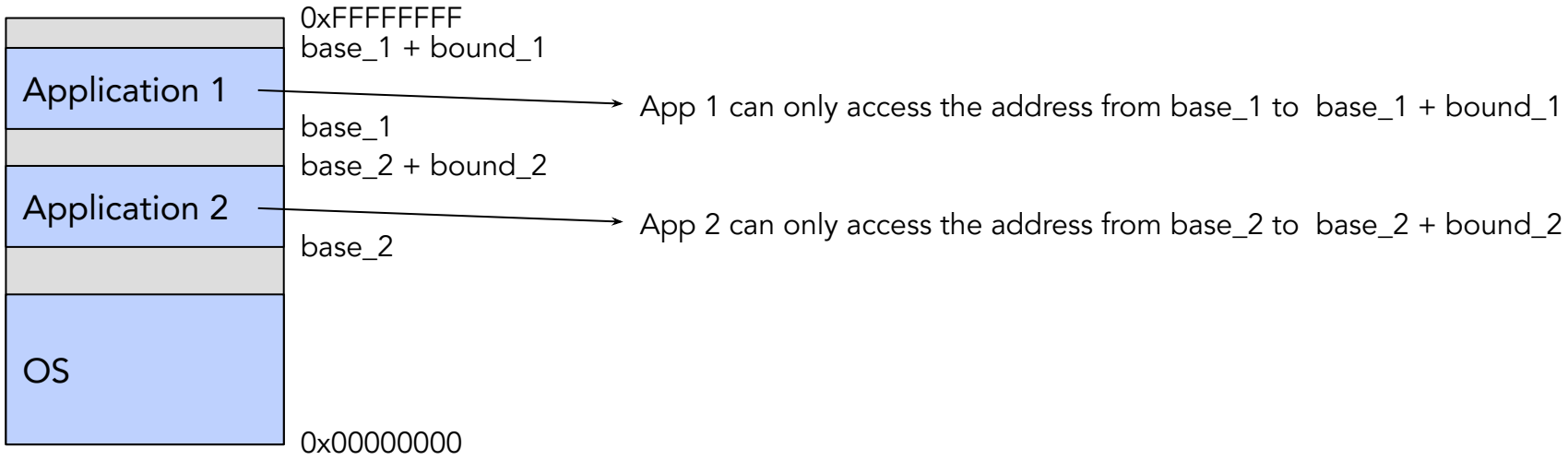
Version 2.0: Base and bound



Version 2.0: Base and bound

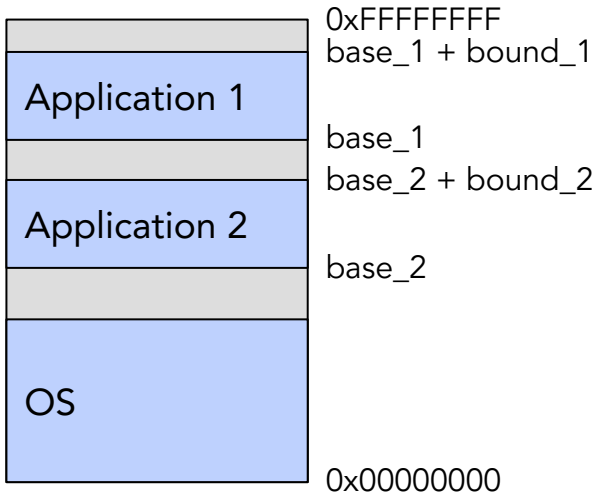


Version 2.0: Base and bound



- The hardware checks if the address is within the range [base, base + bound]

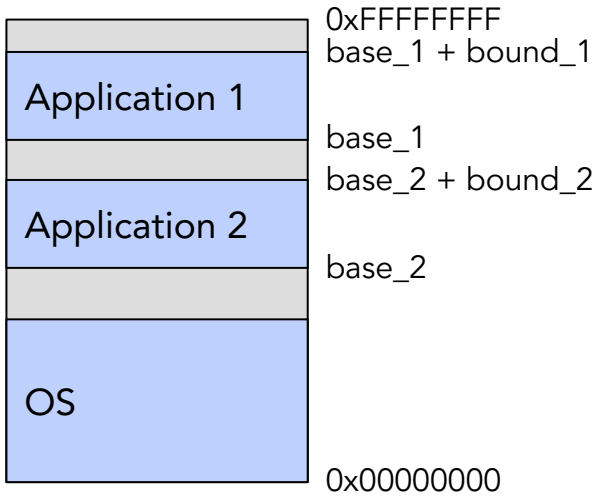
Version 2.0: Base and bound



Isolation	*?*
Sharing	*?*
Virtualization	*?*
Expandable	*?*
Relative addresses	*?*
No internal fragmentation	*?*
No external fragmentation	*?*

➢ The hardware checks if the address is within the range [base, base + bound]

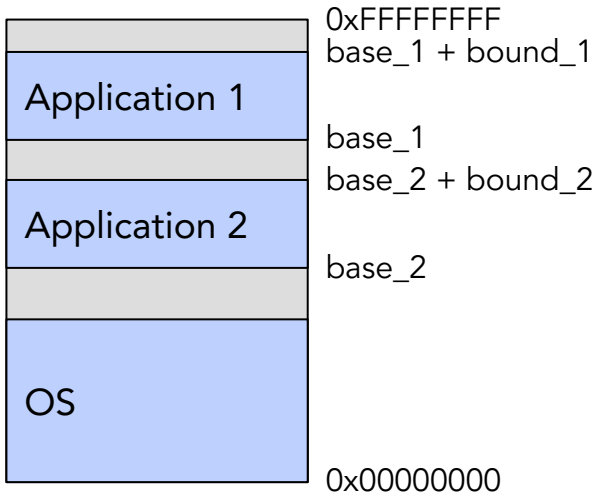
Version 2.0: Base and bound



Isolation	✔
Sharing	✗
Virtualization	✗
Expandable	✗
Relative addresses	✗
No internal fragmentation	✗
No external fragmentation	✗

➢ The hardware checks if the address is within the range [base, base + bound]

Version 2.0: Base and bound

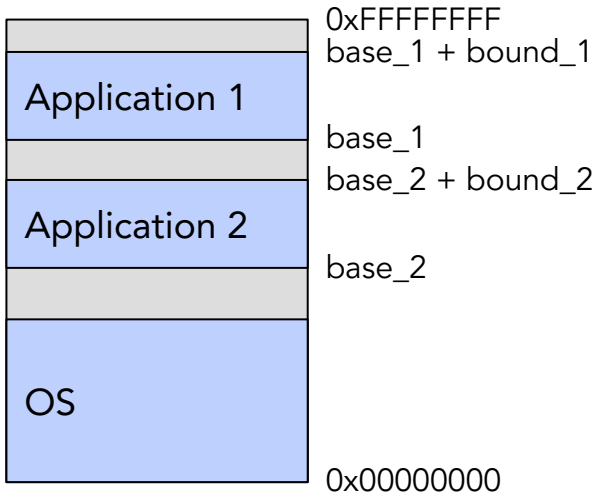


Isolation	✔
Sharing	✗
Virtualization	✗
Expandable	✗
Relative addresses	✗
No internal fragmentation	✗
No external fragmentation	✗

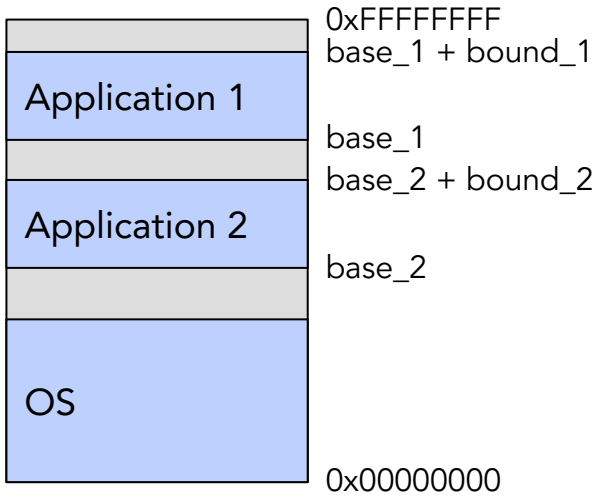
➢ The hardware checks if the address is within the range [base, base + bound]

Version 2.1: Base and bound + relocation

Version 2.1: Base and bound + relocation

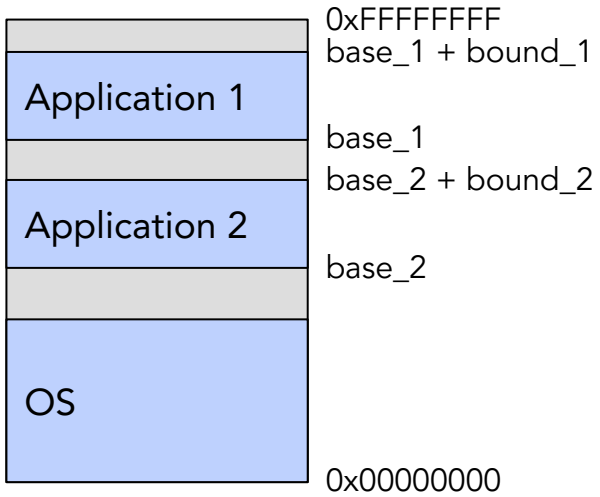


Version 2.1: Base and bound + relocation



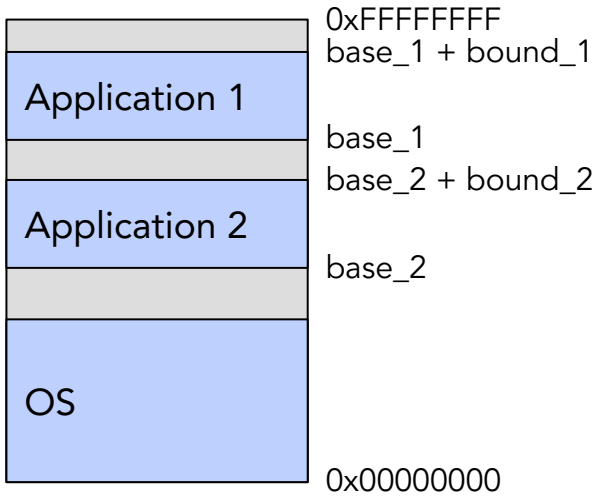
- Each program is written and compiled as if it is loaded at address zero

Version 2.1: Base and bound + relocation



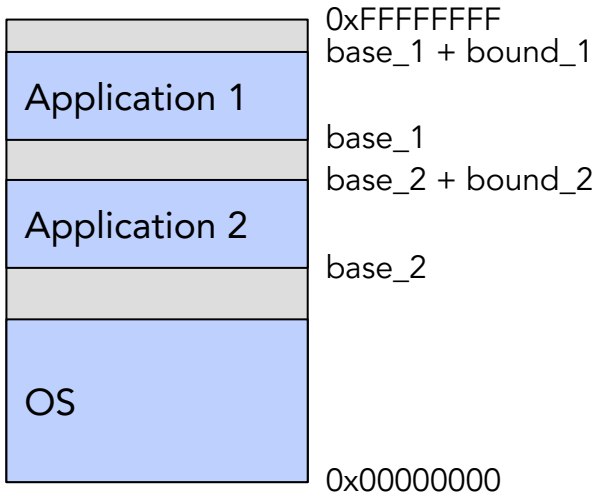
- Each program is written and compiled as if it is loaded at address zero
- For a memory reference, the hardware

Version 2.1: Base and bound + relocation



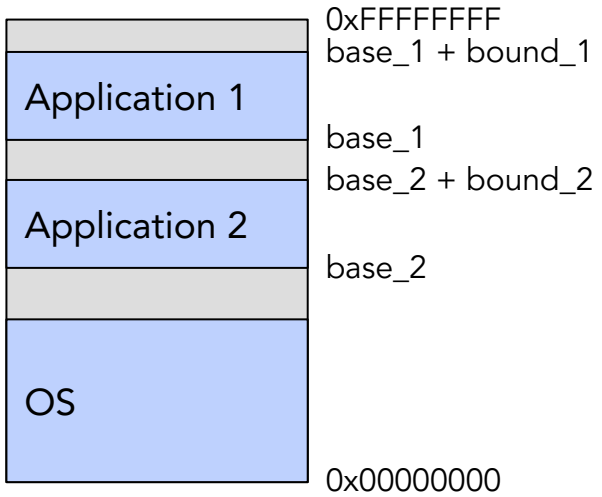
- Each program is written and compiled as if it is loaded at address zero
- For a memory reference, the hardware
  - checks if the virtual address (i.e. address in compiled code) ≤ bound

Version 2.1: Base and bound + relocation



- Each program is written and compiled as if it is loaded at address zero
- For a memory reference, the hardware
  - checks if the virtual address (i.e. address in compiled code) ≤ bound
  - then translates the virtual address to a physical address = virtual address + base

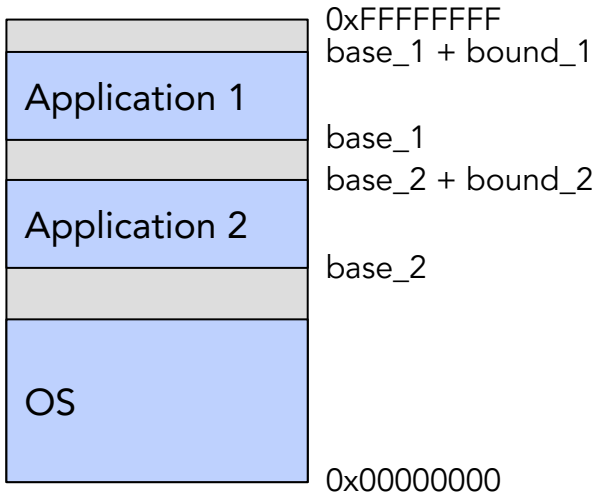
Version 2.1: Base and bound + relocation



Isolation	*?*
Sharing	*?*
Virtualization	*?*
Expandable	*?*
Relative addresses	*?*
No internal fragmentation	*?*
No external fragmentation	*?*

- Each program is written and compiled as if it is loaded at address zero
- For a memory reference, the hardware
  - checks if the virtual address (i.e. address in compiled code) ≤ bound
  - then translates the virtual address to a physical address = virtual address + base

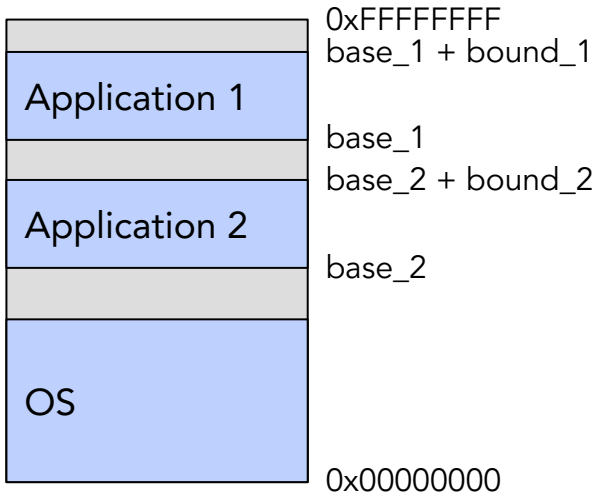
Version 2.1: Base and bound + relocation



Isolation	✔
Sharing	✗
Virtualization	✗
Expandable	✗
Relative addresses	✔
No internal fragmentation	✗
No external fragmentation	✗

- Each program is written and compiled as if it is loaded at address zero
- For a memory reference, the hardware
  - checks if the virtual address (i.e. address in compiled code) ≤ bound
  - then translates the virtual address to a physical address = virtual address + base

Version 2.1: Base and bound + relocation

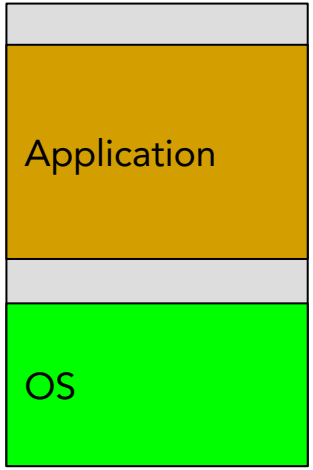


Isolation	✔
Sharing	✗
Virtualization	✗
Expandable	✗
Relative addresses	✔
No internal fragmentation	✗
No external fragmentation	✗

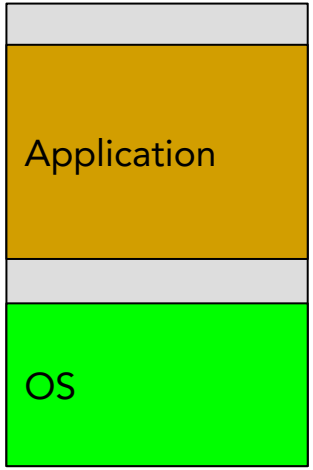
- Each program is written and compiled as if it is loaded at address zero
- For a memory reference, the hardware
  - checks if the virtual address (i.e. address in compiled code) ≤ bound
  - then translates the virtual address to a physical address = virtual address + base

Version 3.0: Segmentation

Version 3.0: Segmentation

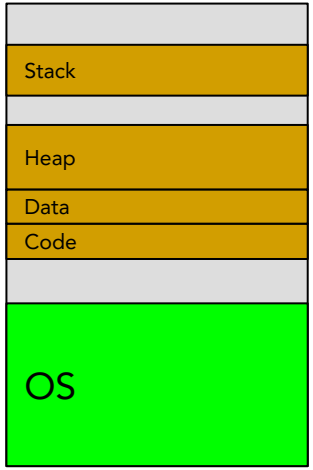


Version 3.0: Segmentation



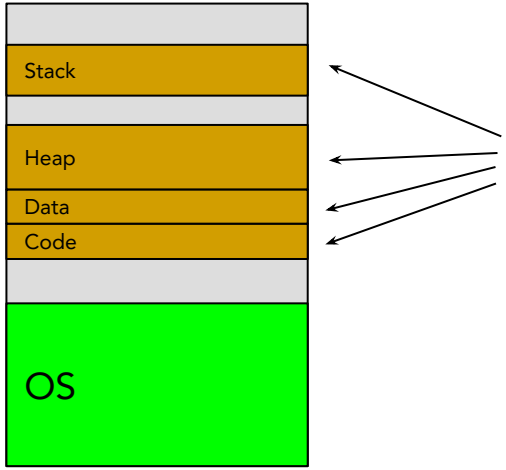
Virtual address space

Version 3.0: Segmentation



Virtual address space

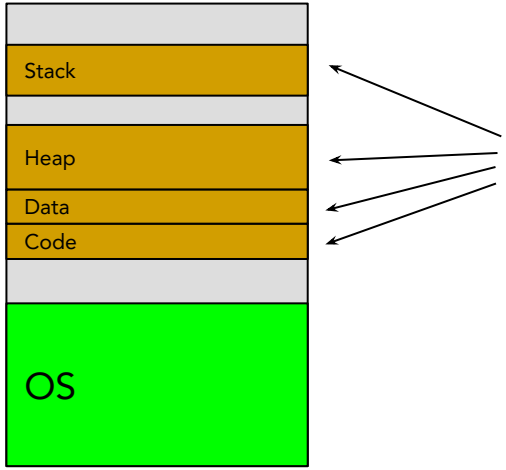
Version 3.0: Segmentation



Segment: a contiguous portion of the address space of a particular length

Virtual address space

Version 3.0: Segmentation

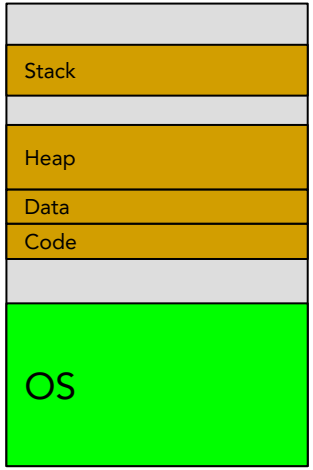


Virtual address space

Segment: a contiguous portion of the address space of a particular length

A program = a set of segments

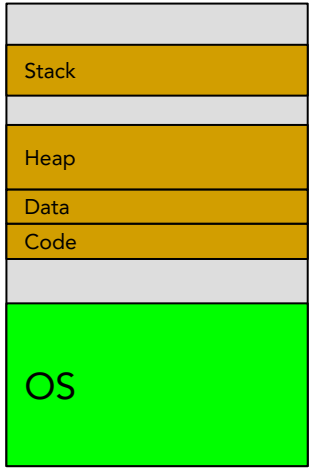
Version 3.0: Segmentation



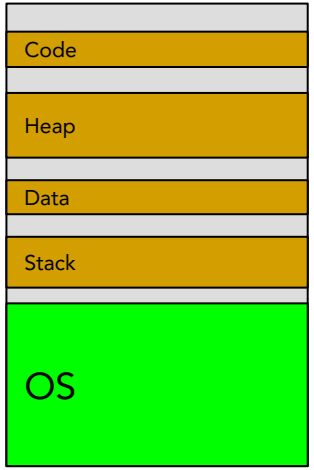
Virtual address space

Physical address space

Version 3.0: Segmentation

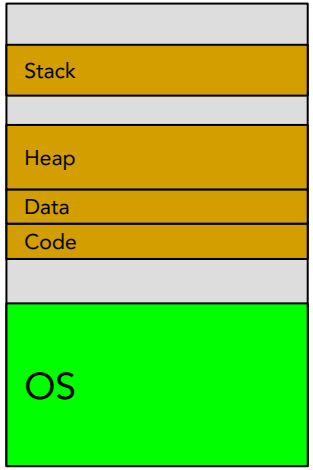


Virtual address space



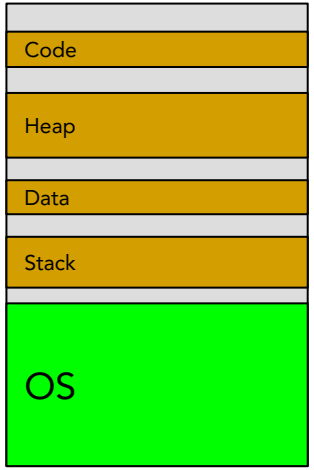
Physical address space

Version 3.0: Segmentation



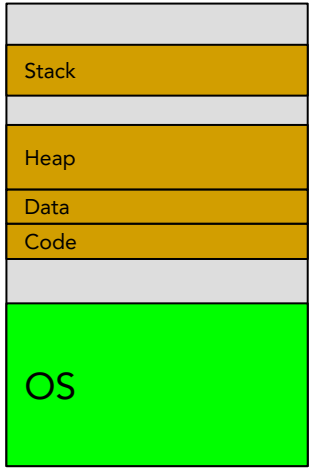
Virtual address space

How to translate virtual addresses →physical addresses?



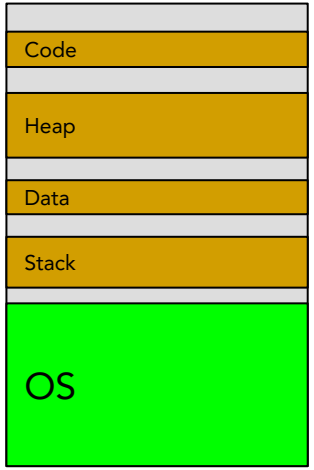
Physical address space

Version 3.0: Segmentation



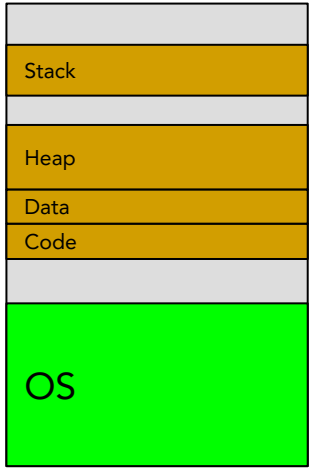
Virtual address space

Segment map



Physical address space

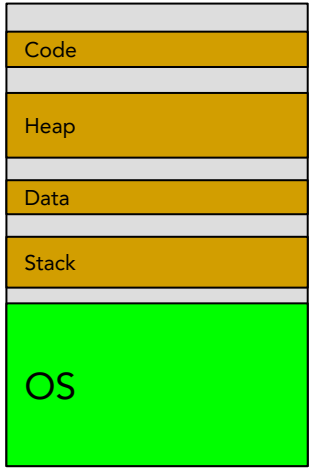
Version 3.0: Segmentation



Virtual address space

Seg#	Base	Limit
0	Base0	Limit0
1	Base1	Limit1
2	Base2	Limit2
3	Base3	Limit3
4	Base4	Limit4
5	Base5	Limit5

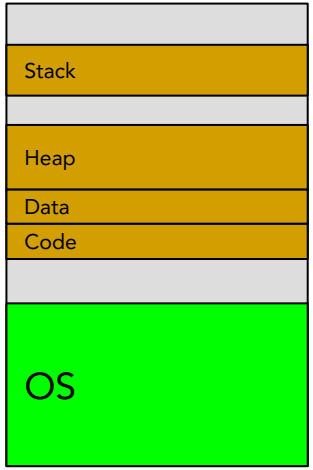
Segment map



Physical address space

Version 3.0: Segmentation

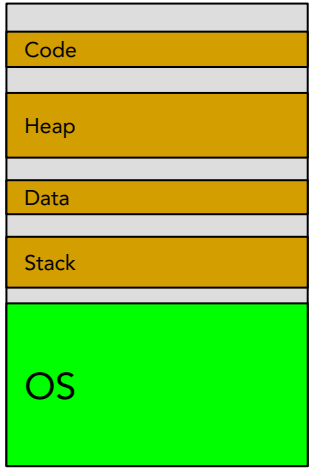
Virtual address



Virtual address space

Seg#	Base	Limit
0	Base0	Limit0
1	Base1	Limit1
2	Base2	Limit2
3	Base3	Limit3
4	Base4	Limit4
5	Base5	Limit5

Segment map

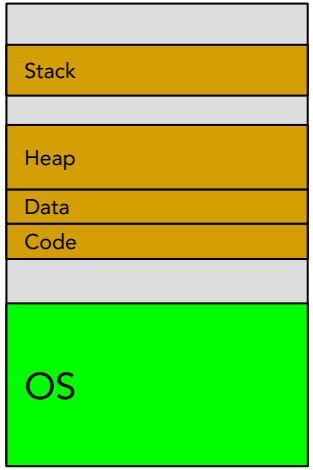


Physical address space

Version 3.0: Segmentation

Virtual address

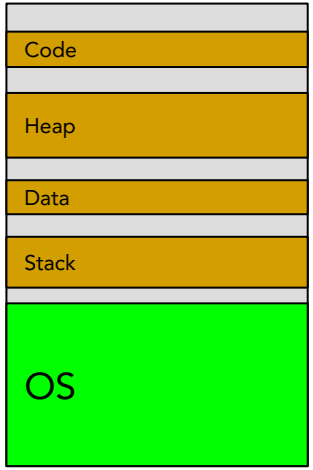
seg#	offset



Virtual address space

Seg#	Base	Limit
0	Base0	Limit0
1	Base1	Limit1
2	Base2	Limit2
3	Base3	Limit3
4	Base4	Limit4
5	Base5	Limit5

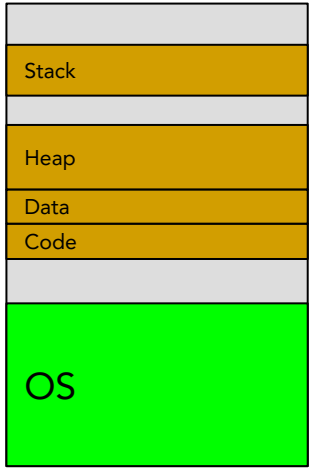
Segment map



Physical address space

Version 3.0: Segmentation

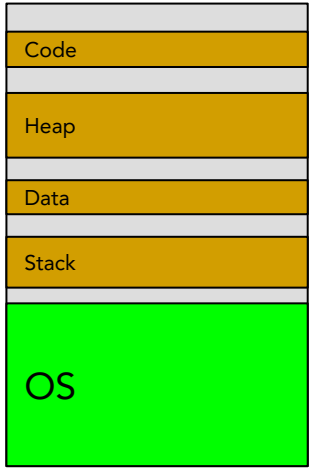
Virtual address



Virtual address space

Seg#	Base	Limit
0	Base0	Limit0
1	Base1	Limit1
2	Base2	Limit2
3	Base3	Limit3
4	Base4	Limit4
5	Base5	Limit5

Segment map

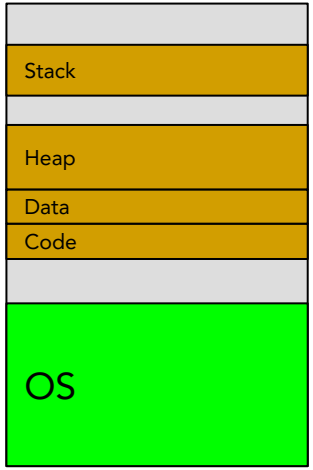


Physical address space

Version 3.0: Segmentation

Virtual address

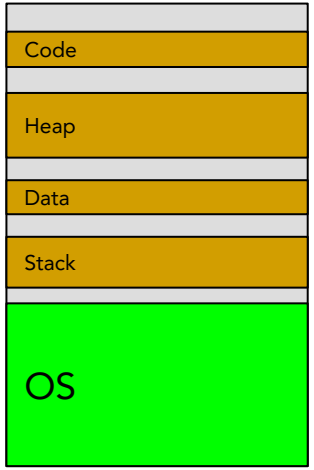
seg#	offset



Virtual address space

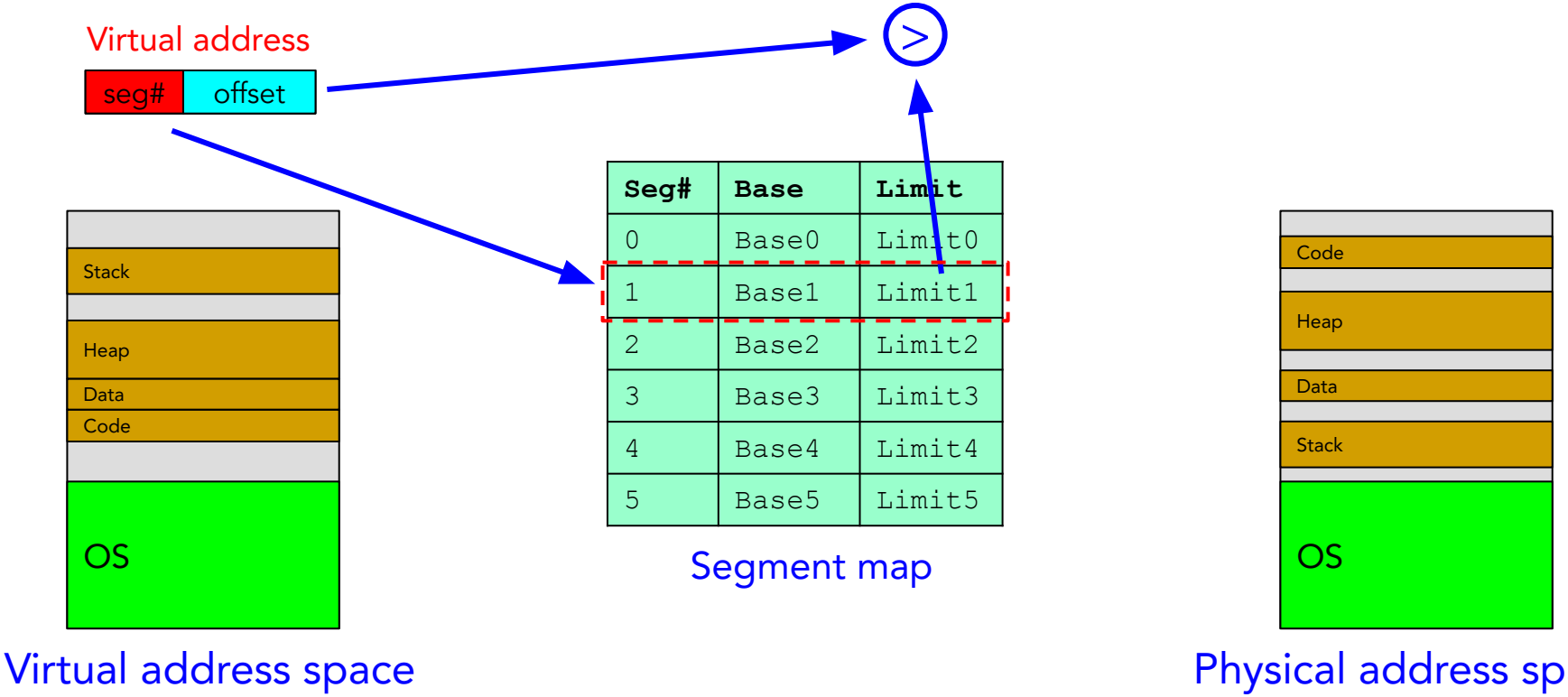
Seg#	Base	Limit
0	Base0	Limit0
1	Base1	Limit1
2	Base2	Limit2
3	Base3	Limit3
4	Base4	Limit4
5	Base5	Limit5

Segment map



Physical address space

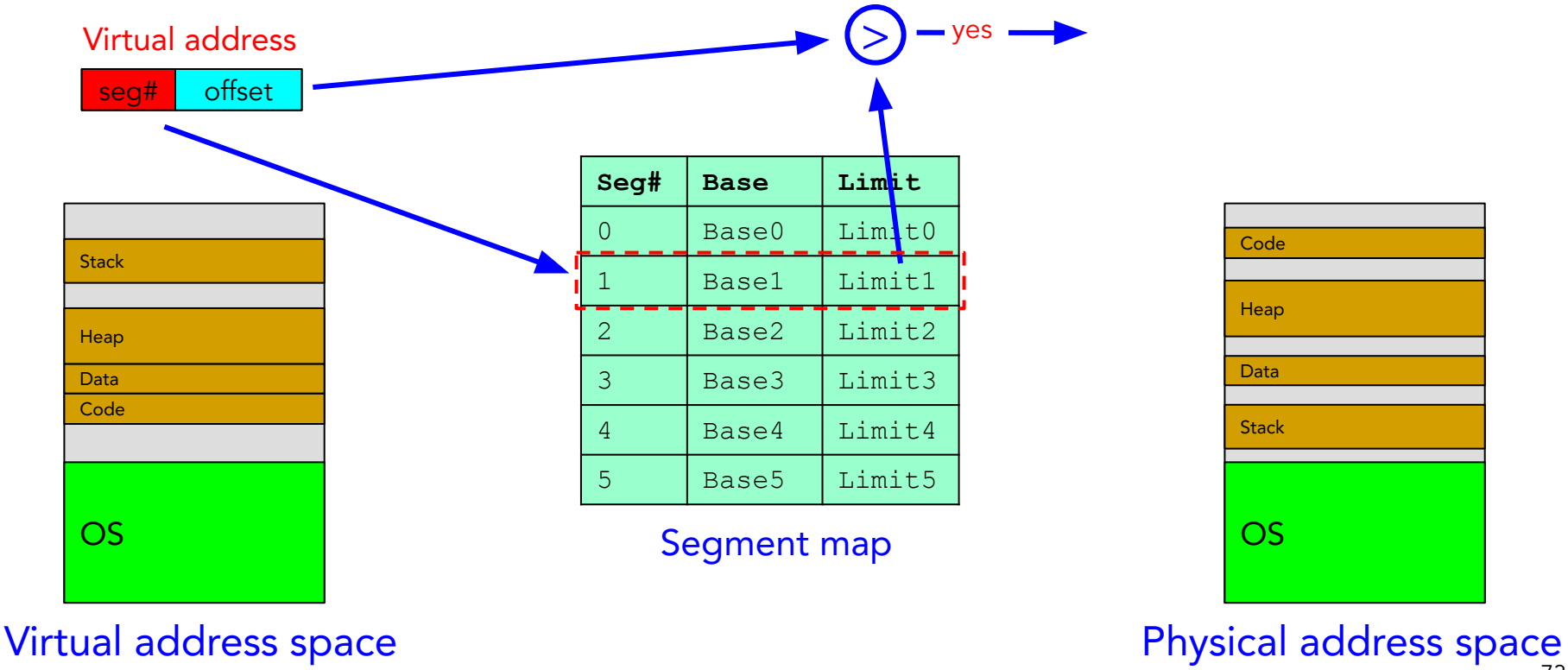
Version 3.0: Segmentation



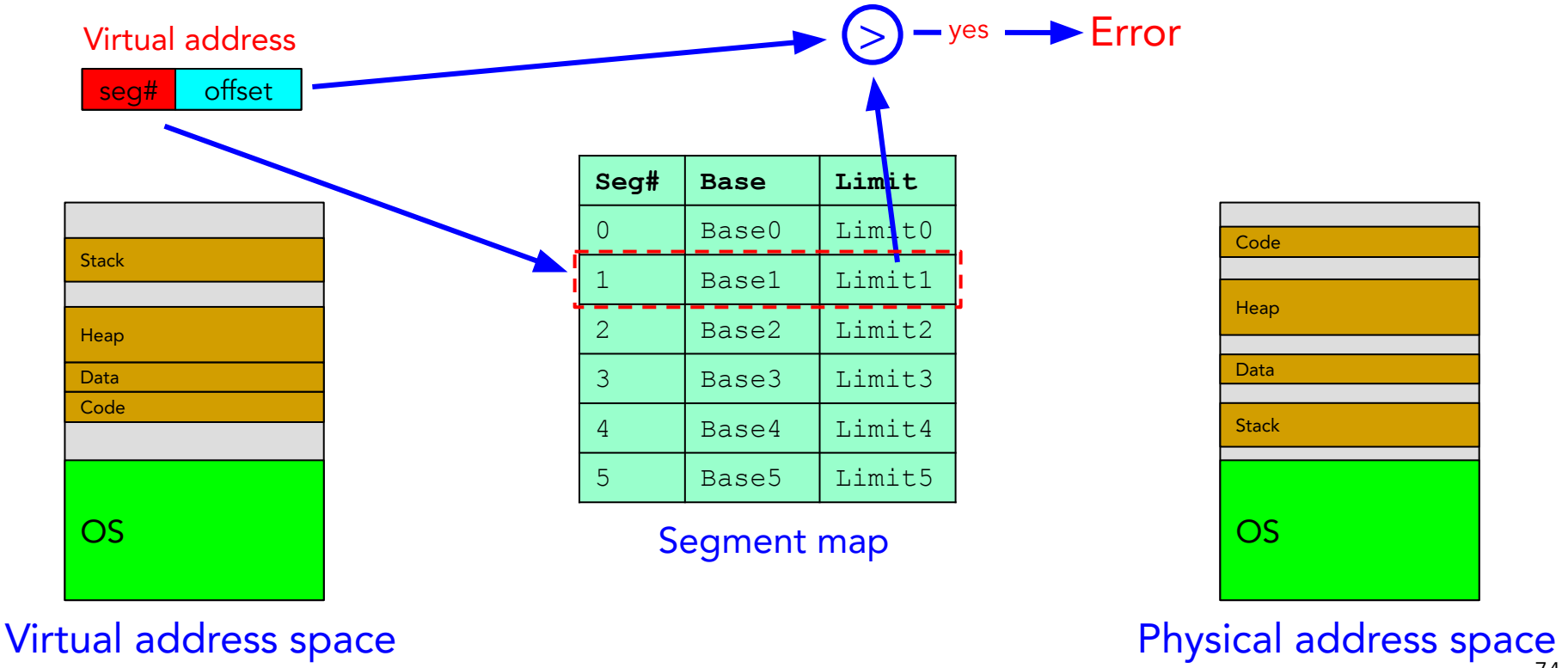
Seg#	Base	Limit
0	Base0	Limit0
1	Base1	Limit1
2	Base2	Limit2
3	Base3	Limit3
4	Base4	Limit4
5	Base5	Limit5

Segment map

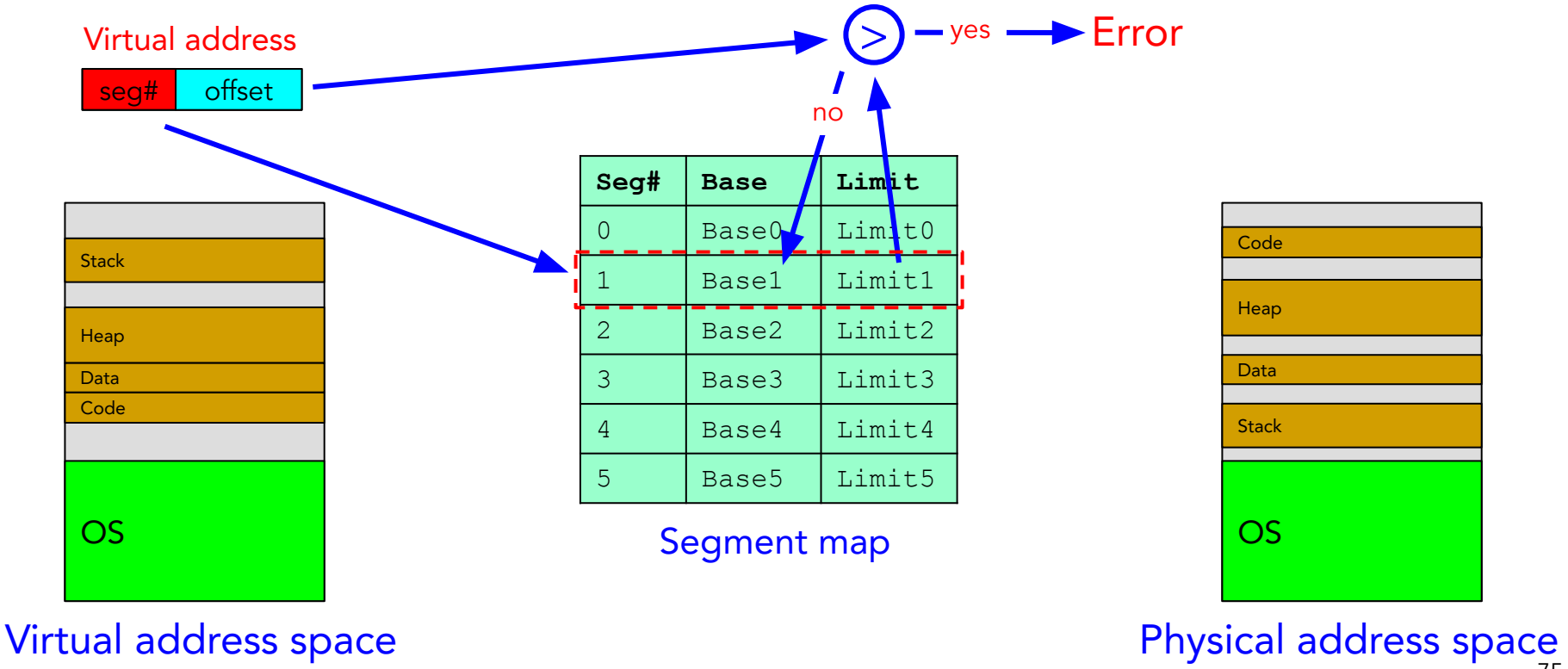
Version 3.0: Segmentation



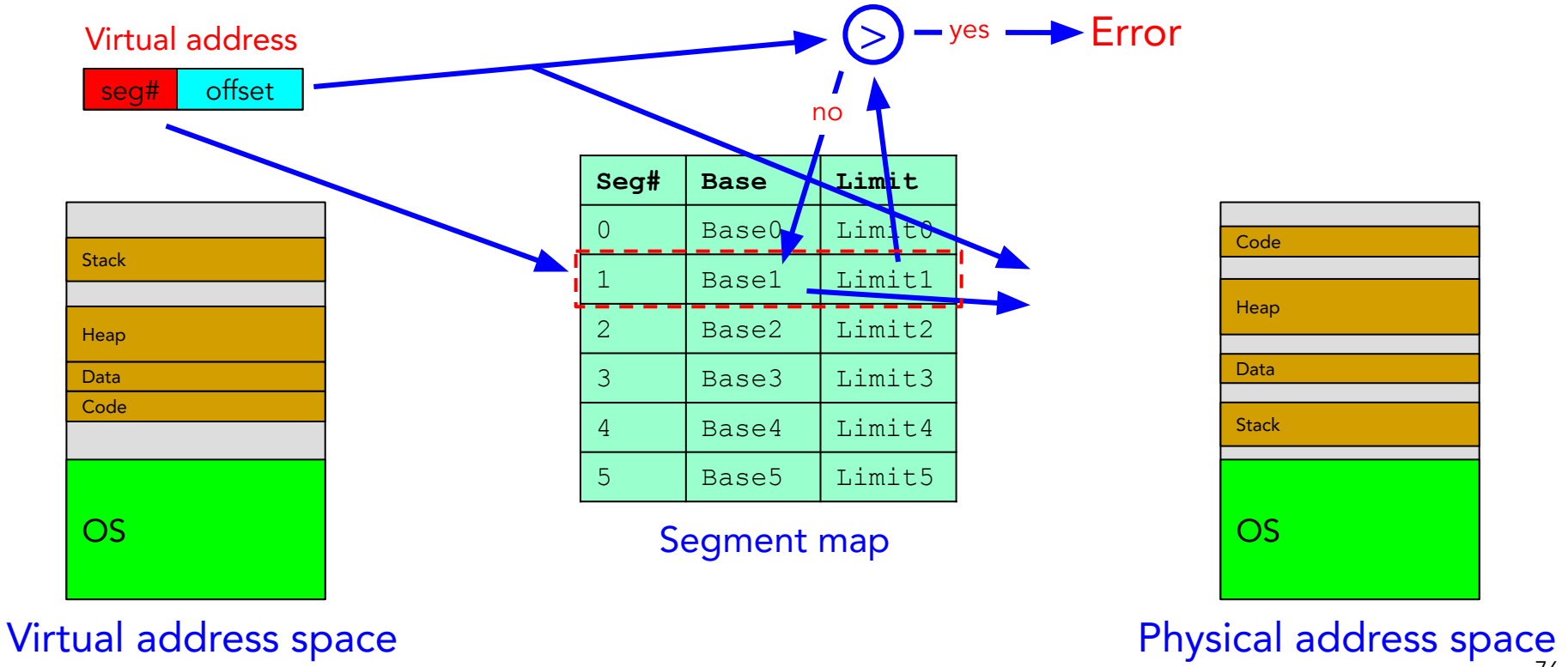
Version 3.0: Segmentation



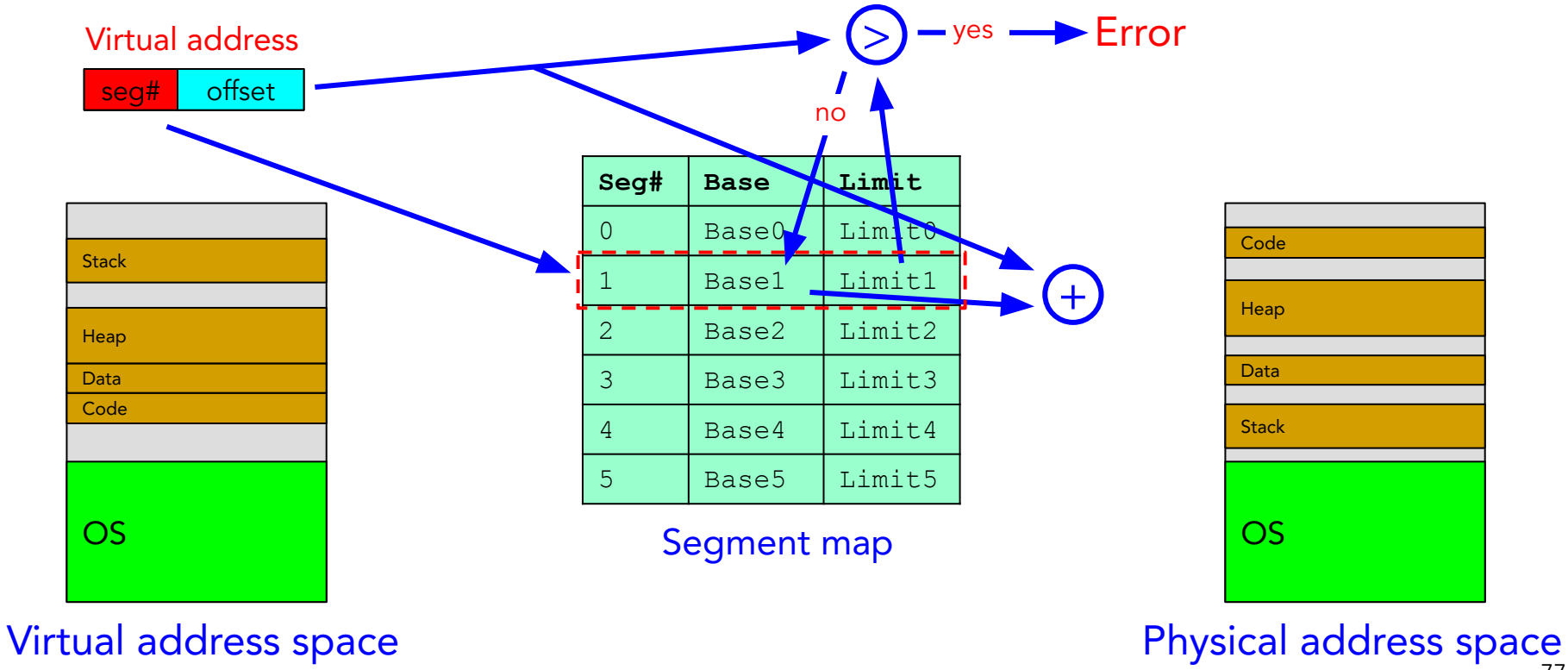
Version 3.0: Segmentation



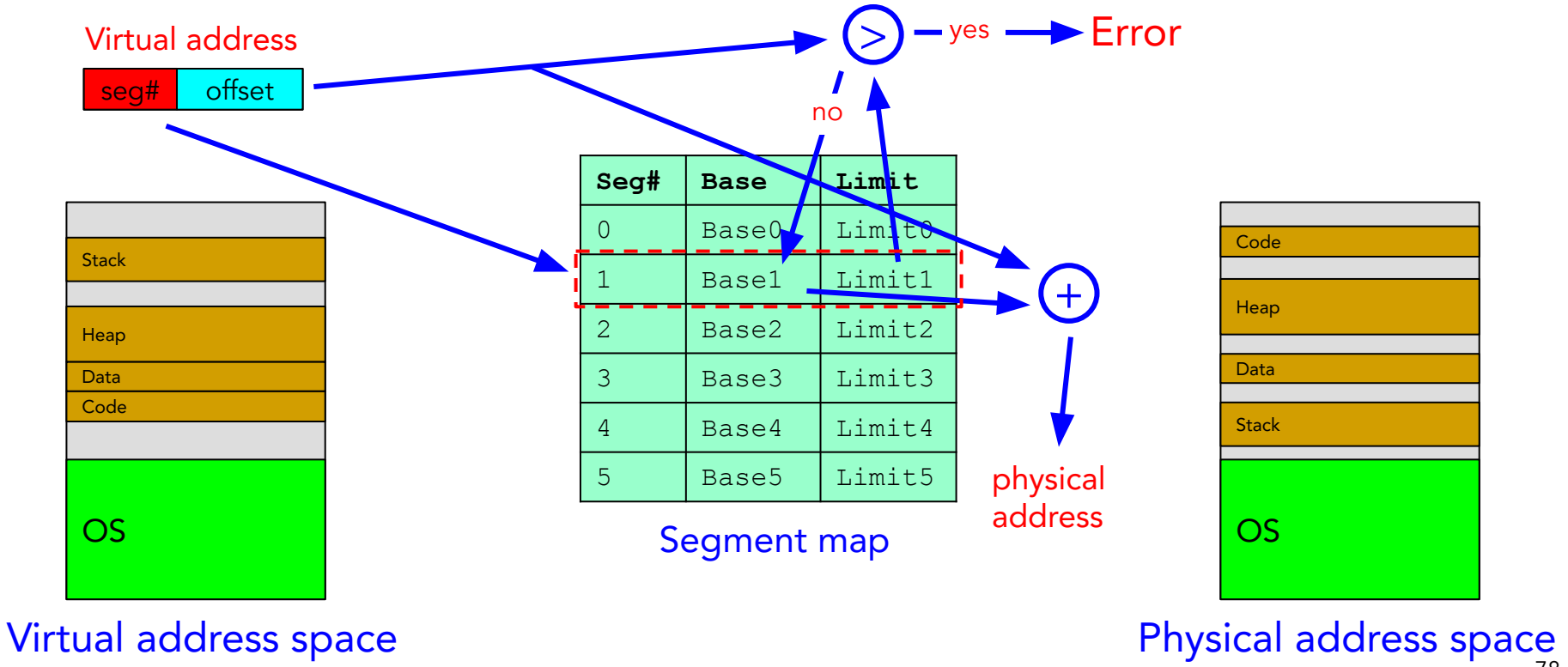
Version 3.0: Segmentation



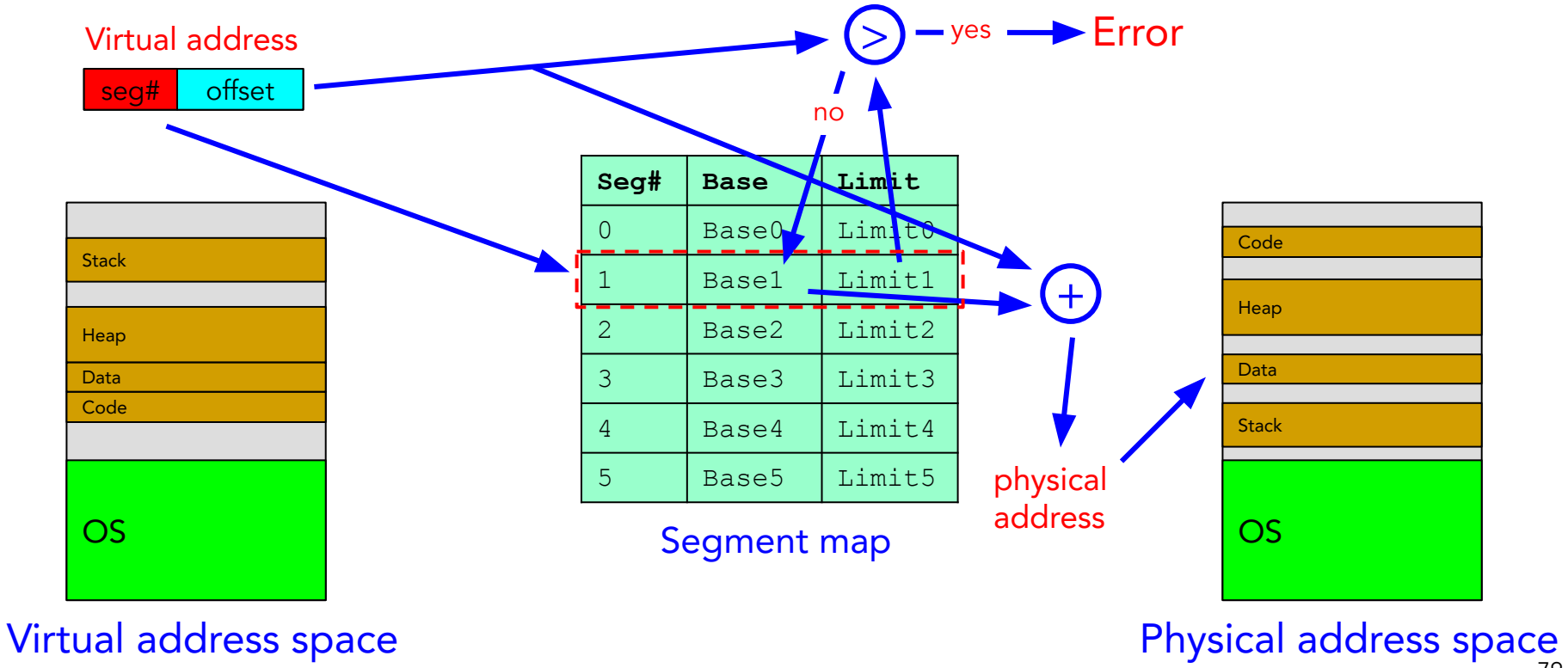
Version 3.0: Segmentation



Version 3.0: Segmentation



Version 3.0: Segmentation



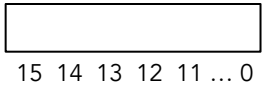
Version 3.0: Example

Version 3.0: Example

16-bit address

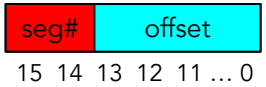
Version 3.0: Example

16-bit address



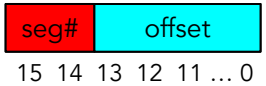
Version 3.0: Example

16-bit address



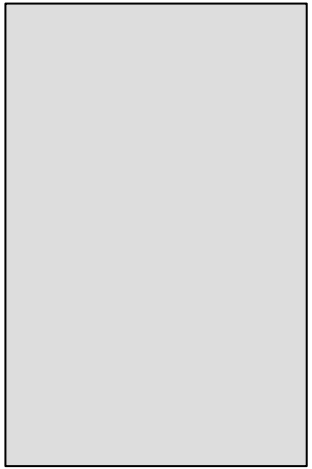
Version 3.0: Example

16-bit address



Seg#	Base	Limit
0	0x4000	0x0800
1	0x4800	0x1400
2	0xF000	0x1000
3	0x0000	0x3000

Segment map

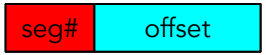


Virtual address space

Physical address space

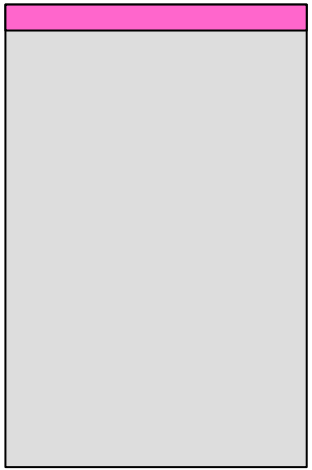
Version 3.0: Example

16-bit address



15 14 13 12 11 … 0

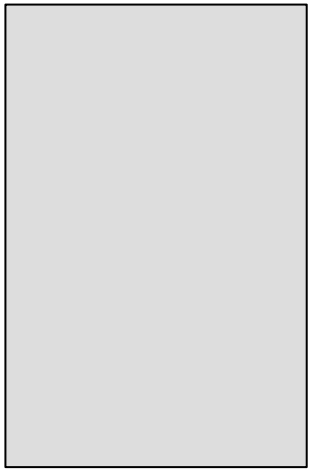
0x0000



Virtual address space

Seg#	Base	Limit
0	0x4000	0x0800
1	0x4800	0x1400
2	0xF000	0x1000
3	0x0000	0x3000

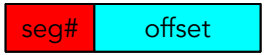
Segment map

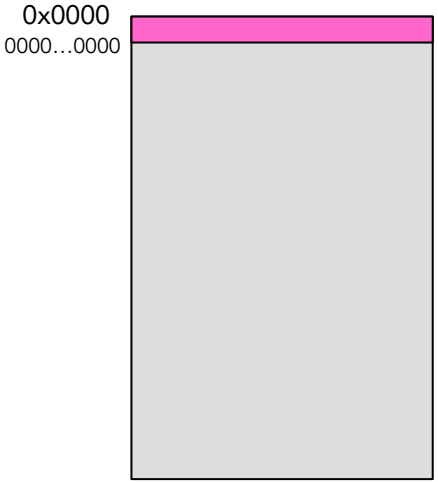


Physical address space

Version 3.0: Example

16-bit address

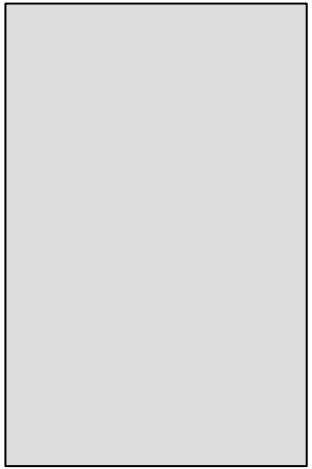




Virtual address space

Seg#	Base	Limit
0	0x4000	0x0800
1	0x4800	0x1400
2	0xF000	0x1000
3	0x0000	0x3000

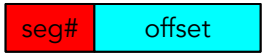
Segment map



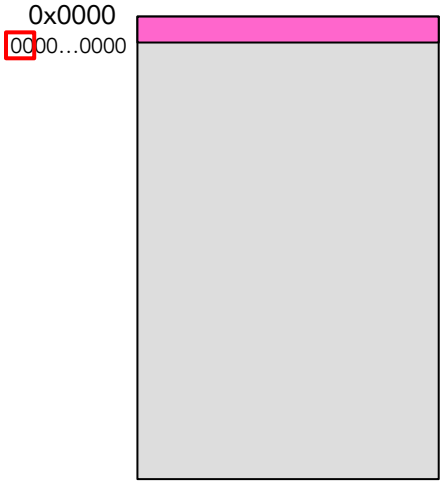
Physical address space

Version 3.0: Example

16-bit address



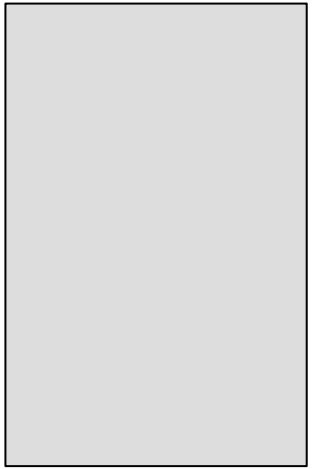
15 14 13 12 11 … 0



Virtual address space

Seg#	Base	Limit
0	0x4000	0x0800
1	0x4800	0x1400
2	0xF000	0x1000
3	0x0000	0x3000

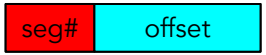
Segment map



Physical address space

Version 3.0: Example

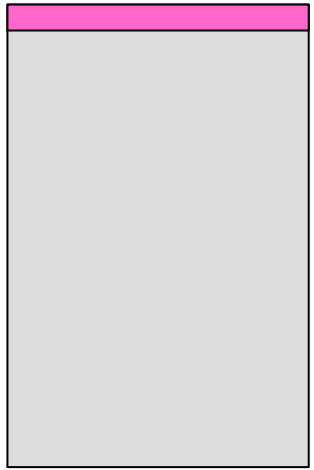
16-bit address



15 14 13 12 11 … 0

0x0000

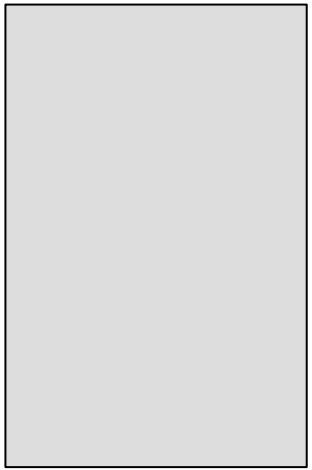
0000…0000



Seg#	Base	Limit
0	0x4000	0x0800
1	0x4800	0x1400
2	0xF000	0x1000
3	0x0000	0x3000

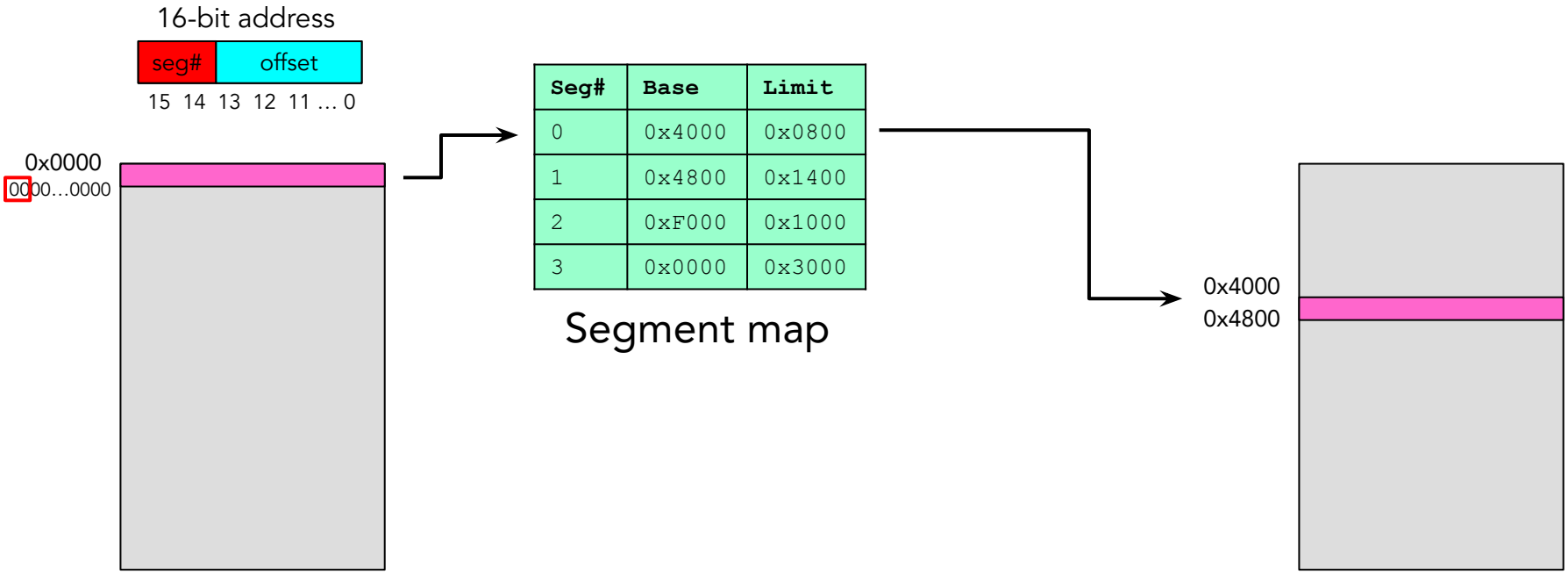
Segment map

Virtual address space



Physical address space

Version 3.0: Example



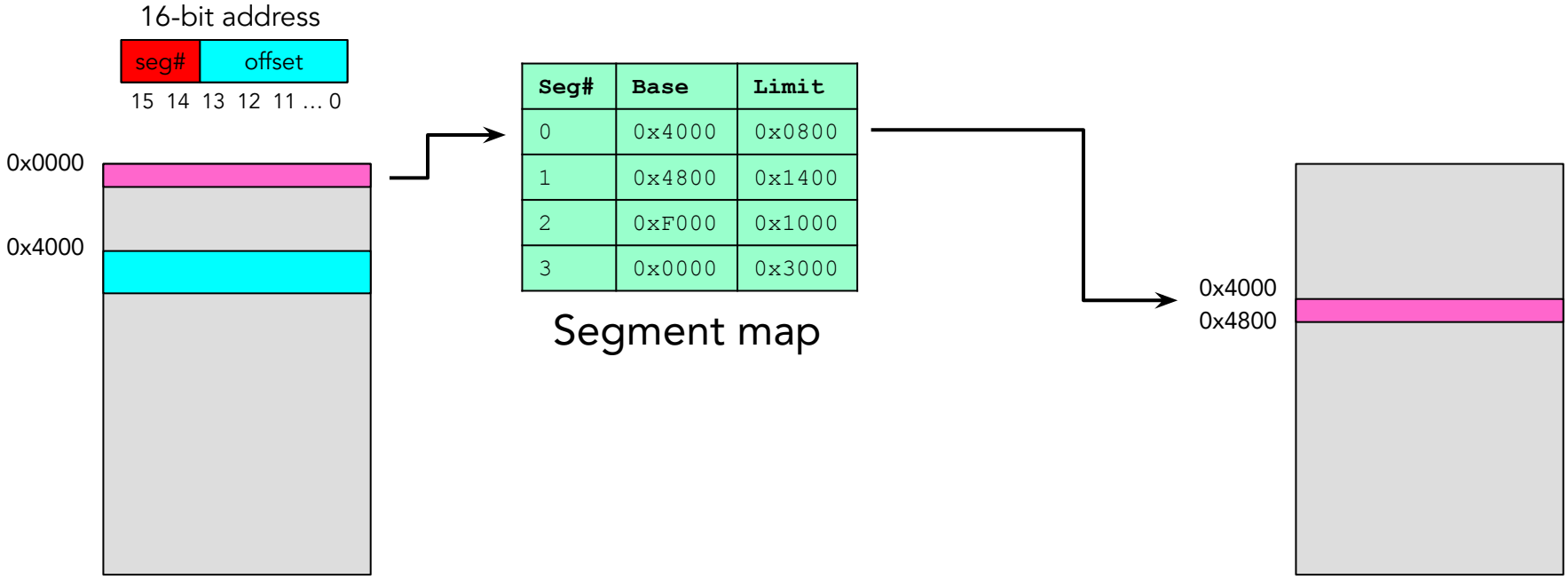
Seg#	Base	Limit
0	0x4000	0x0800
1	0x4800	0x1400
2	0xF000	0x1000
3	0x0000	0x3000

Segment map

Virtual address space

Physical address space

Version 3.0: Example



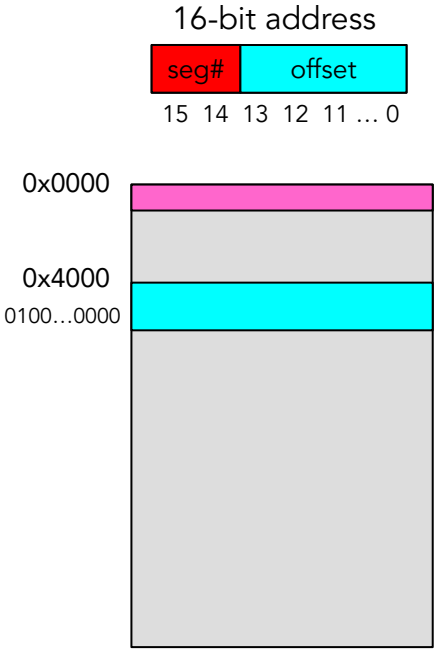
Seg#	Base	Limit
0	0x4000	0x0800
1	0x4800	0x1400
2	0xF000	0x1000
3	0x0000	0x3000

Segment map

Virtual address space

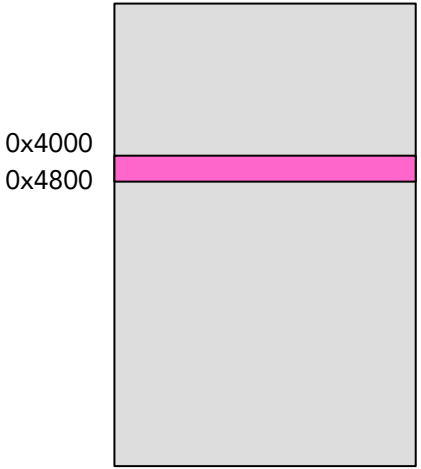
Physical address space

Version 3.0: Example



Seg#	Base	Limit
0	0x4000	0x0800
1	0x4800	0x1400
2	0xF000	0x1000
3	0x0000	0x3000

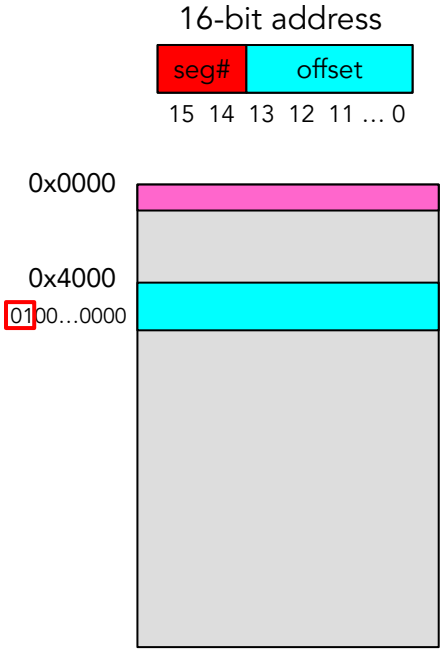
Segment map



Virtual address space

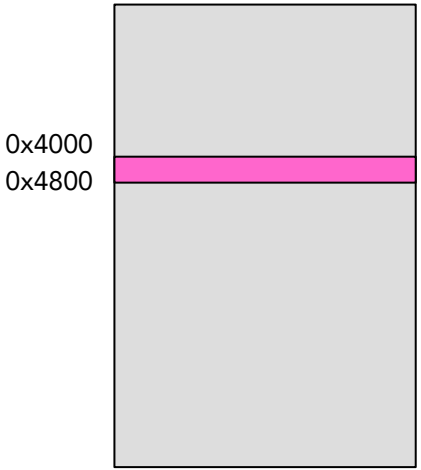
Physical address space

Version 3.0: Example



Seg#	Base	Limit
0	0x4000	0x0800
1	0x4800	0x1400
2	0xF000	0x1000
3	0x0000	0x3000

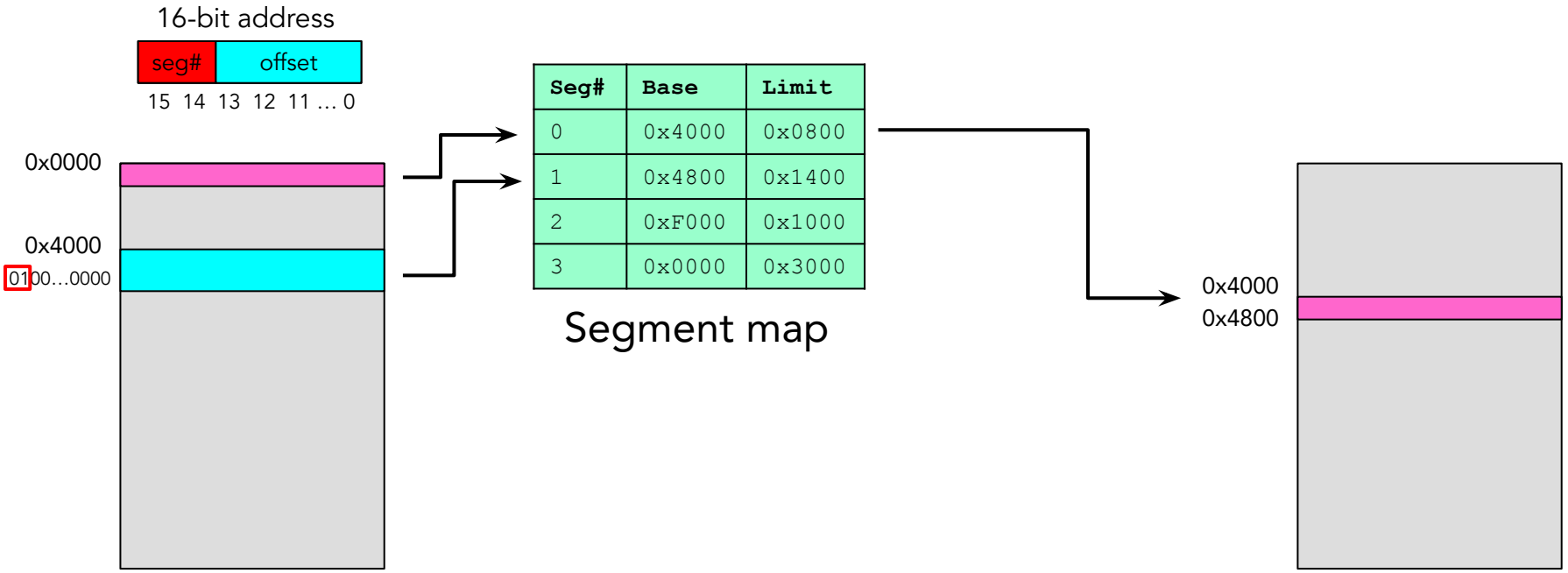
Segment map



Virtual address space

Physical address space

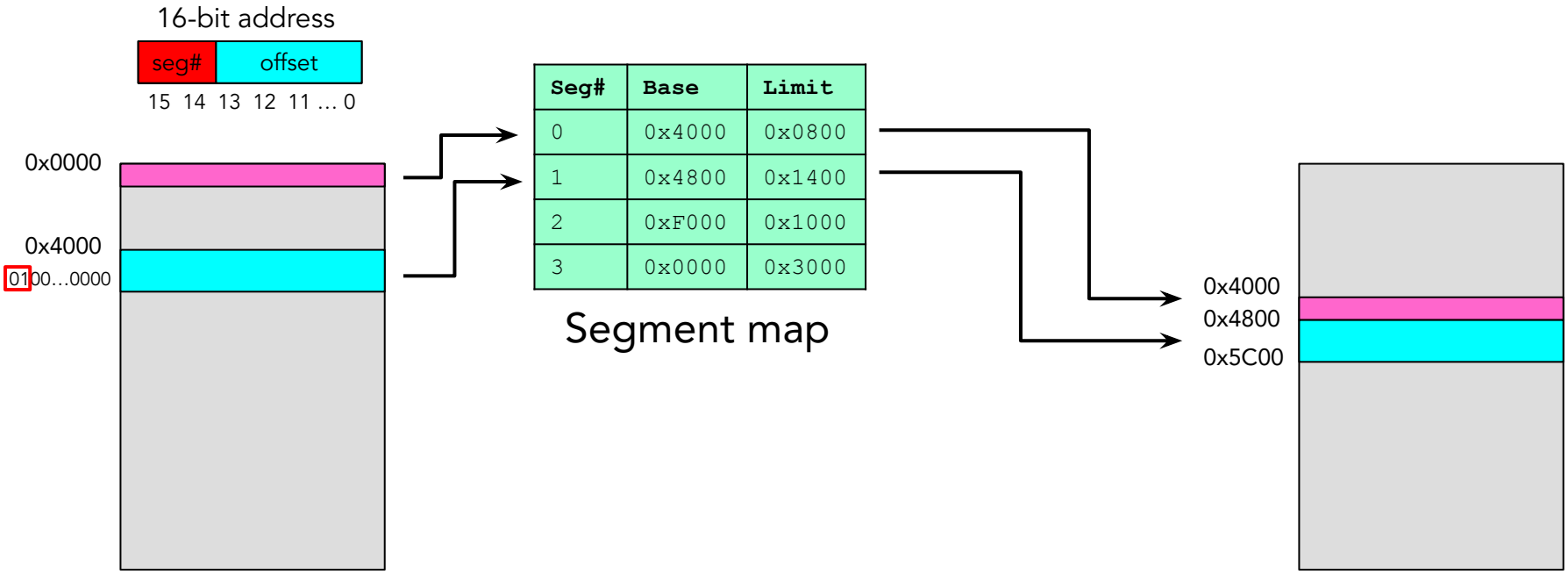
Version 3.0: Example



Virtual address space

Physical address space

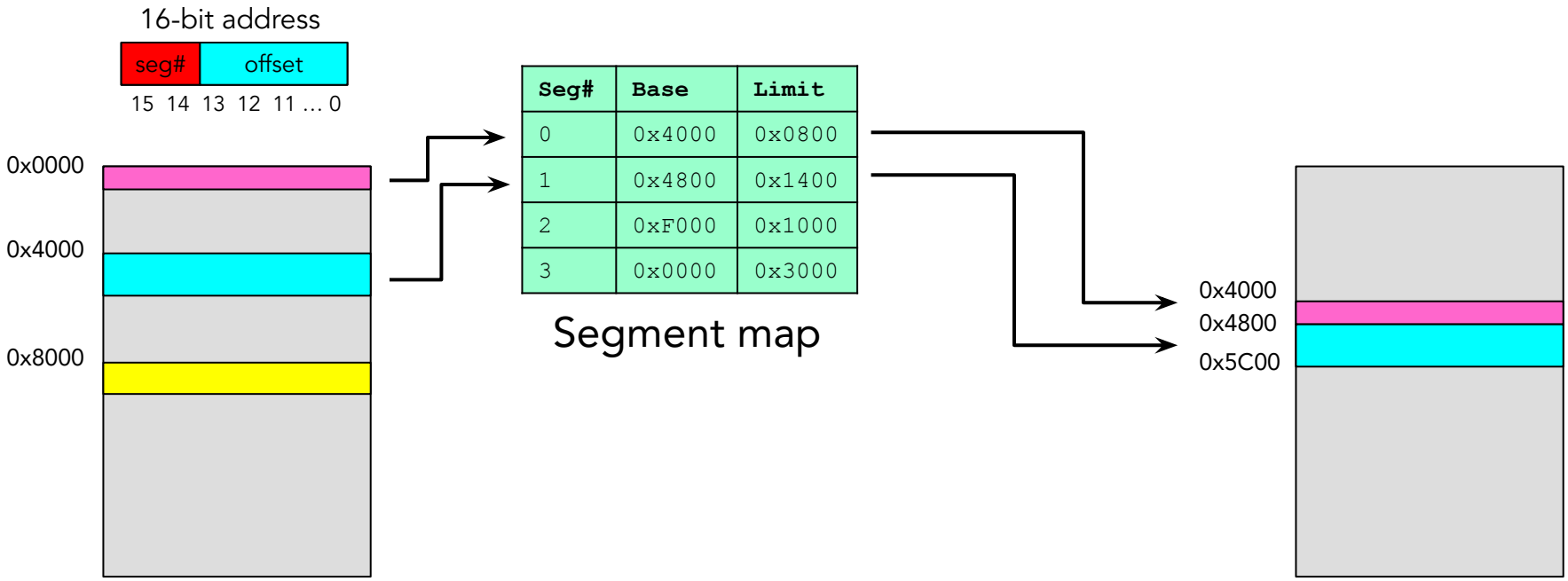
Version 3.0: Example



Virtual address space

Physical address space

Version 3.0: Example



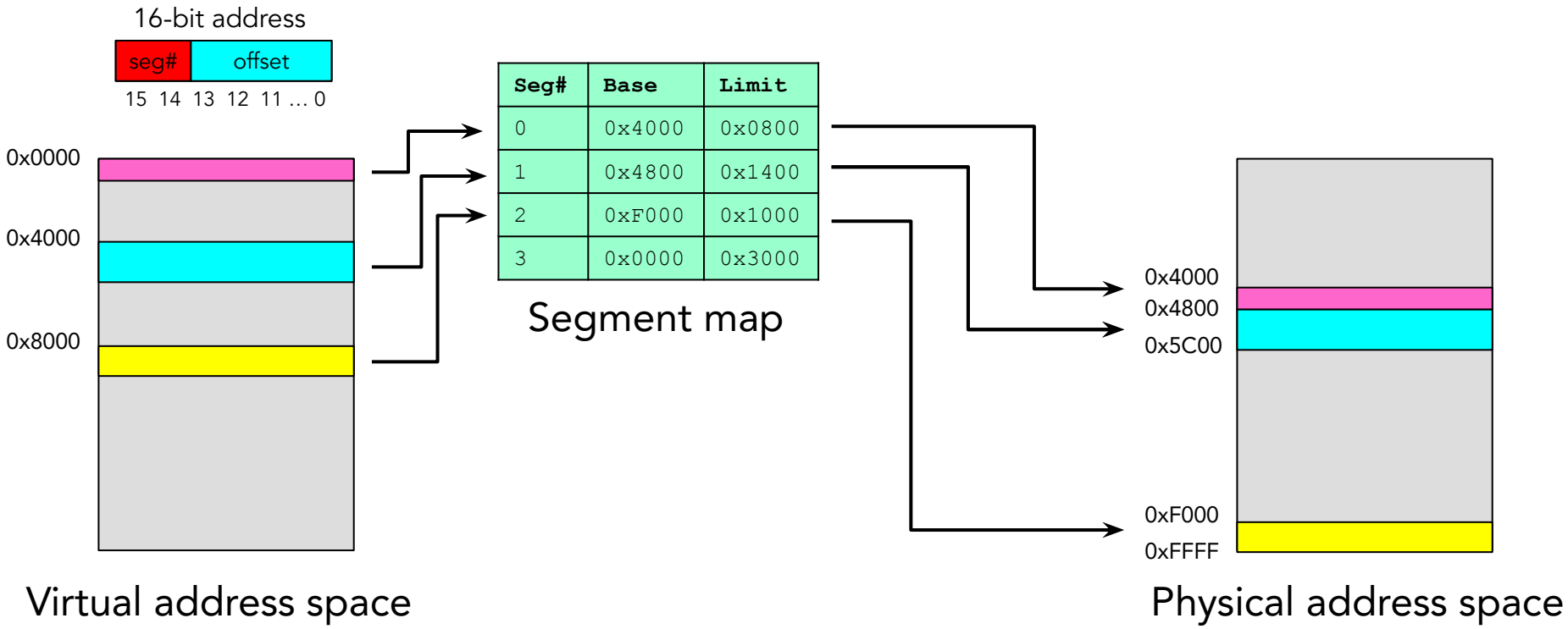
Seg#	Base	Limit
0	0x4000	0x0800
1	0x4800	0x1400
2	0xF000	0x1000
3	0x0000	0x3000

Segment map

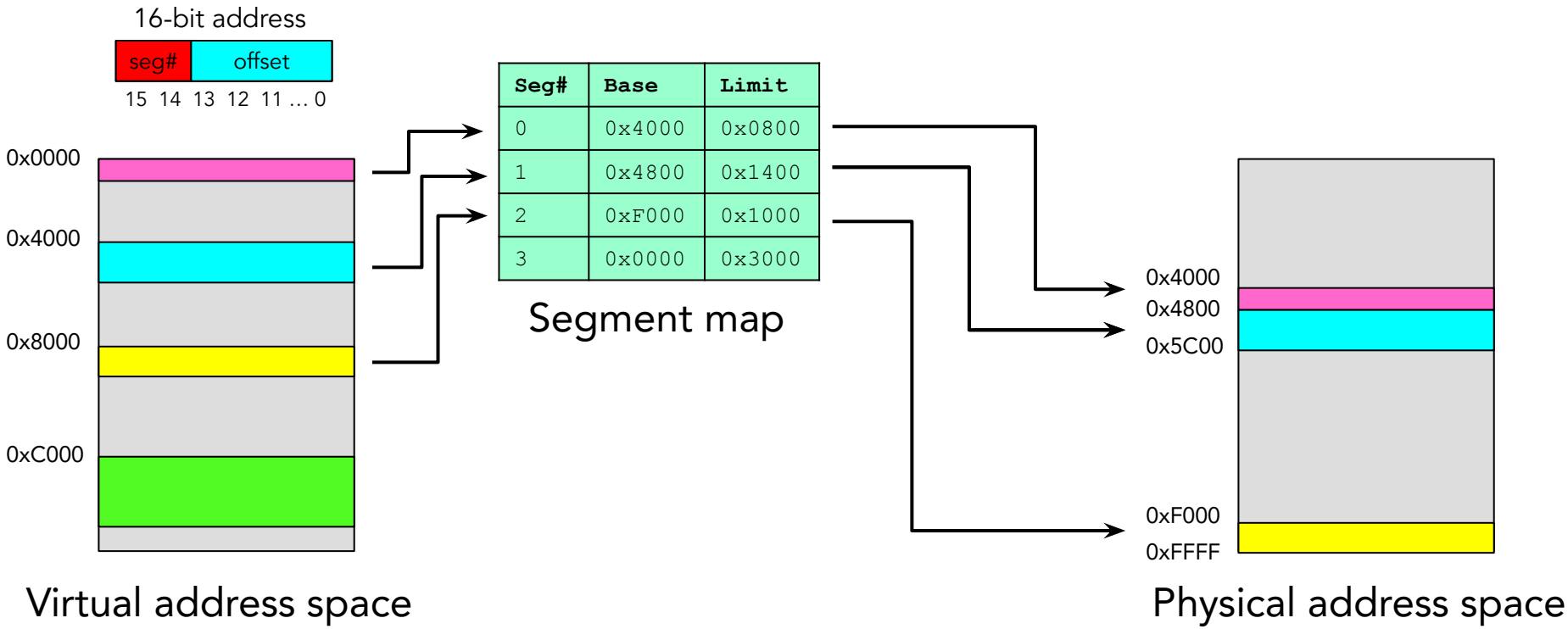
Virtual address space

Physical address space

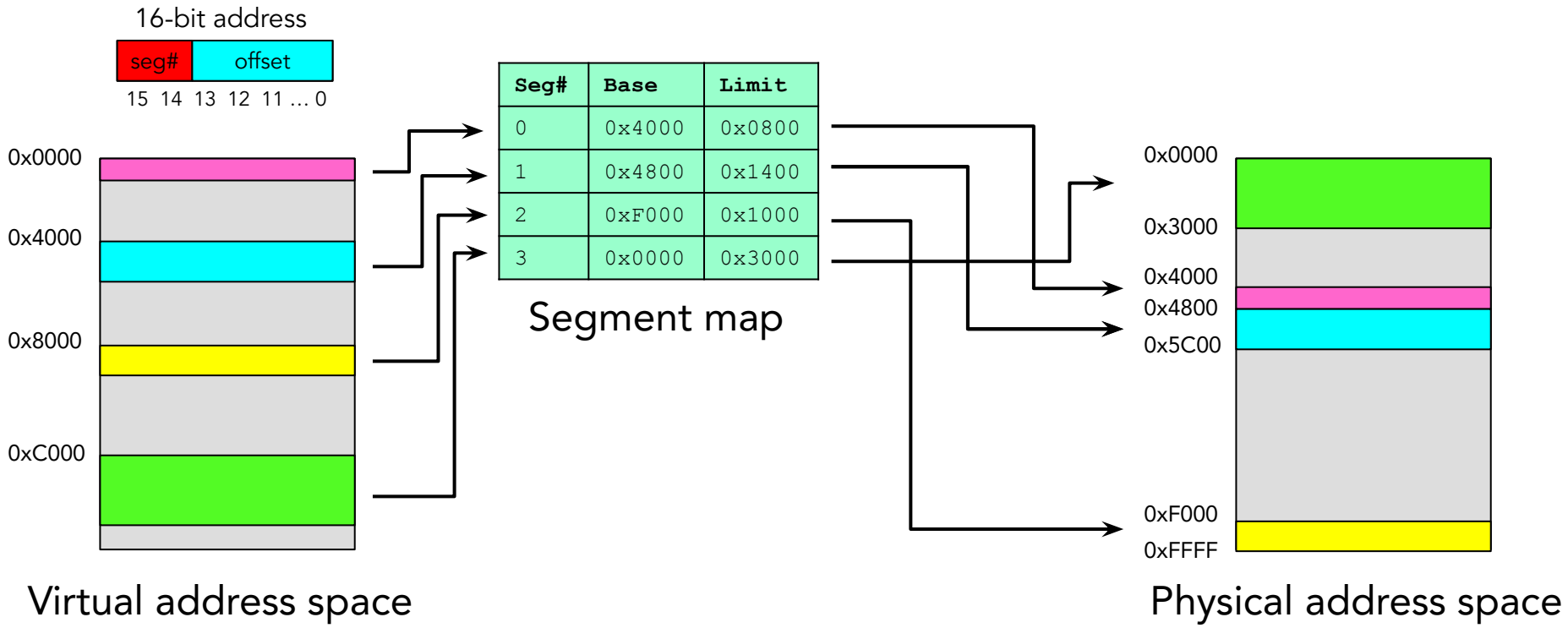
Version 3.0: Example



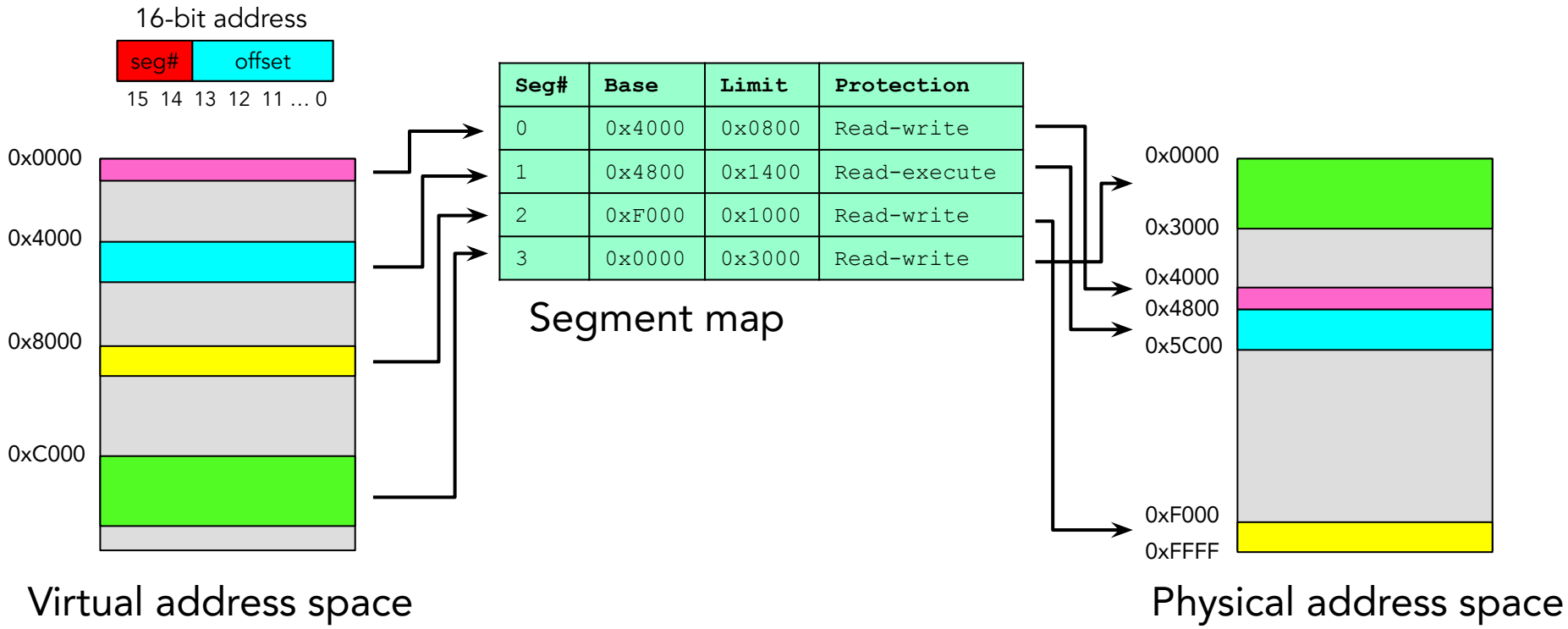
Version 3.0: Example



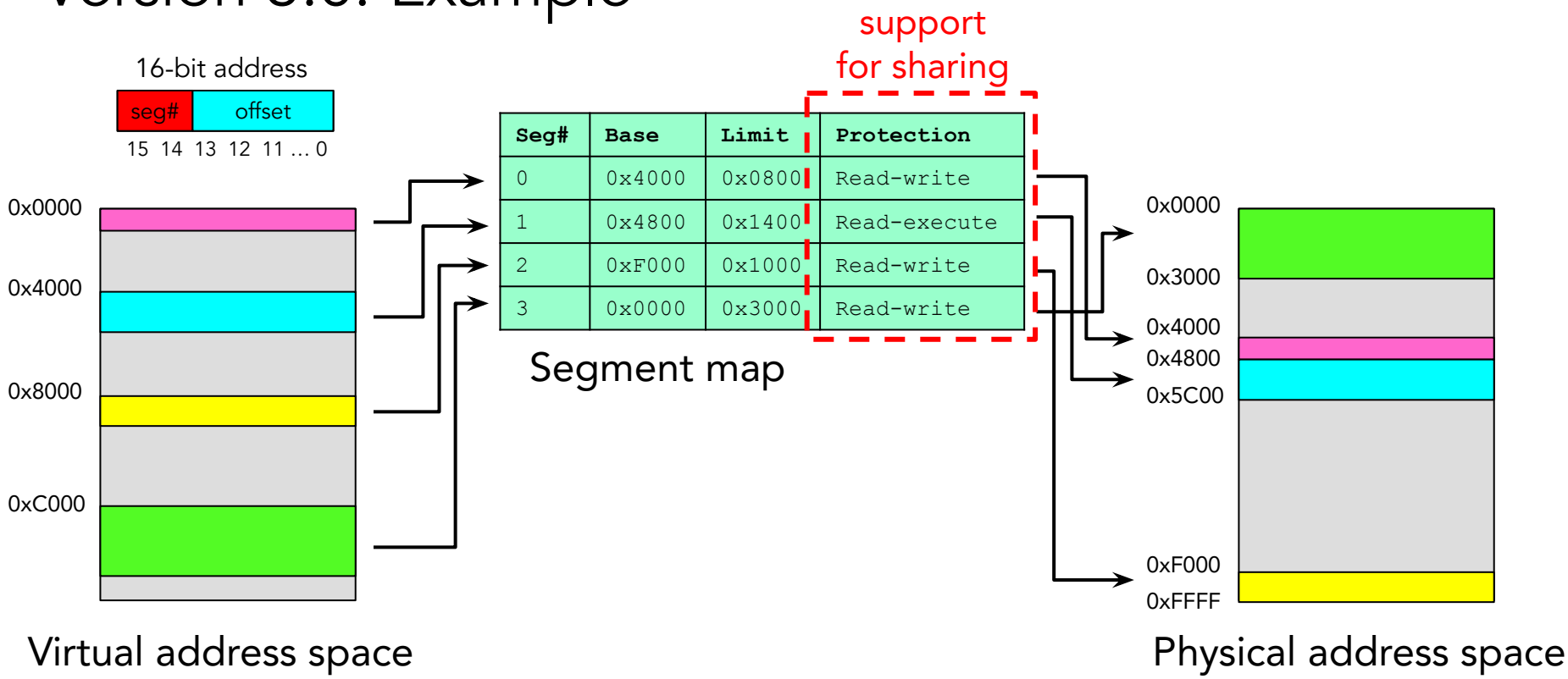
Version 3.0: Example



Version 3.0: Example



Version 3.0: Example

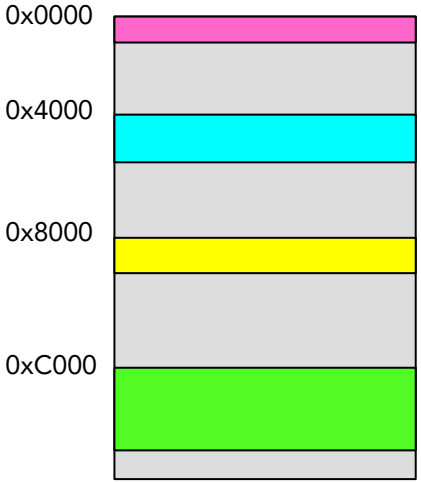


Version 3.0: Example

16-bit address

seg#	offset

15 14 13 12 11 … 0



Virtual address space

Seg#	Base	Limit	Protection
0	0x4000	0x0800	Read-write
1	0x4800	0x1400	Read-execute
2	0xF000	0x1000	Read-write
3	0x0000	0x3000	Read-write

Isolation ?

Sharing ?

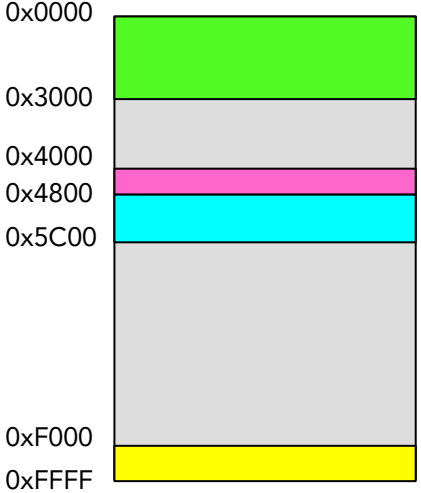
Virtualization ?

Expandable ?

Relative addresses ?

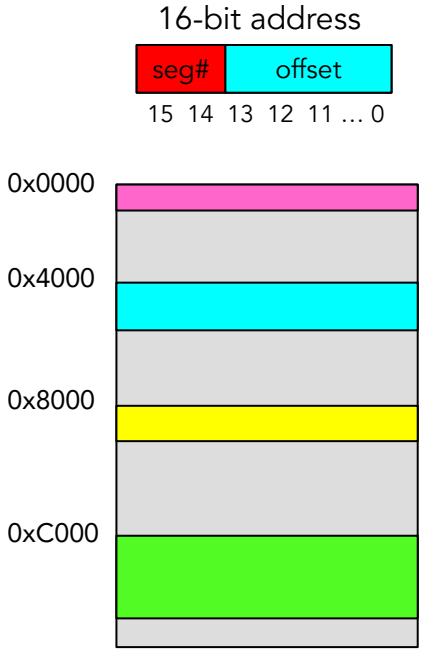
No internal fragmentation ?

No external fragmentation ?

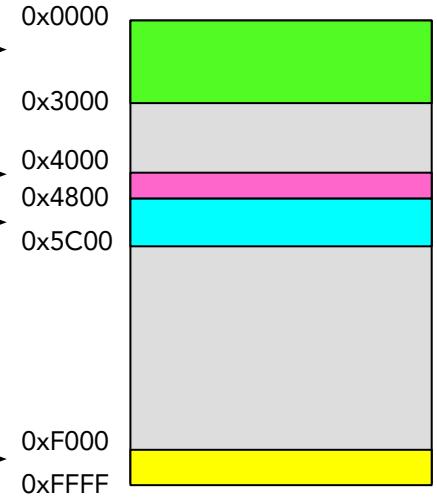


Physical address space

Version 3.0: Example



Seg#	Base	Limit	Protection
0	0x4000	0x0800	Read-write
1	0x4800	0x1400	Read-execute
2	0xF000	0x1000	Read-write
3	0x0000	0x3000	Read-write



Virtual address space

Isolation	✔
Sharing	✔
Virtualization	✗
Expandable	✔
Relative addresses	✔
No internal fragmentation	✔
No external fragmentation	✗

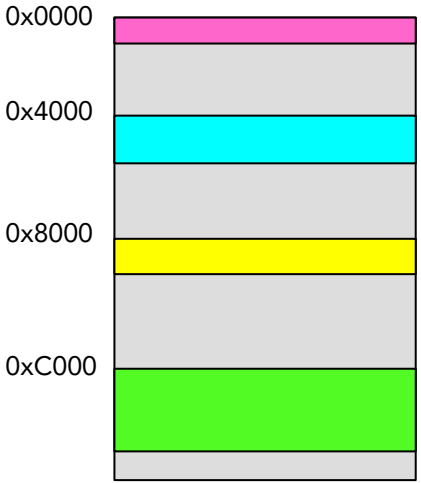
Physical address space

Version 3.0: Example

16-bit address

seg#	offset

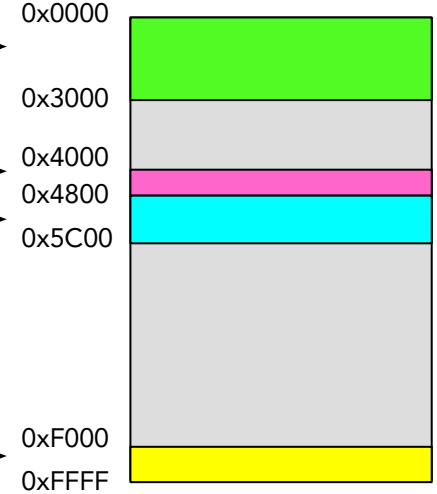
15 14 13 12 11 … 0



Virtual address space

Seg#	Base	Limit	Protection
0	0x4000	0x0800	Read-write
1	0x4800	0x1400	Read-execute
2	0xF000	0x1000	Read-write
3	0x0000	0x3000	Read-write

Isolation	✔
Sharing	✔
Virtualization	✗
Expandable	✔
Relative addresses	✔
No internal fragmentation	✔
No external fragmentation	✗



Physical address space

Version 4.0: Paging

Version 4.0: Paging

- How to minimize external fragmentation?

Version 4.0: Paging

- How to minimize external fragmentation?
  - Allocate physical memory in fixed size chunks (“pages”)

Version 4.0: Paging

- How to minimize external fragmentation?
  - Allocate physical memory in fixed size chunks (“pages”)
  - Page size

Version 4.0: Paging

- How to minimize external fragmentation?
  - Allocate physical memory in fixed size chunks (“pages”)
  - Page size
    - Too big?

Version 4.0: Paging

- How to minimize external fragmentation?
  - Allocate physical memory in fixed size chunks (“pages”)
  - Page size
    - Too big?
    - Too small?

Version 4.0: Paging

- How to minimize external fragmentation?
  - Allocate physical memory in fixed size chunks ("pages")
  - Page size
    - Too big?
    - Too small?

Physical page#	Permission
page #0	R
page #1	R
page #2	W
page #3	W
…	…

Version 4.0: Paging

- How to minimize external fragmentation?
  - Allocate physical memory in fixed size chunks ("pages")
  - Page size
    - Too big?
    - Too small?

Physical page#	Permission
page #0	R
page #1	R
page #2	W
page #3	W
…	…

Page table

Version 4.0: Paging

- How to minimize external fragmentation?
  - Allocate physical memory in fixed size chunks ("pages")
  - Page size
    - Too big?
    - Too small?

virtual address

Physical page#	Permission
page #0	R
page #1	R
page #2	W
page #3	W
…	…

Page table

Version 4.0: Paging

- How to minimize external fragmentation?
  - Allocate physical memory in fixed size chunks ("pages")
  - Page size
    - Too big?
    - Too small?

virtual address

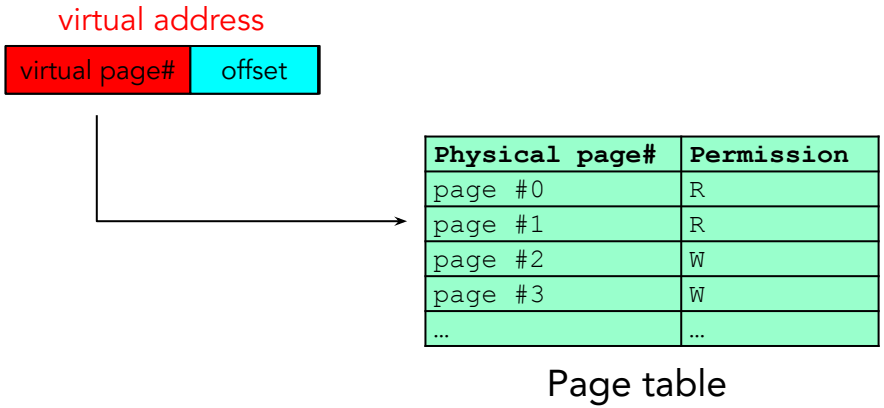
virtual page#	offset

Physical page#	Permission
page #0	R
page #1	R
page #2	W
page #3	W
…	…

Page table

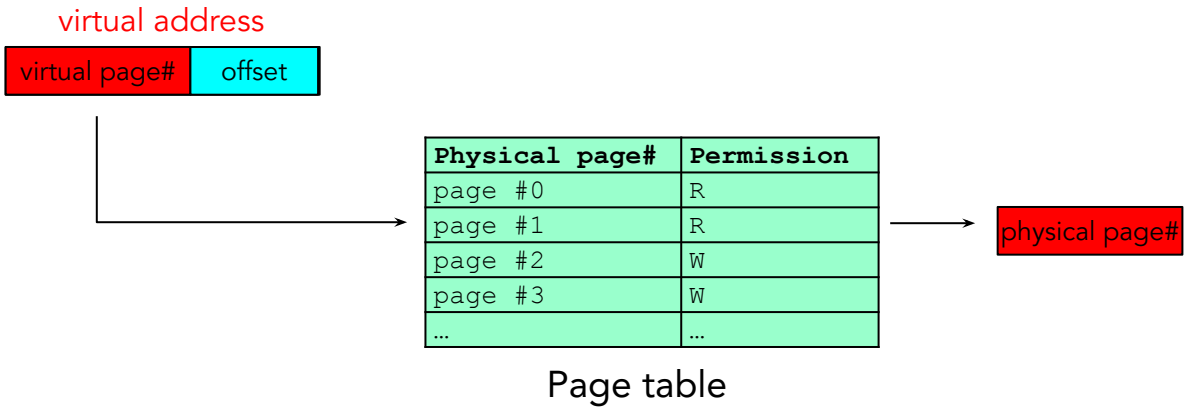
Version 4.0: Paging

- How to minimize external fragmentation?
  - Allocate physical memory in fixed size chunks ("pages")
  - Page size
    - Too big?
    - Too small?



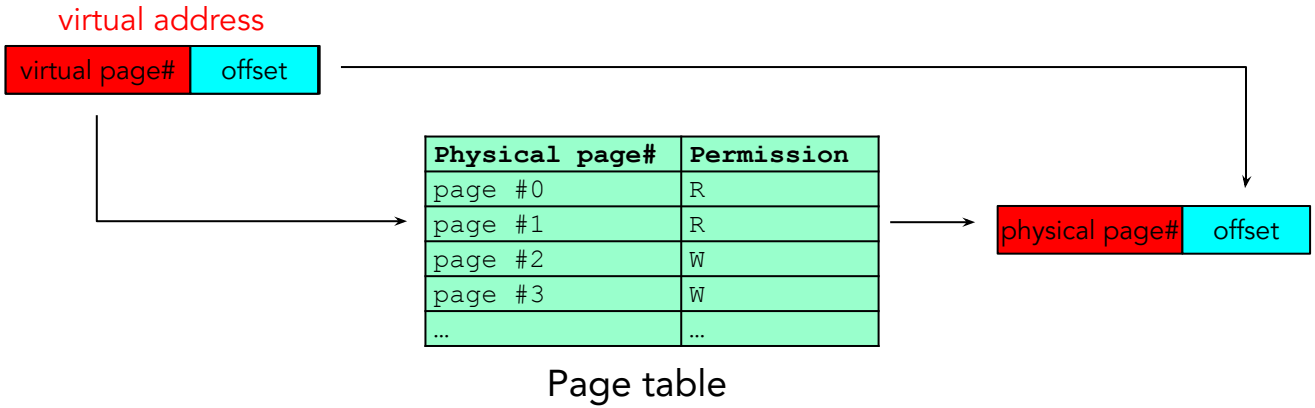
Version 4.0: Paging

- How to minimize external fragmentation?
  - Allocate physical memory in fixed size chunks ("pages")
  - Page size
    - Too big?
    - Too small?



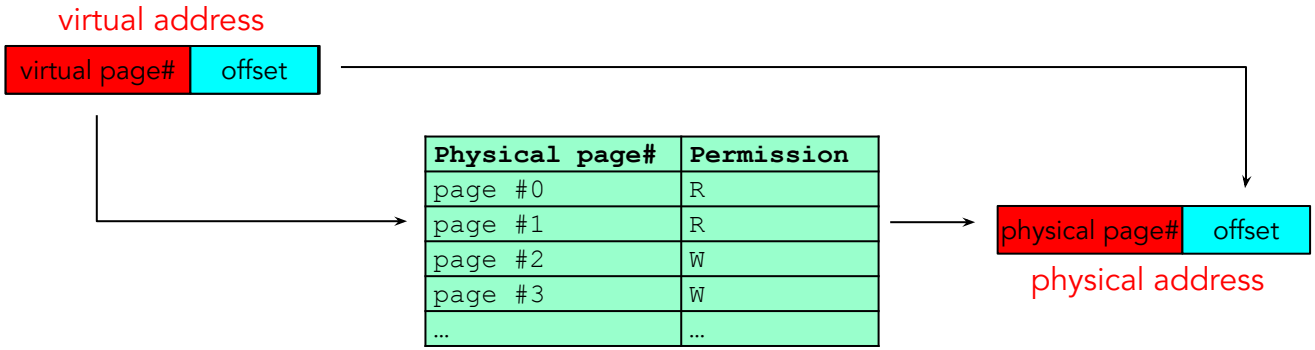
Version 4.0: Paging

- How to minimize external fragmentation?
  - Allocate physical memory in fixed size chunks ("pages")
  - Page size
    - Too big?
    - Too small?



Version 4.0: Paging

- How to minimize external fragmentation?
  - Allocate physical memory in fixed size chunks ("pages")
  - Page size
    - Too big?
    - Too small?



Page table

Version 4.0: Example

Version 4.0: Example

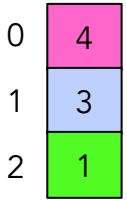
- 8-bit addresses and 4-byte pages

Version 4.0: Example

- 8-bit addresses and 4-byte pages
  - First 6 bits → virtual page number
  - Last 2 bits → offset

Version 4.0: Example

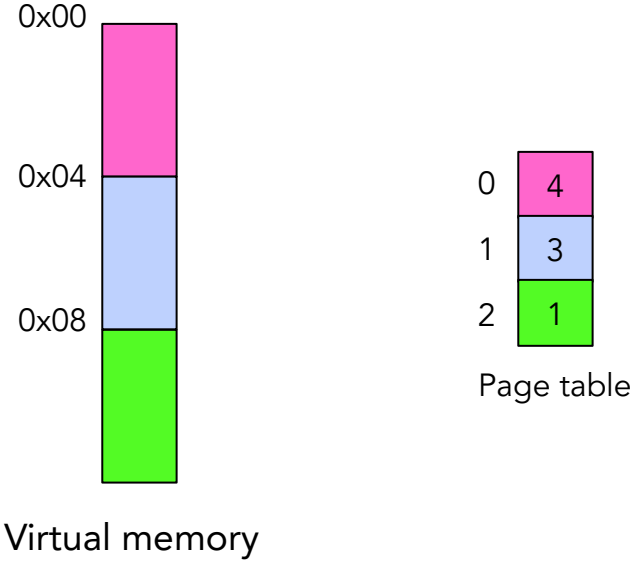
- 8-bit addresses and 4-byte pages
  - First 6 bits → virtual page number
  - Last 2 bits → offset



Page table

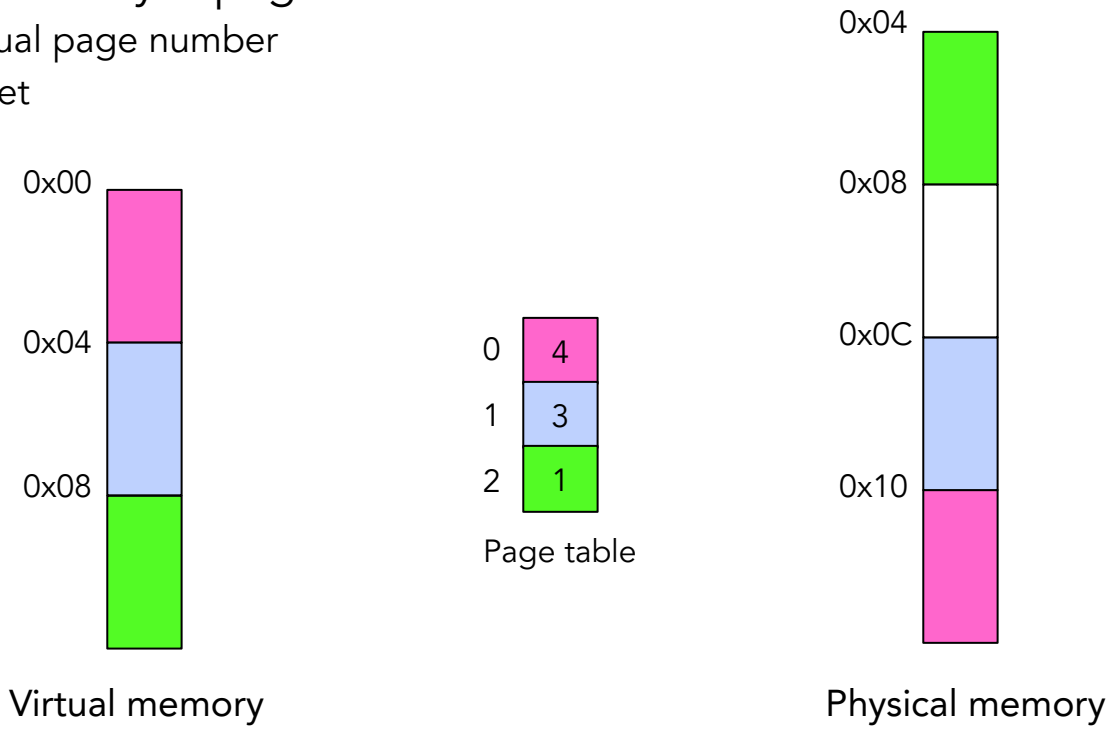
Version 4.0: Example

- 8-bit addresses and 4-byte pages
  - First 6 bits → virtual page number
  - Last 2 bits → offset



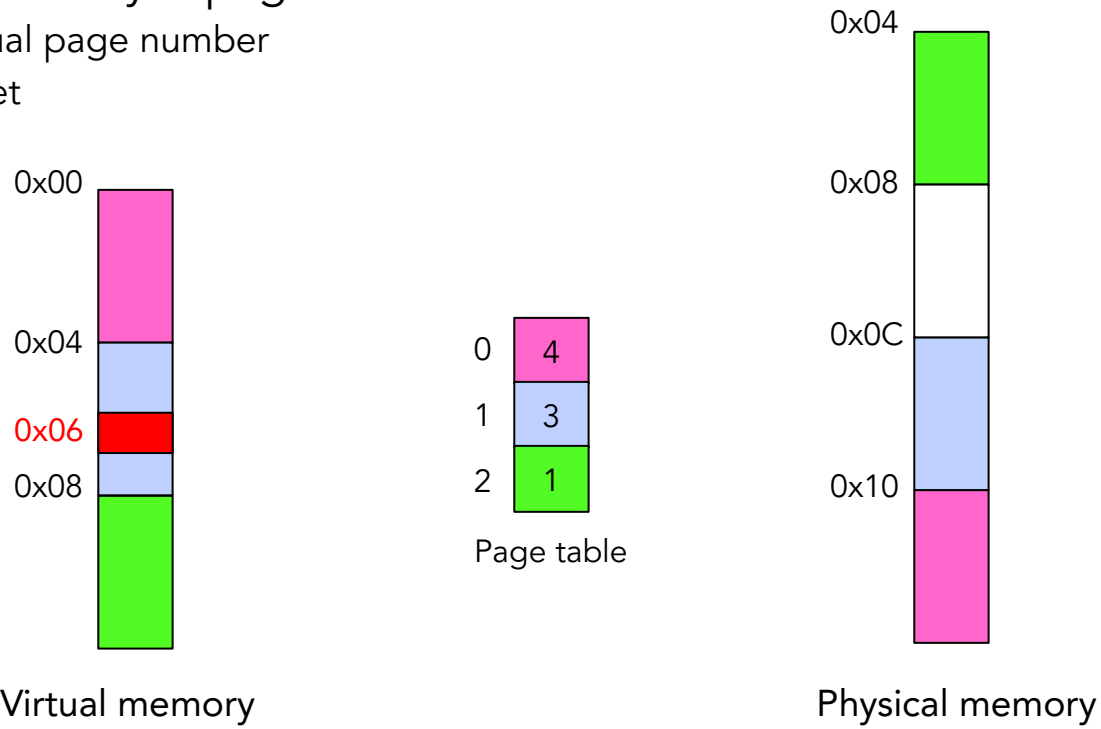
Version 4.0: Example

- 8-bit addresses and 4-byte pages
  - First 6 bits → virtual page number
  - Last 2 bits → offset



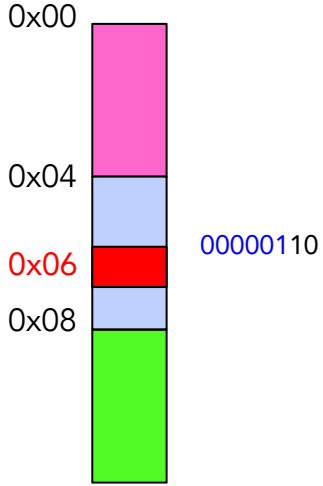
Version 4.0: Example

- 8-bit addresses and 4-byte pages
  - First 6 bits → virtual page number
  - Last 2 bits → offset

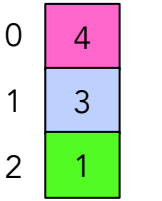


Version 4.0: Example

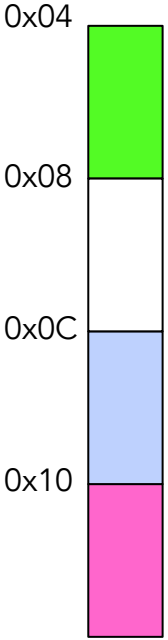
- 8-bit addresses and 4-byte pages
  - First 6 bits → virtual page number
  - Last 2 bits → offset



Virtual memory



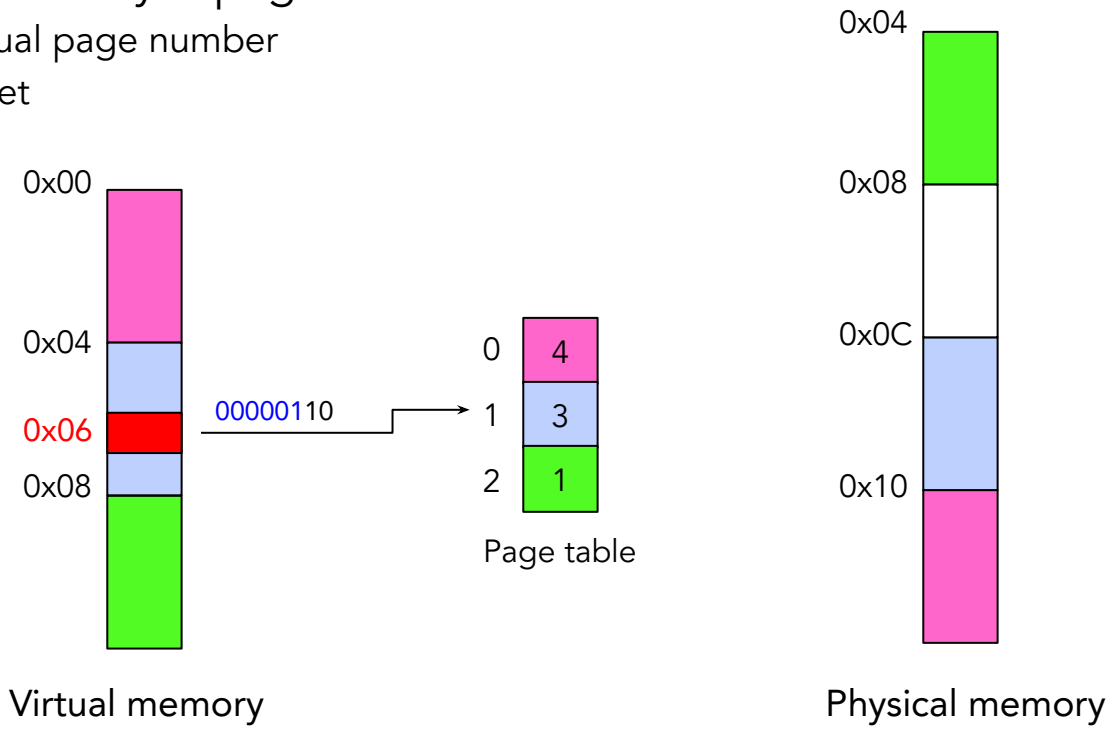
Page table



Physical memory

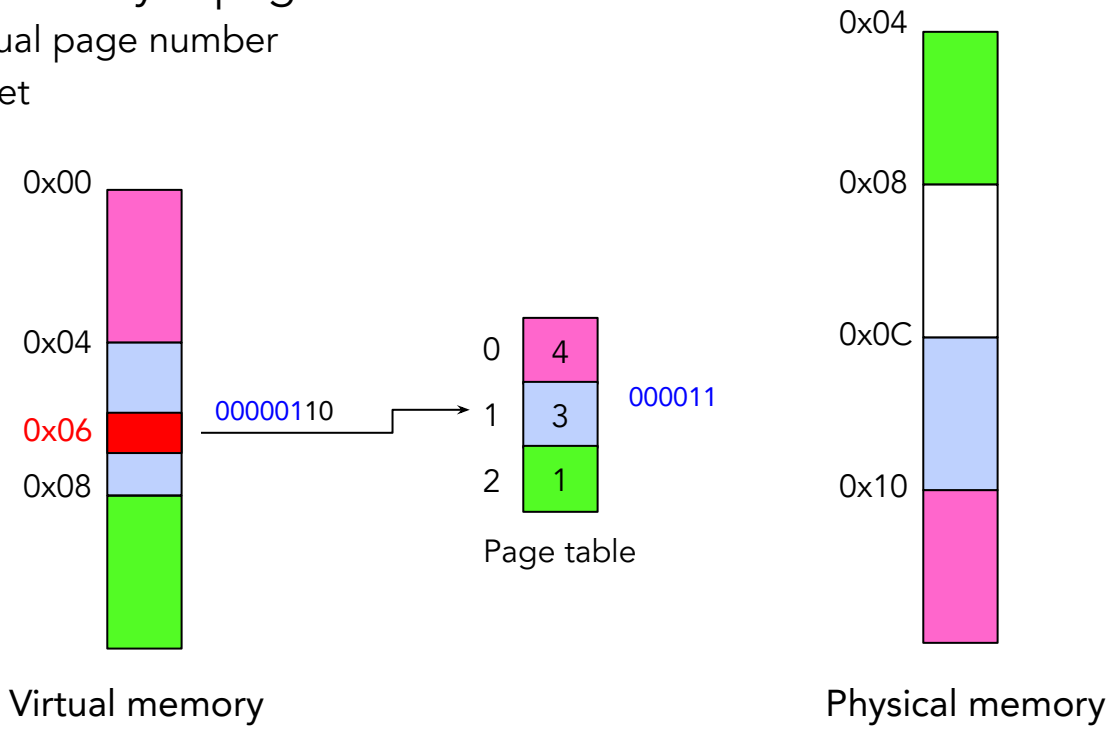
Version 4.0: Example

- 8-bit addresses and 4-byte pages
  - First 6 bits → virtual page number
  - Last 2 bits → offset



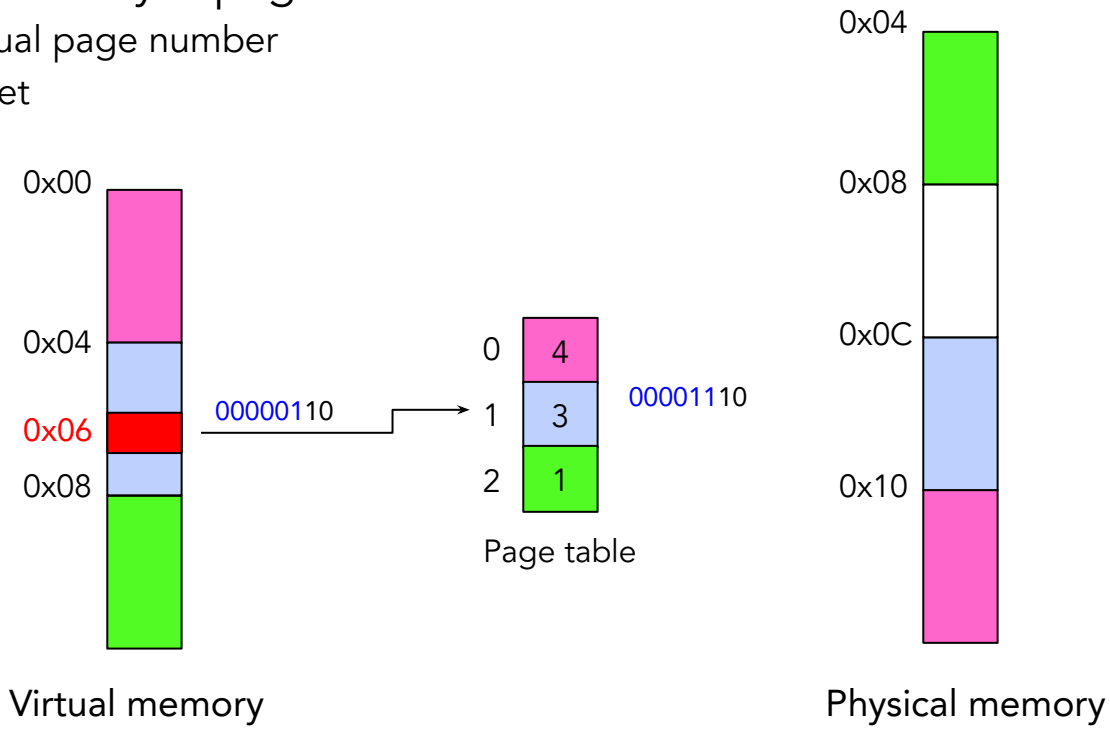
Version 4.0: Example

- 8-bit addresses and 4-byte pages
  - First 6 bits → virtual page number
  - Last 2 bits → offset



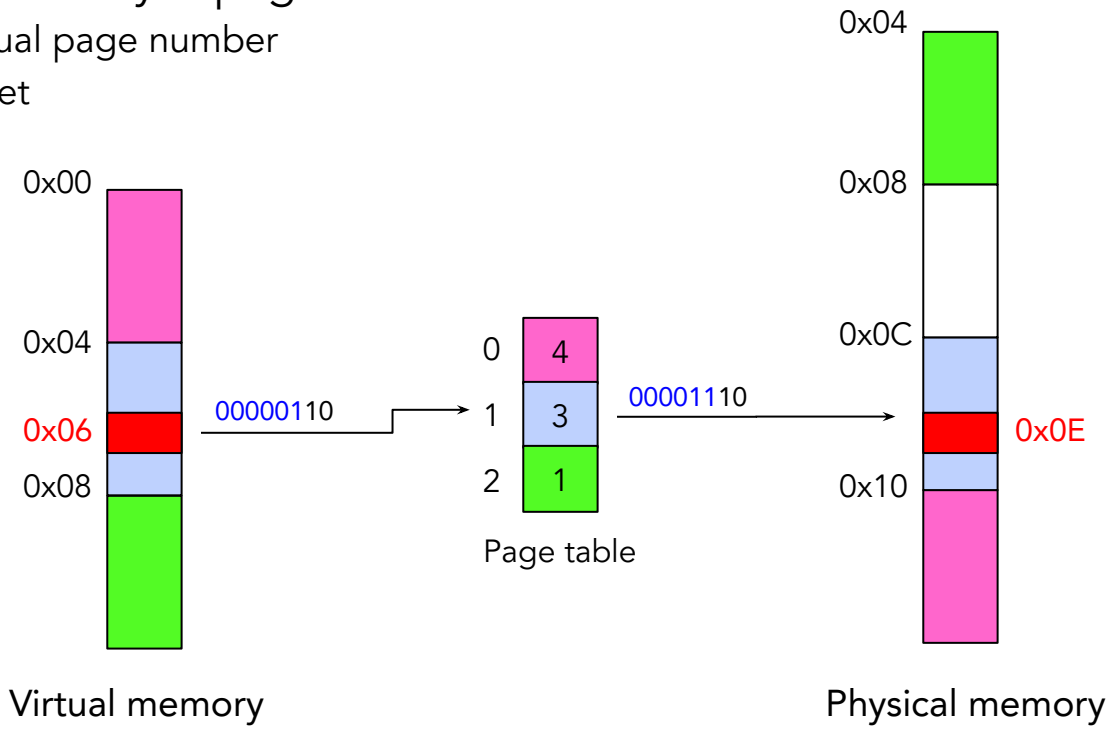
Version 4.0: Example

- 8-bit addresses and 4-byte pages
  - First 6 bits → virtual page number
  - Last 2 bits → offset



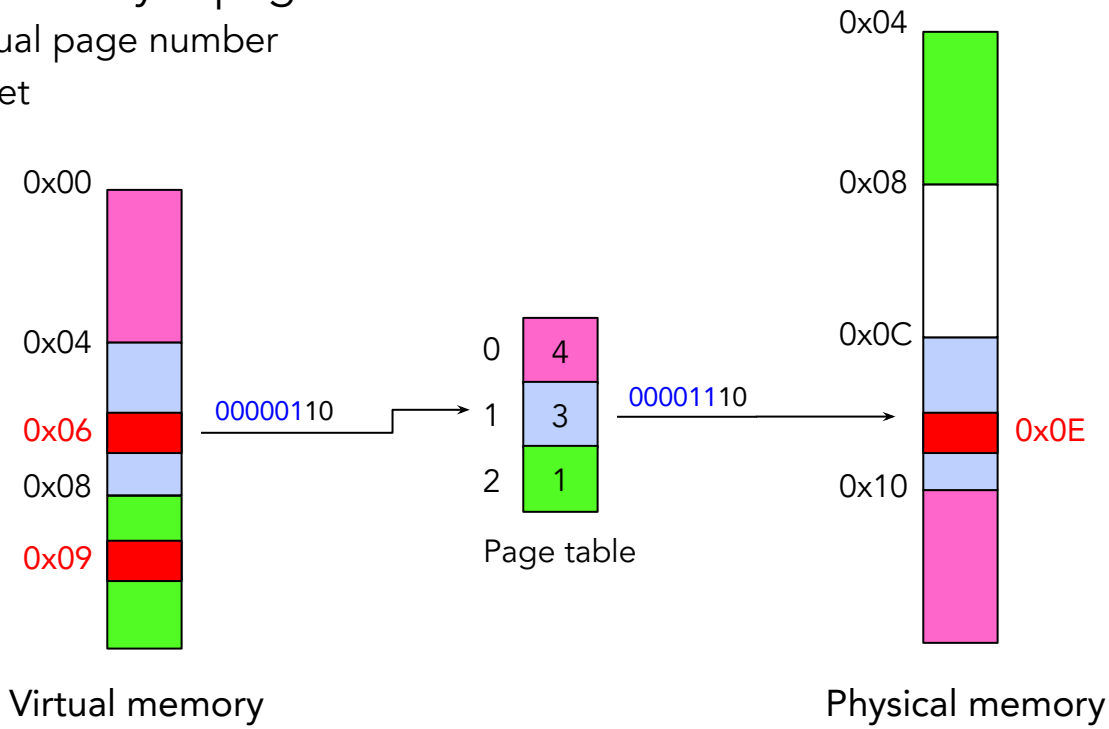
Version 4.0: Example

- 8-bit addresses and 4-byte pages
  - First 6 bits → virtual page number
  - Last 2 bits → offset



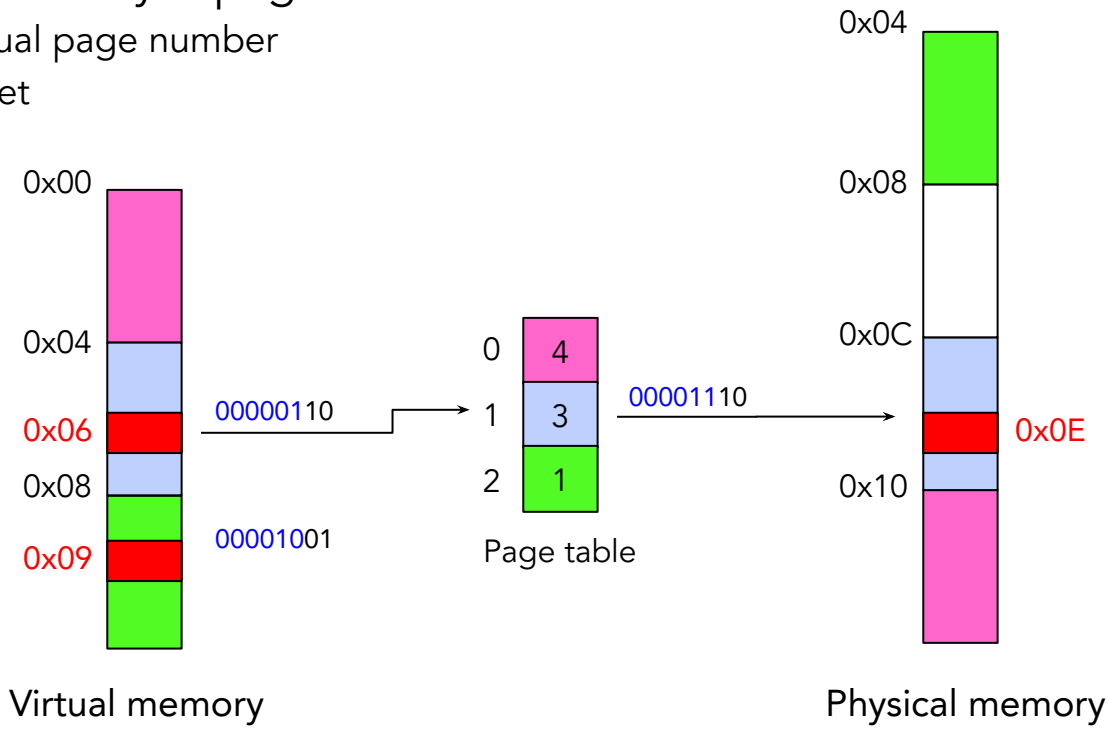
Version 4.0: Example

- 8-bit addresses and 4-byte pages
  - First 6 bits → virtual page number
  - Last 2 bits → offset



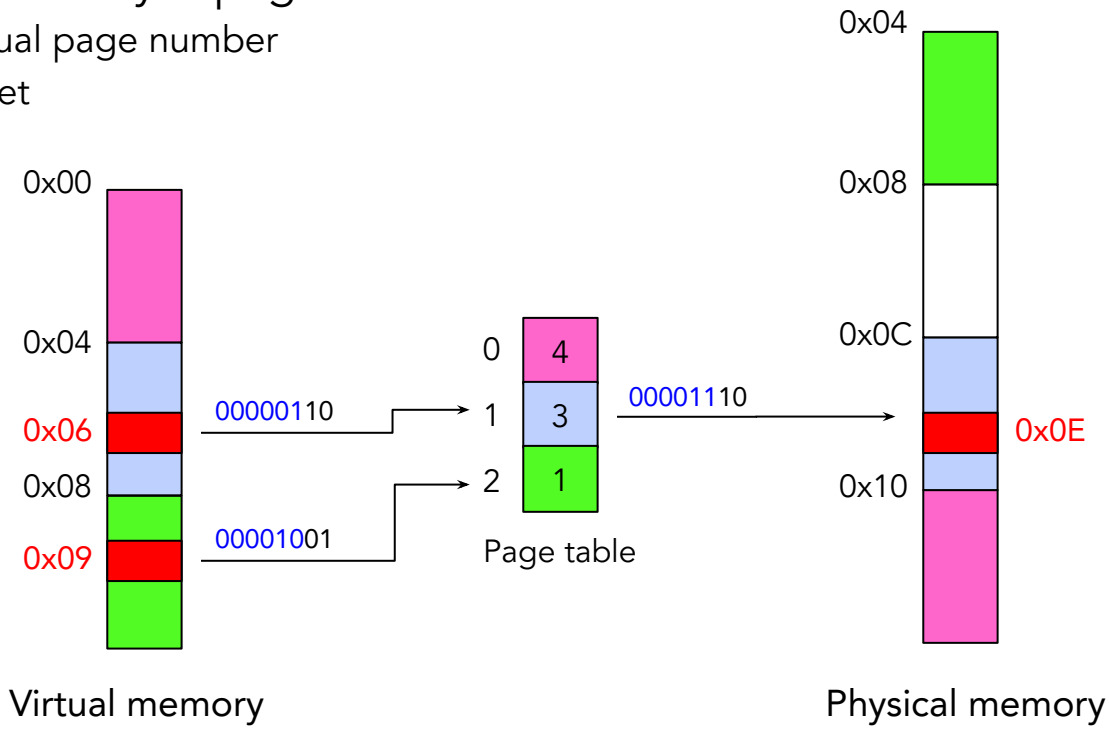
Version 4.0: Example

- 8-bit addresses and 4-byte pages
  - First 6 bits → virtual page number
  - Last 2 bits → offset



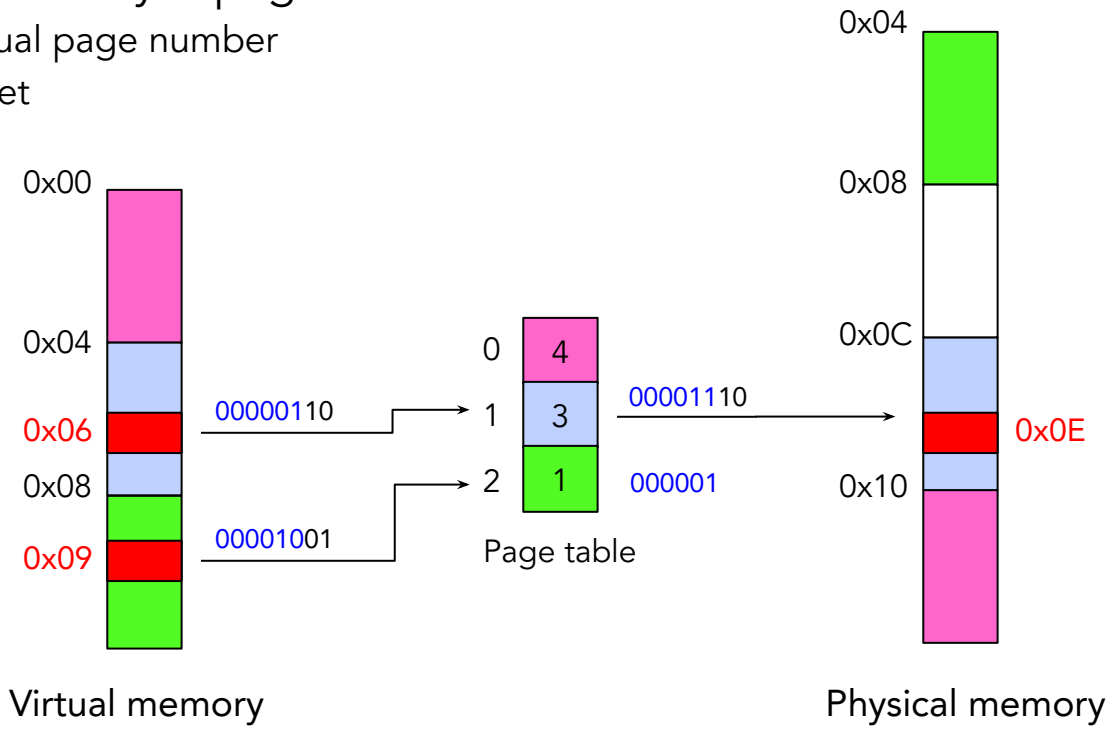
Version 4.0: Example

- 8-bit addresses and 4-byte pages
  - First 6 bits → virtual page number
  - Last 2 bits → offset



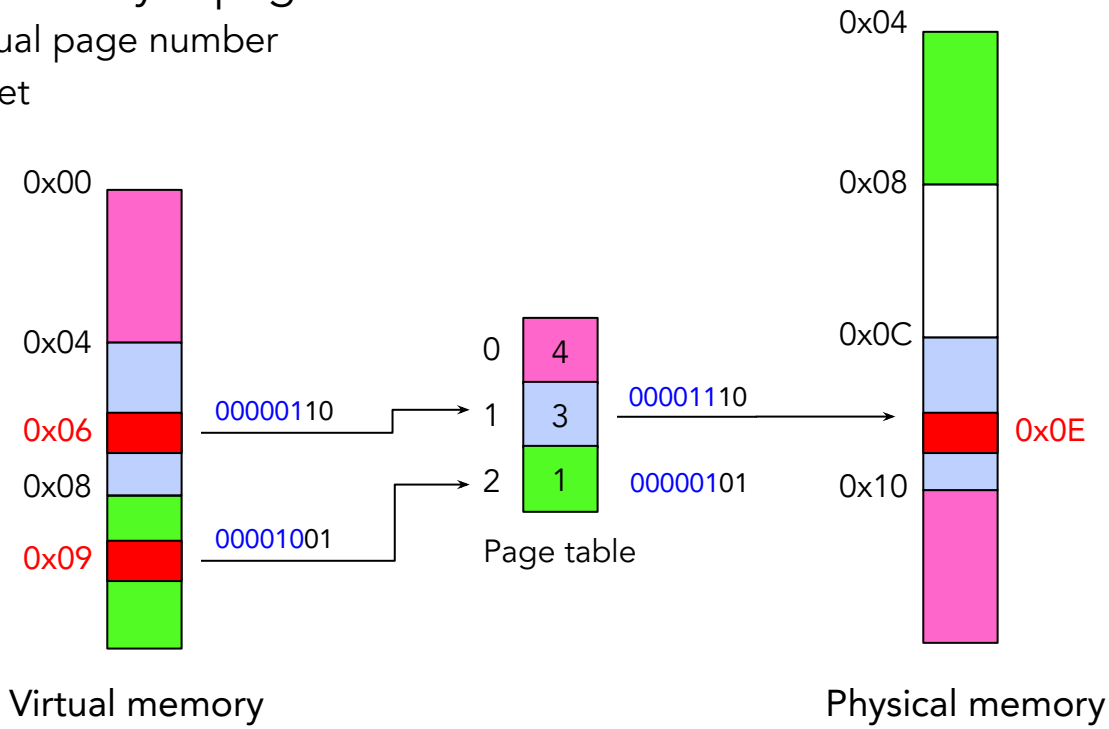
Version 4.0: Example

- 8-bit addresses and 4-byte pages
  - First 6 bits → virtual page number
  - Last 2 bits → offset



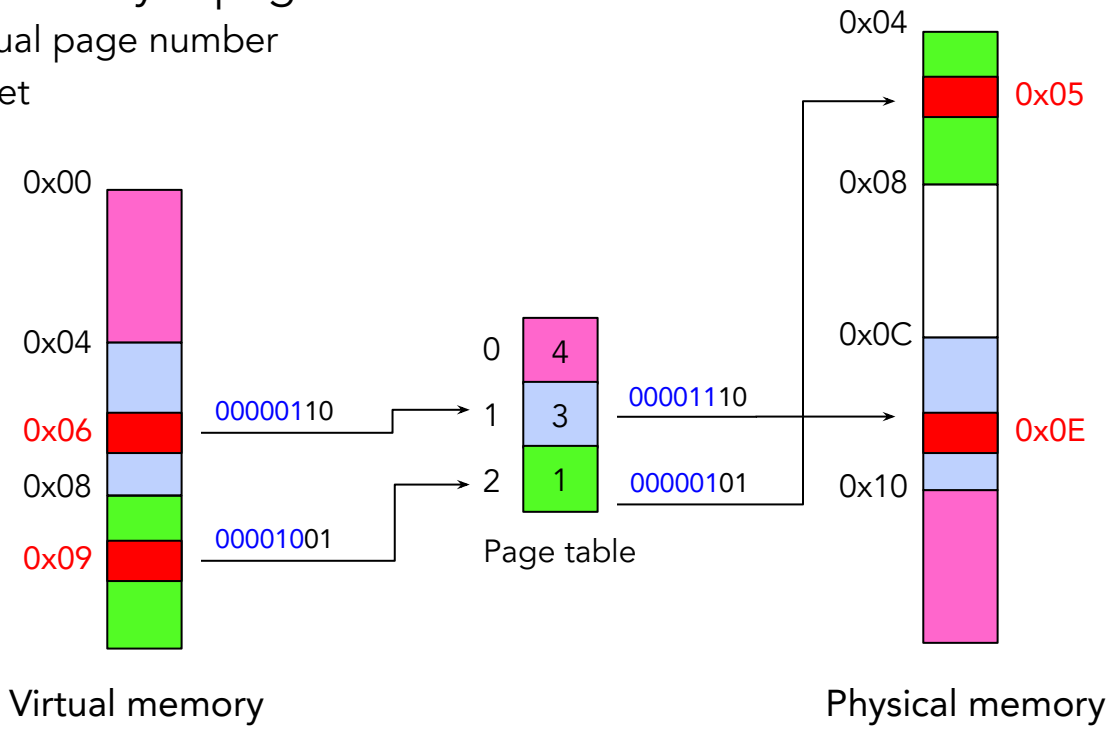
Version 4.0: Example

- 8-bit addresses and 4-byte pages
  - First 6 bits → virtual page number
  - Last 2 bits → offset

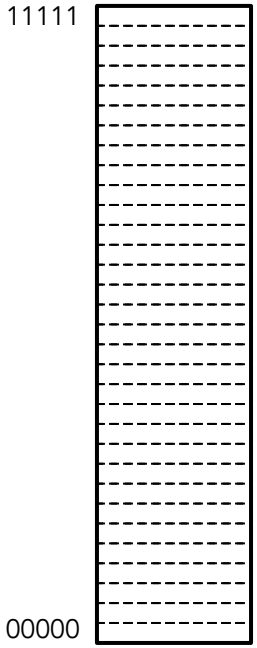


Version 4.0: Example

- 8-bit addresses and 4-byte pages
  - First 6 bits → virtual page number
  - Last 2 bits → offset

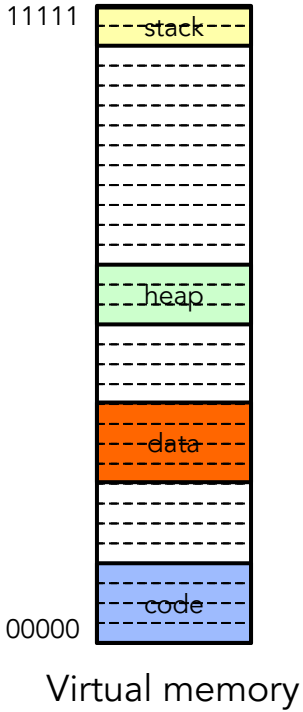


Version 4.0: Dynamic allocation

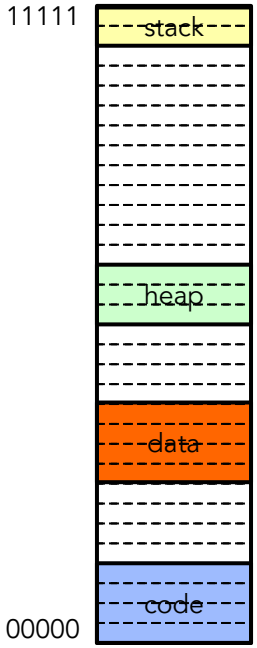


Virtual memory

Version 4.0: Dynamic allocation



Version 4.0: Dynamic allocation

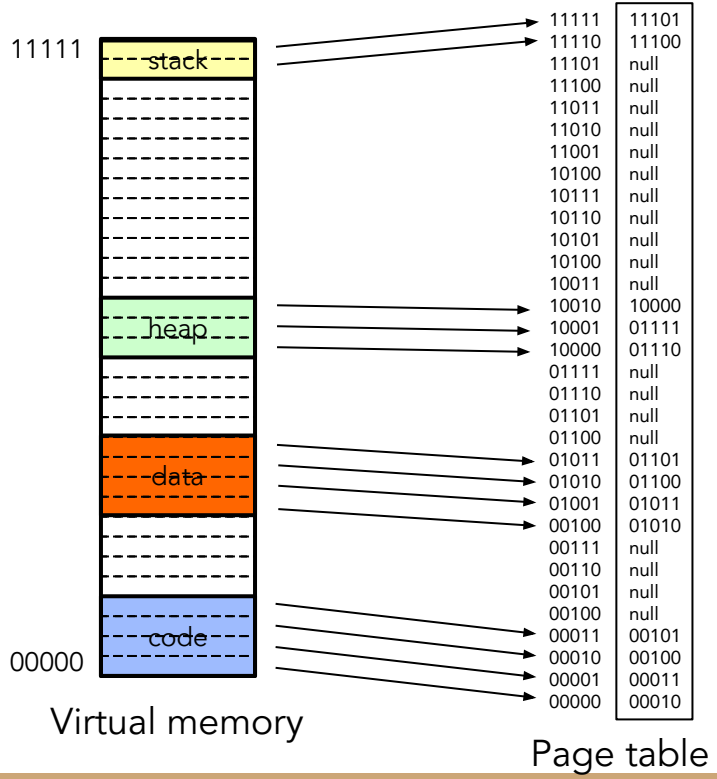


Virtual memory

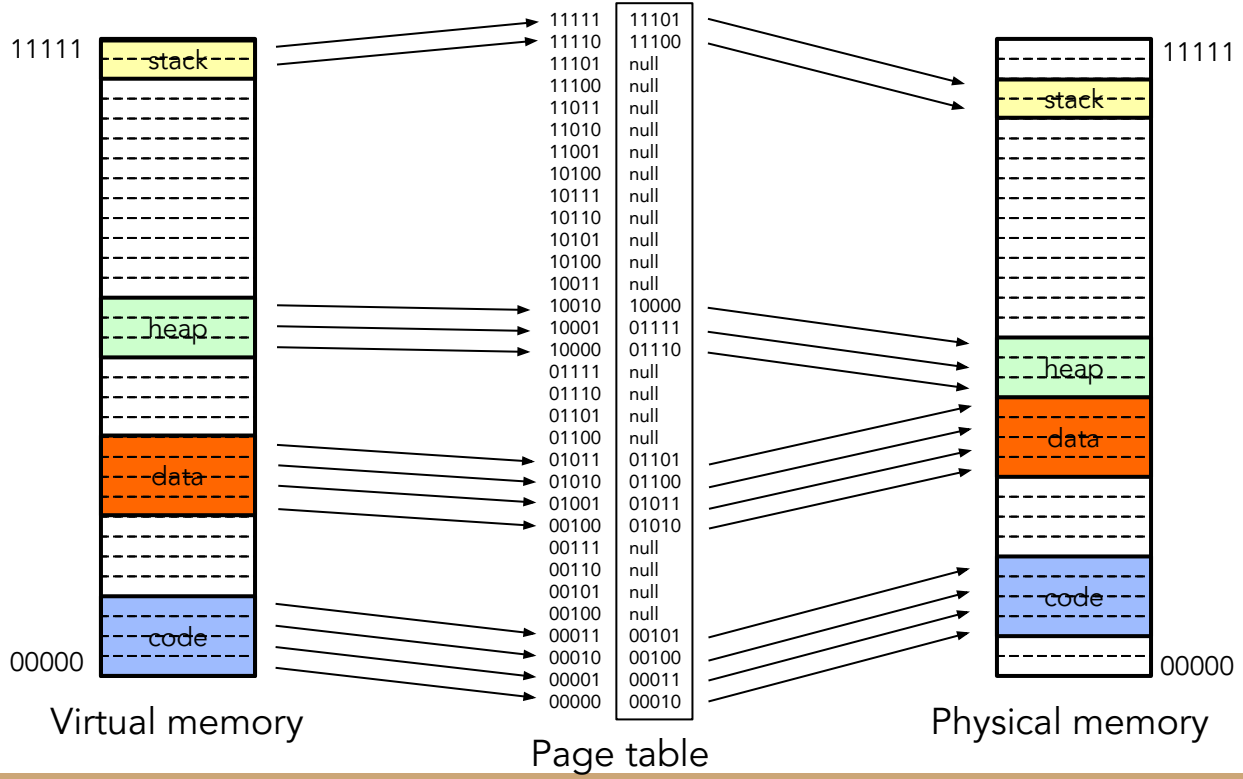
11111	11101
11110	11100
11101	null
11100	null
11011	null
11010	null
11001	null
10100	null
10111	null
10110	null
10101	null
10100	null
10011	null
10010	10000
10001	01111
10000	01110
01111	null
01110	null
01101	null
01100	null
01011	01101
01010	01100
01001	01011
00100	01010
00111	null
00110	null
00101	null
00100	null
00011	00101
00010	00100
00001	00011
00000	00010

Page table

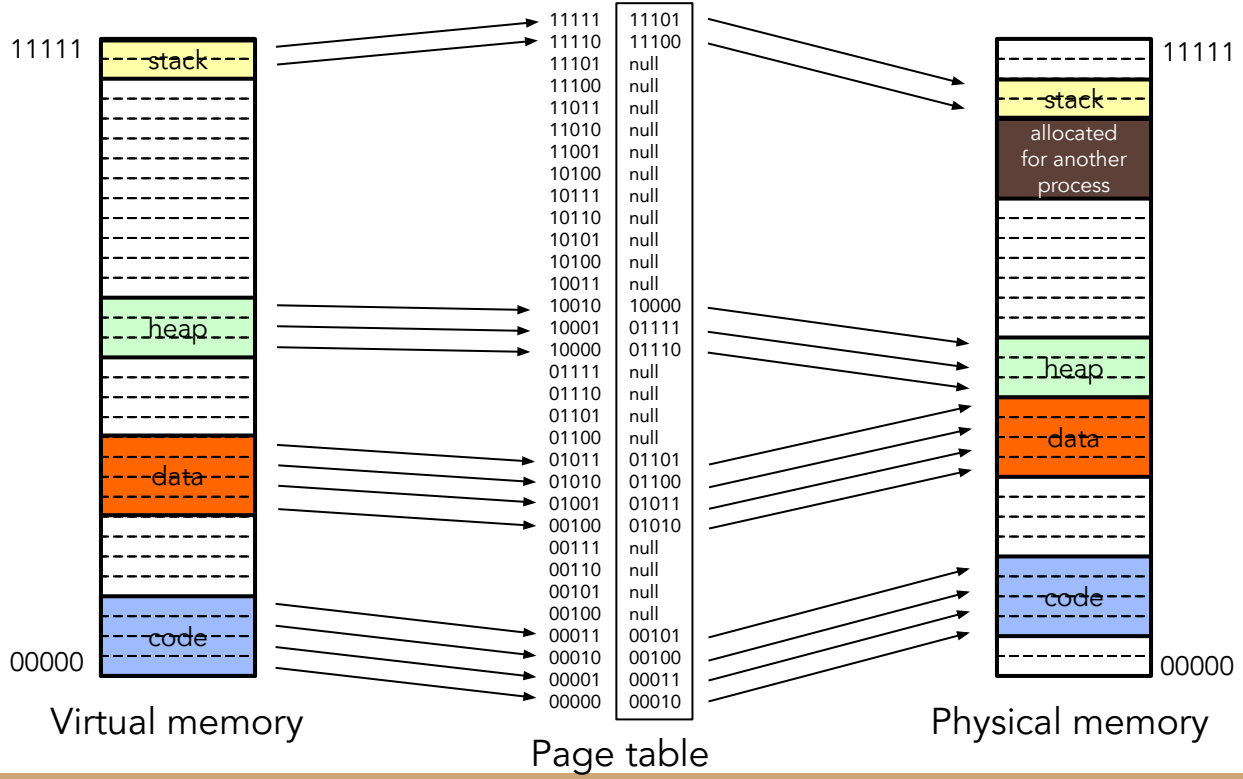
Version 4.0: Dynamic allocation



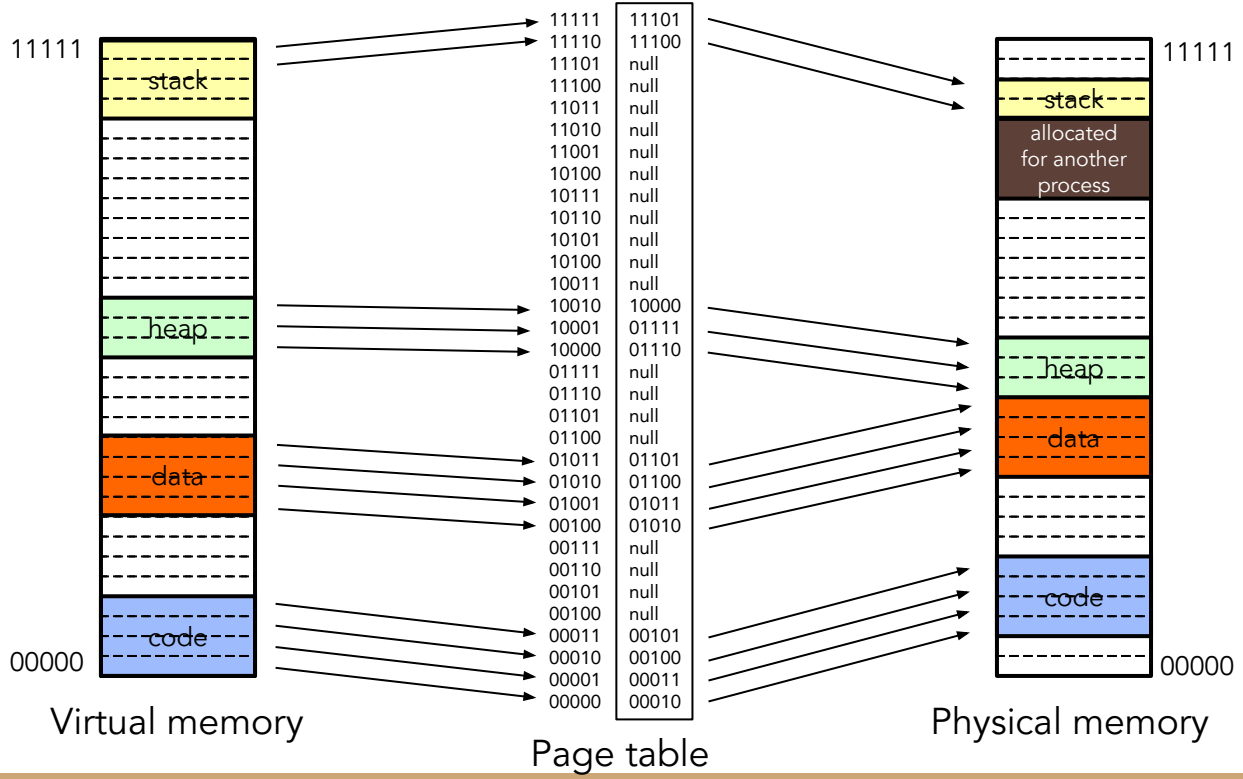
Version 4.0: Dynamic allocation



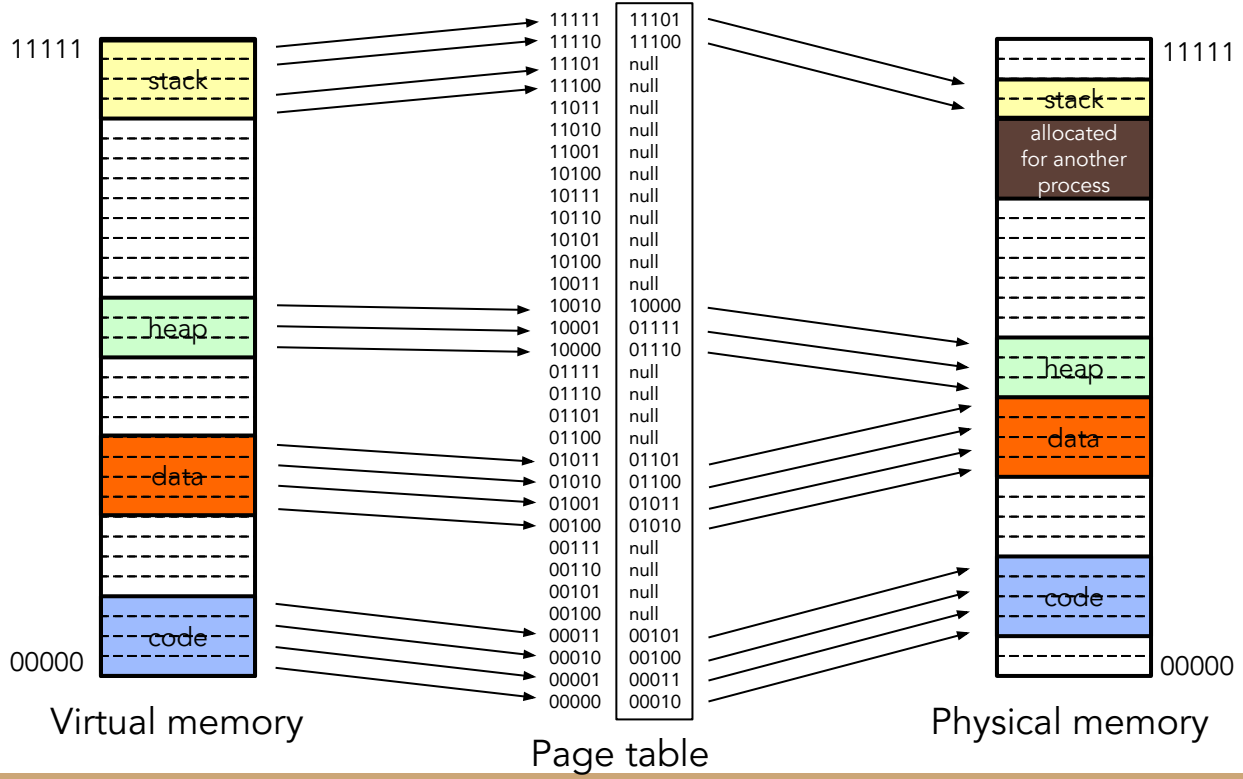
Version 4.0: Dynamic allocation



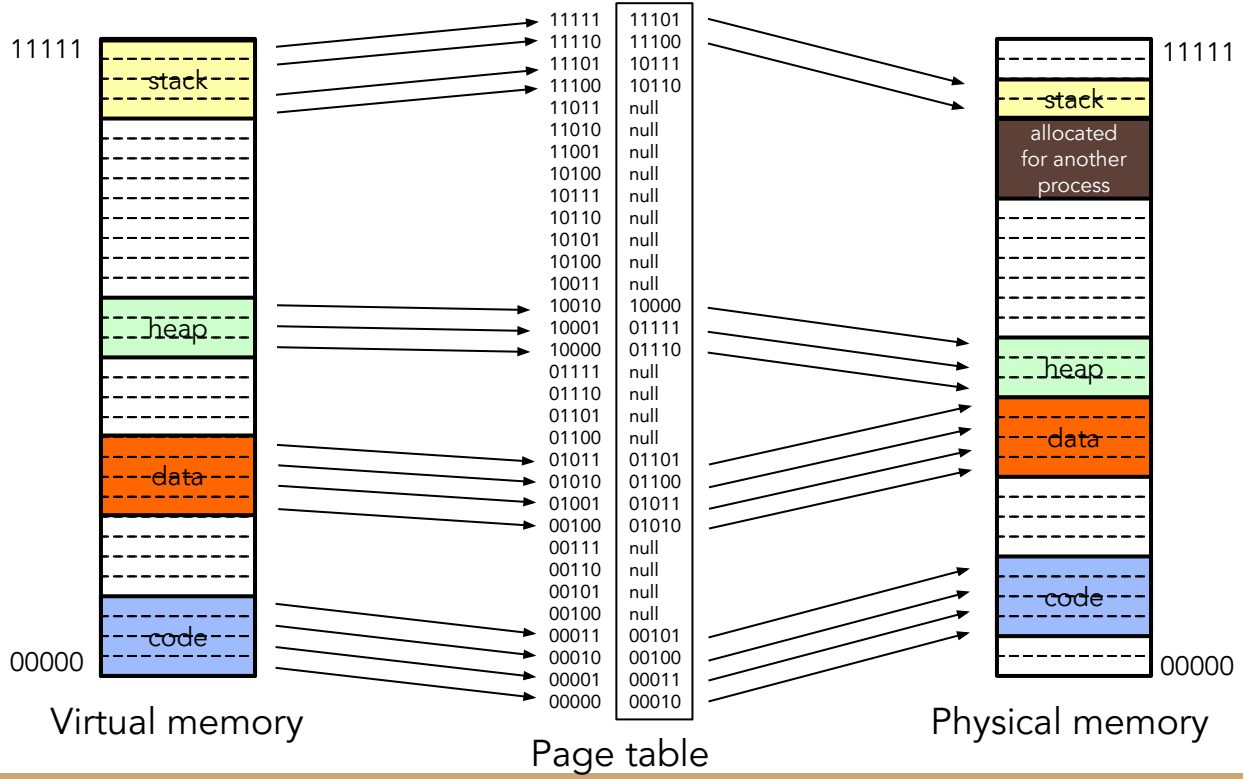
Version 4.0: Dynamic allocation



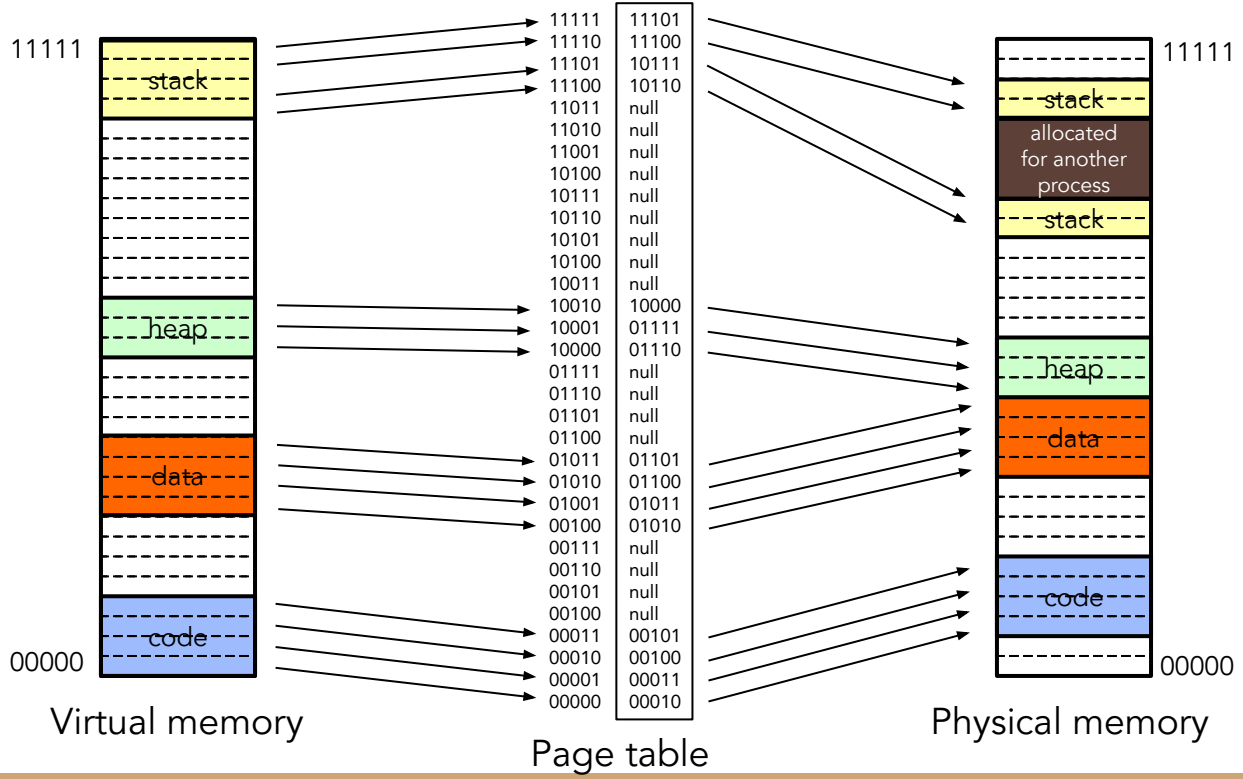
Version 4.0: Dynamic allocation



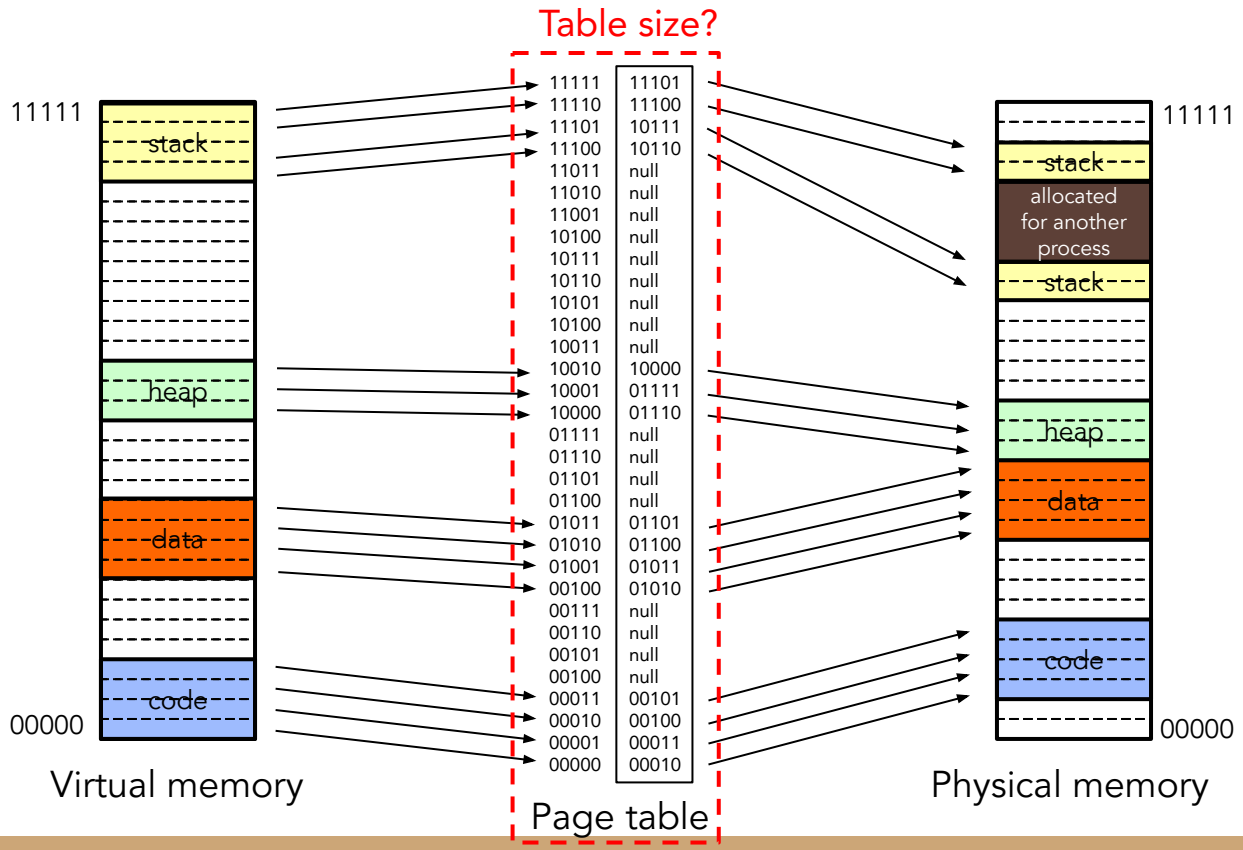
Version 4.0: Dynamic allocation



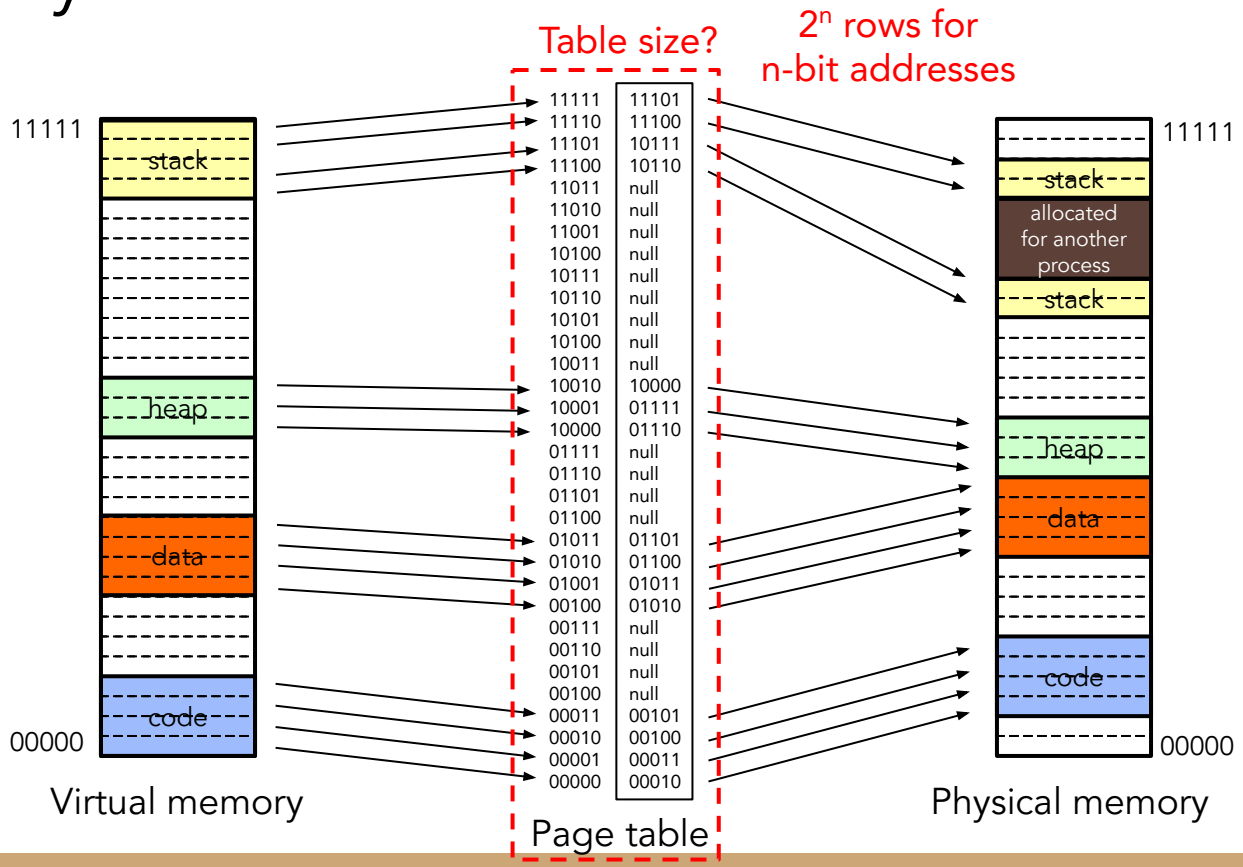
Version 4.0: Dynamic allocation



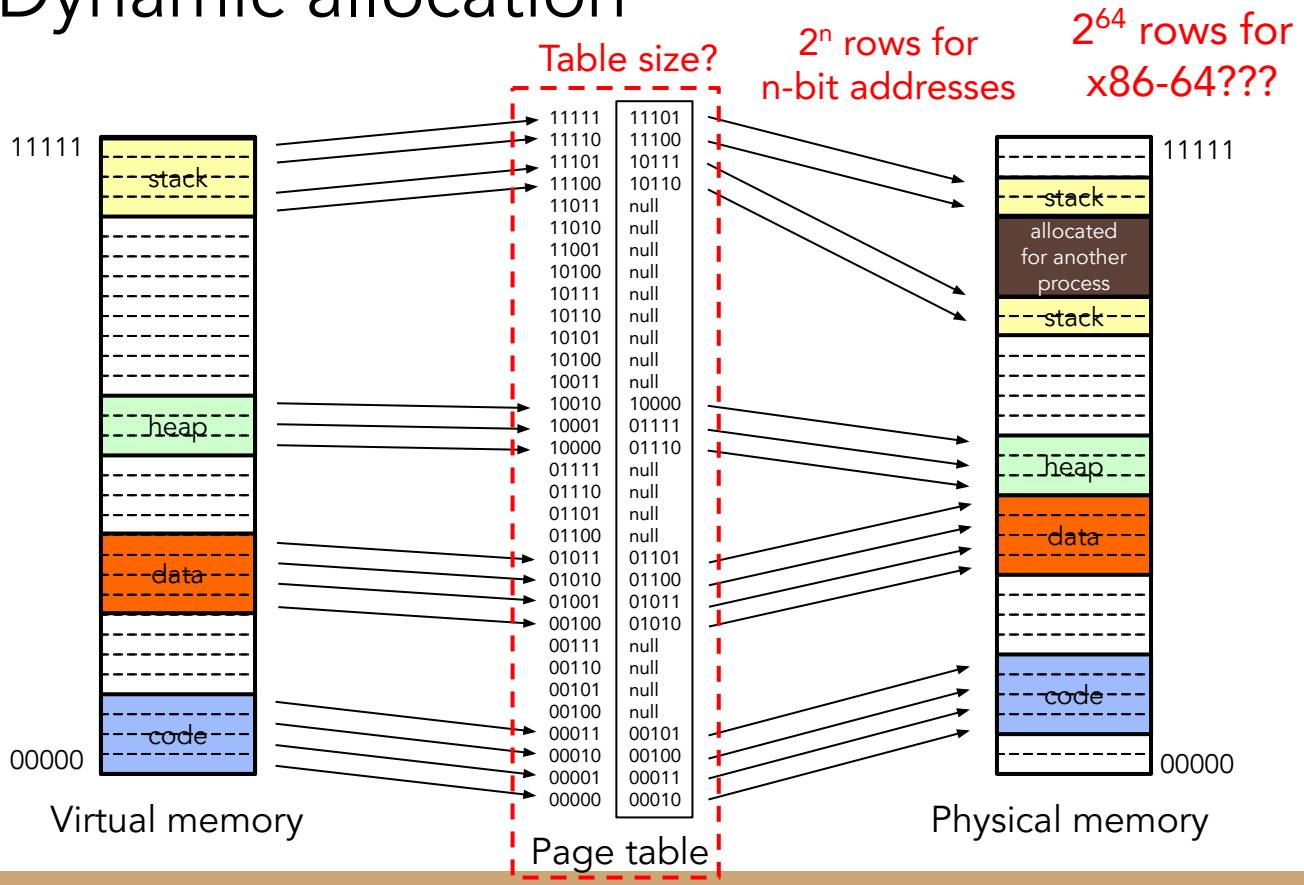
Version 4.0: Dynamic allocation



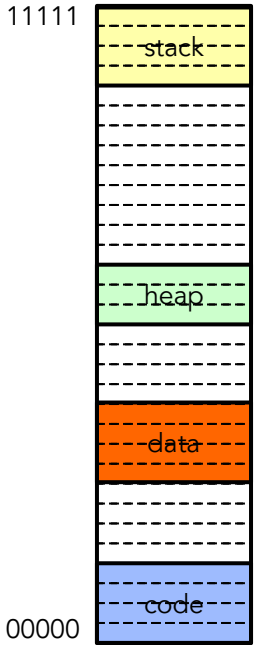
Version 4.0: Dynamic allocation



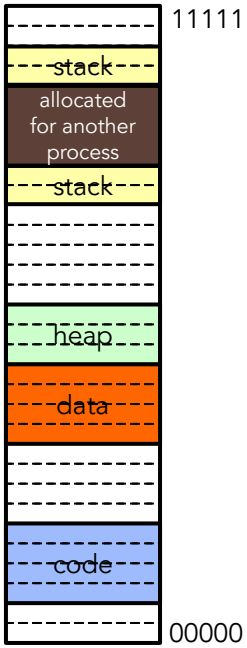
Version 4.0: Dynamic allocation



Version 4.0: 2-level page tables



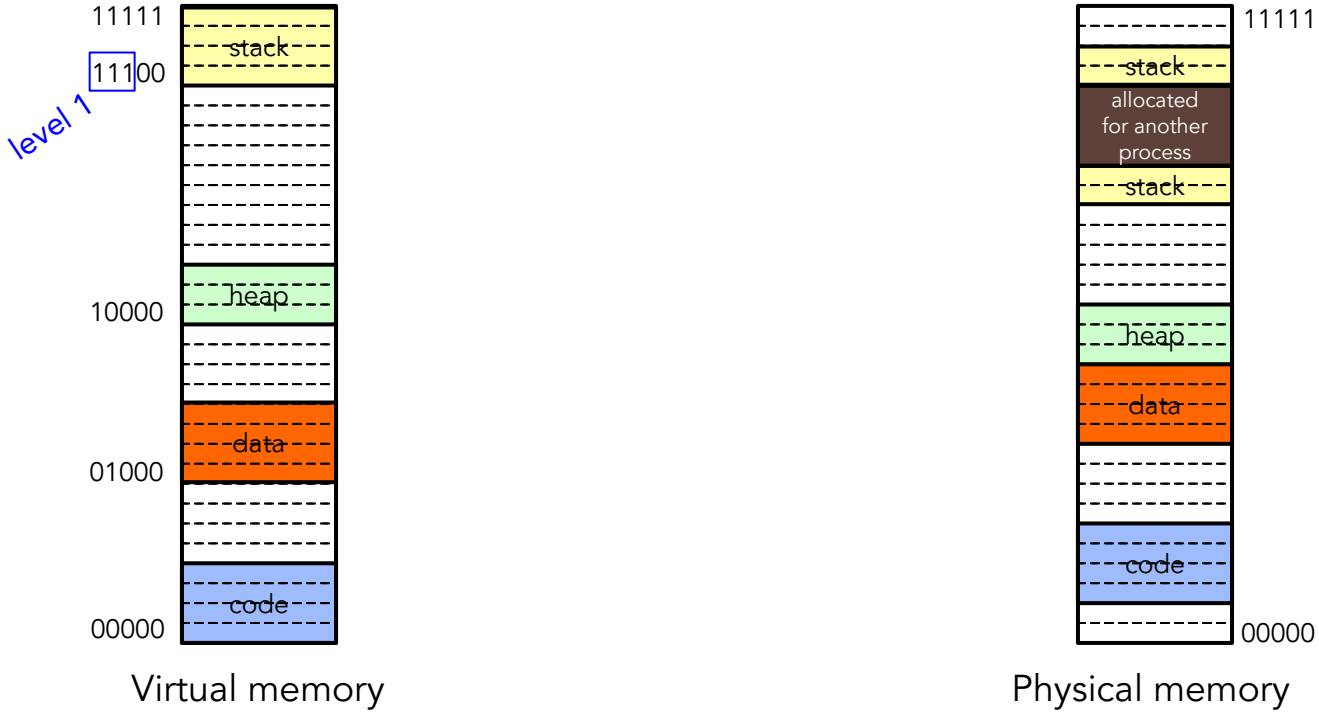
Virtual memory



Physical memory

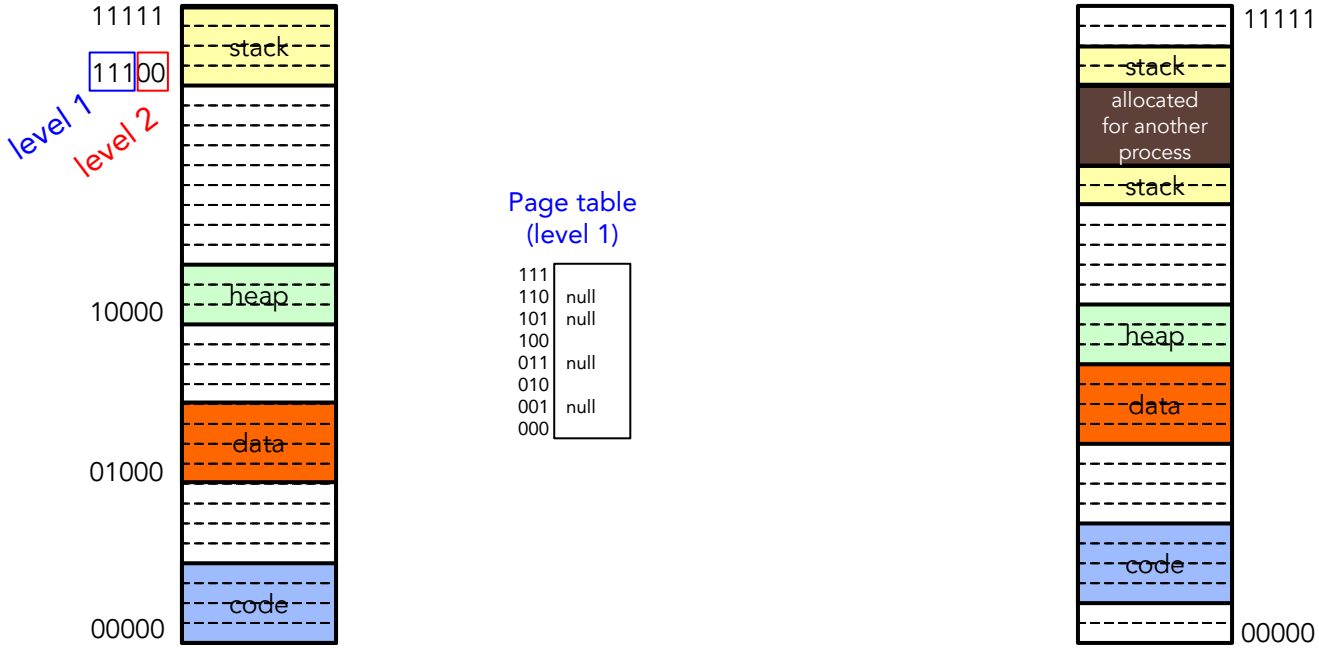
Version 4.0: 2-level page tables

Version 4.0: 2-level page tables

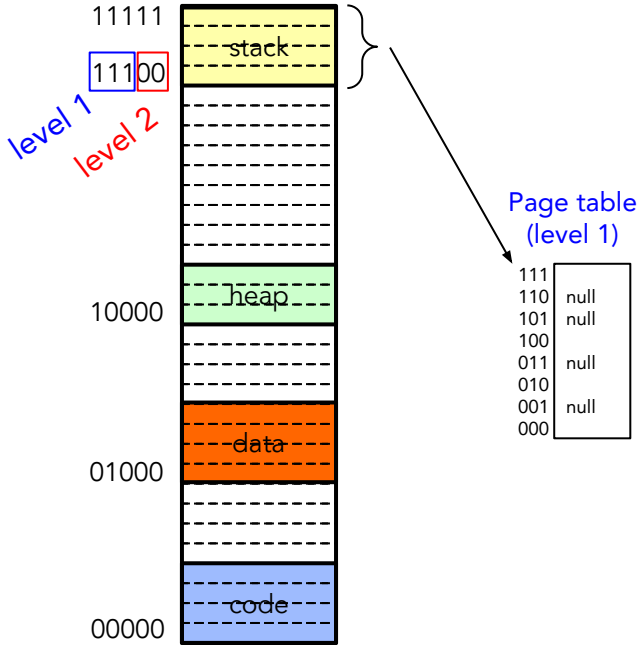


Version 4.0: 2-level page tables

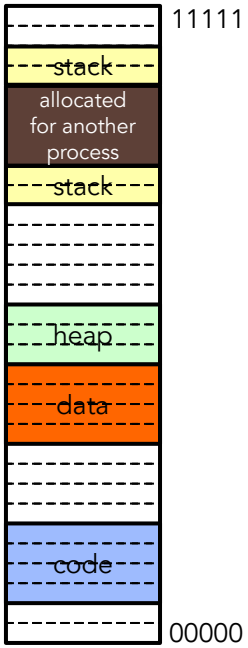
Version 4.0: 2-level page tables



Version 4.0: 2-level page tables

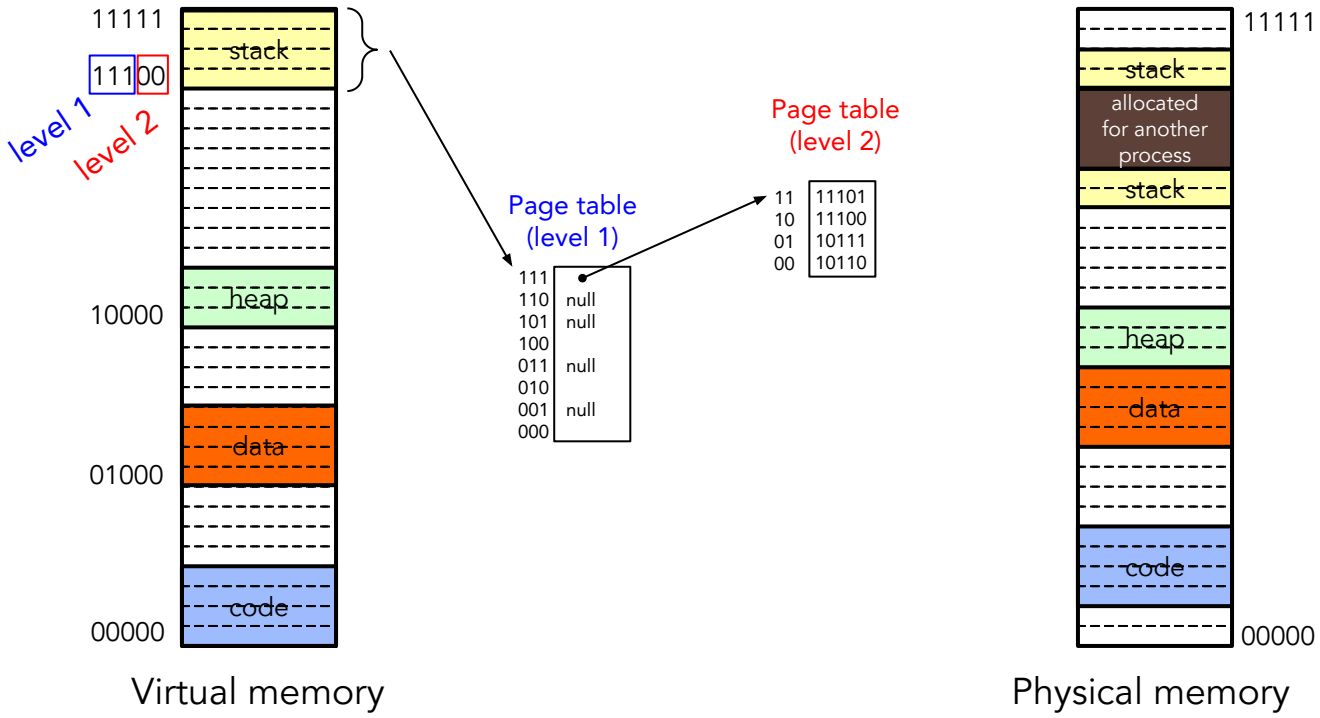


Virtual memory

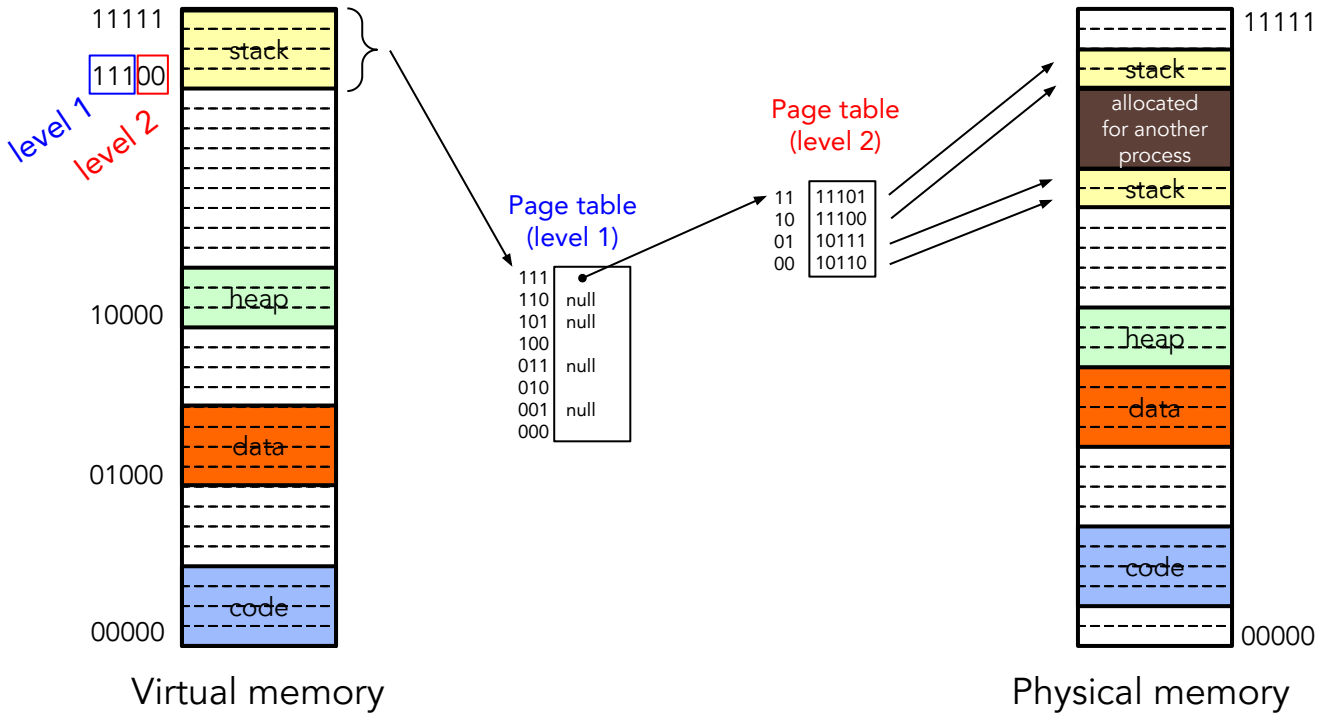


Physical memory

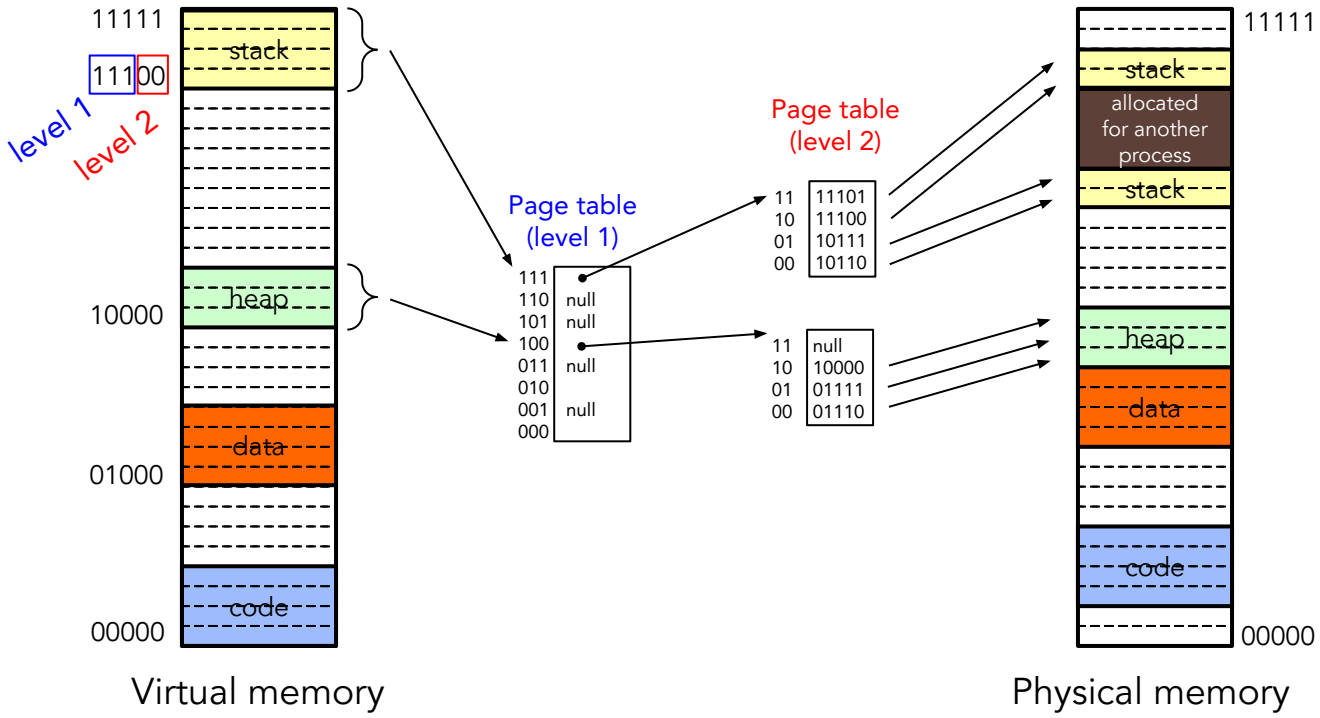
Version 4.0: 2-level page tables



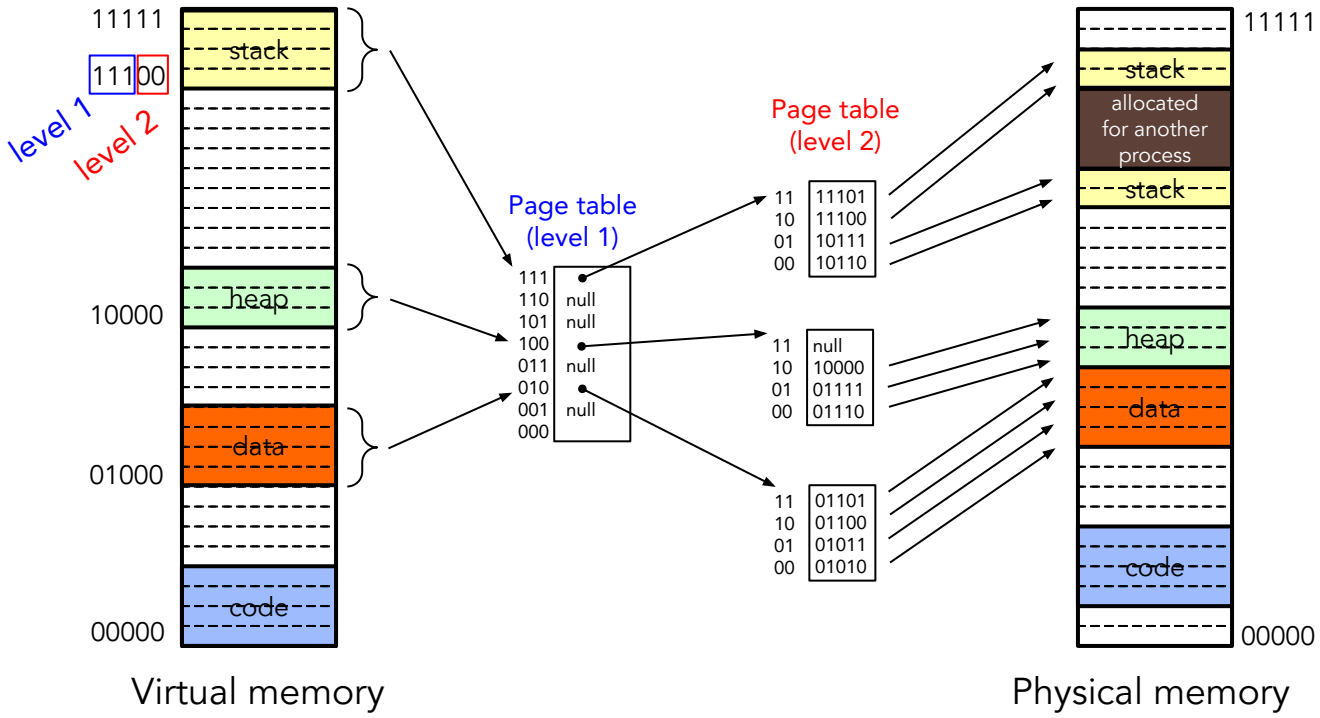
Version 4.0: 2-level page tables



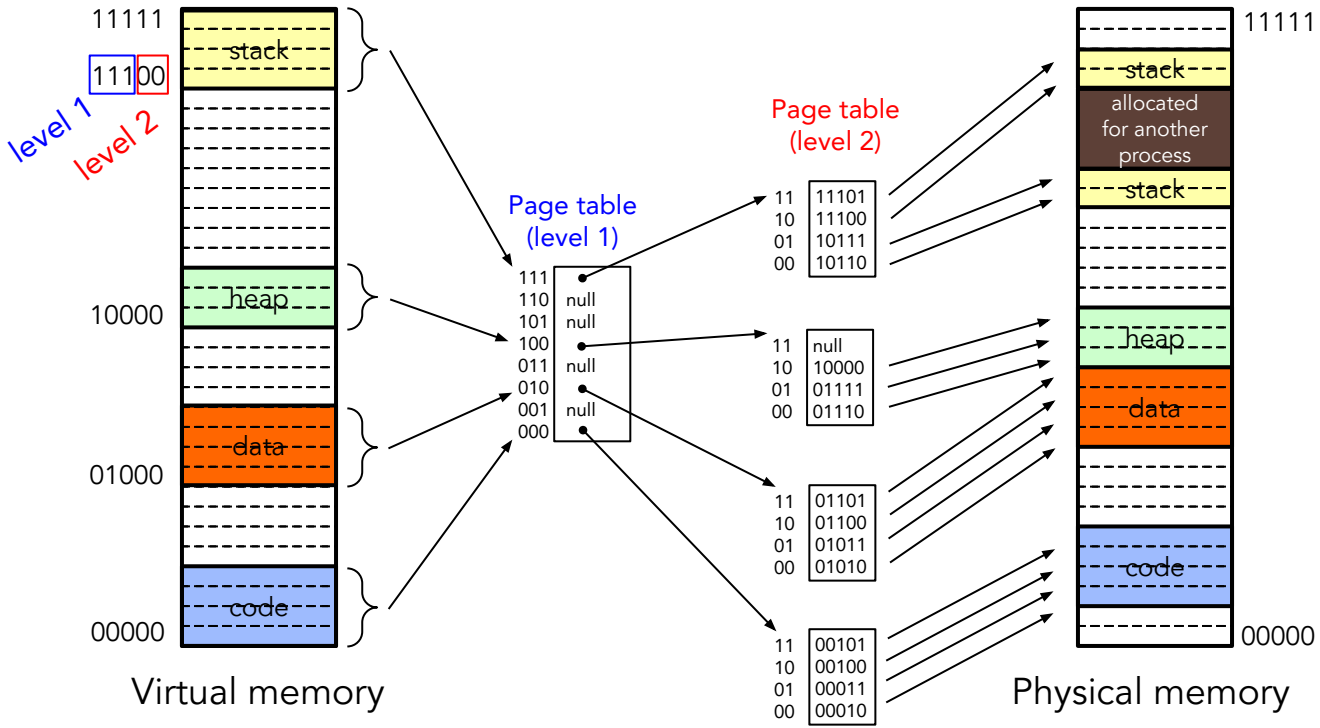
Version 4.0: 2-level page tables



Version 4.0: 2-level page tables

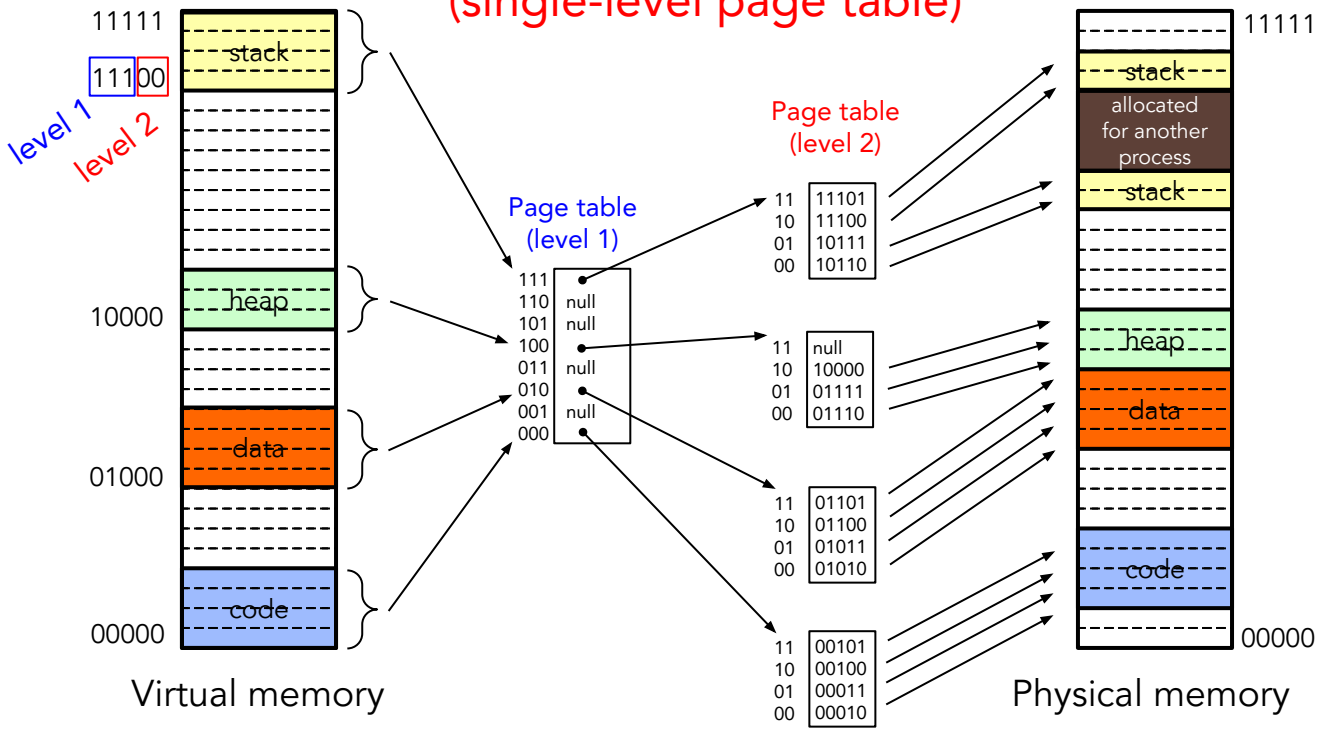


Version 4.0: 2-level page tables



Version 4.0: 2-level page tables

Only need 24 rows (2-level page table) instead of 32 rows (single-level page table)



Version 4.0: Multilevel page tables

- How many levels does Linux kernel use?
  - https://www.kernel.org/doc/html/v5.8/x86/x86_64/mm.html

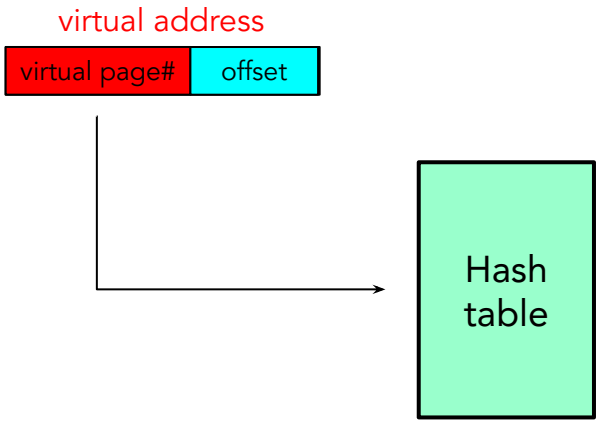
Version 4.0: Inverted page table

Version 4.0: Inverted page table

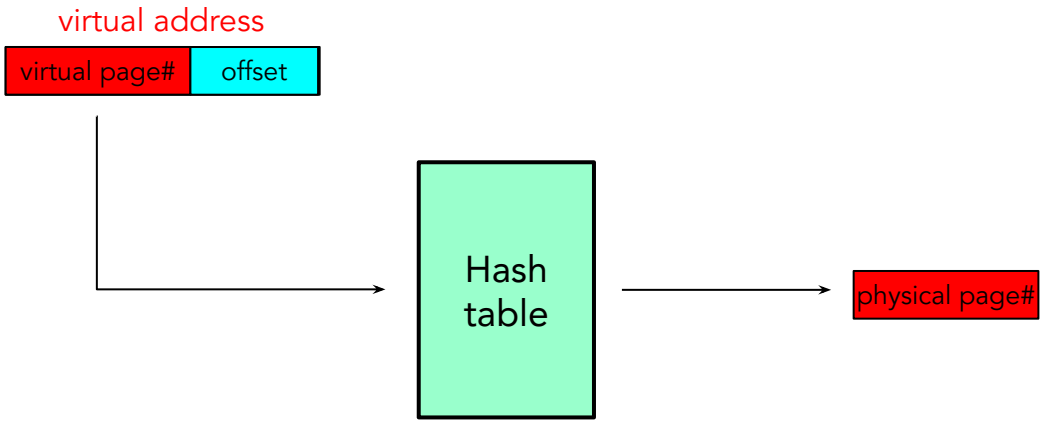
virtual address

virtual page#	offset

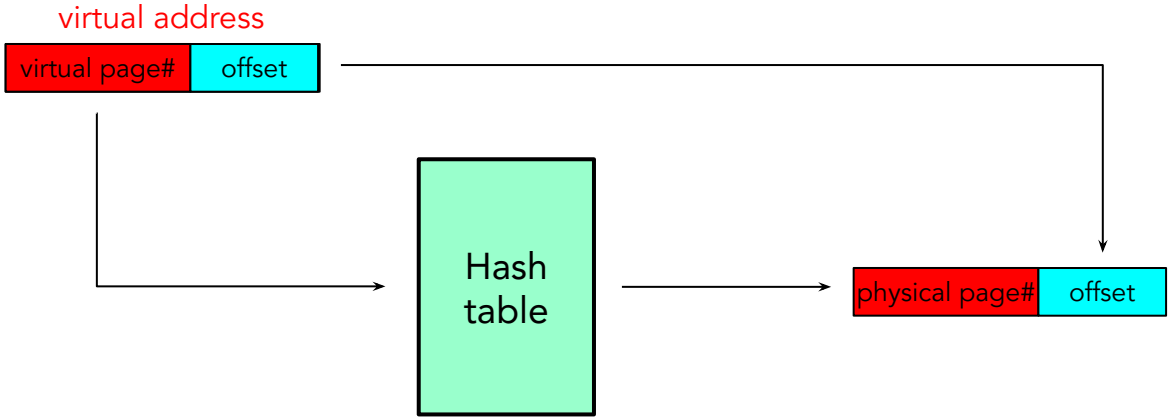
Version 4.0: Inverted page table



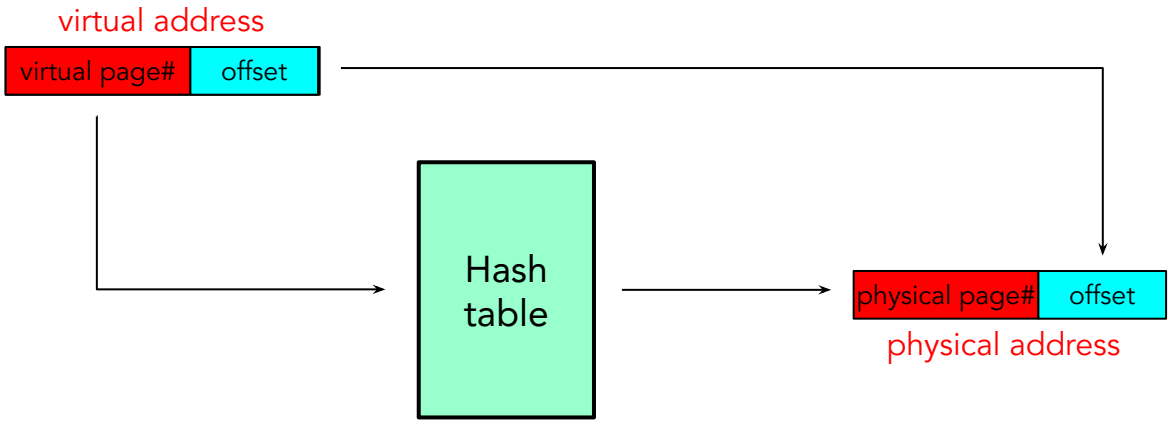
Version 4.0: Inverted page table



Version 4.0: Inverted page table

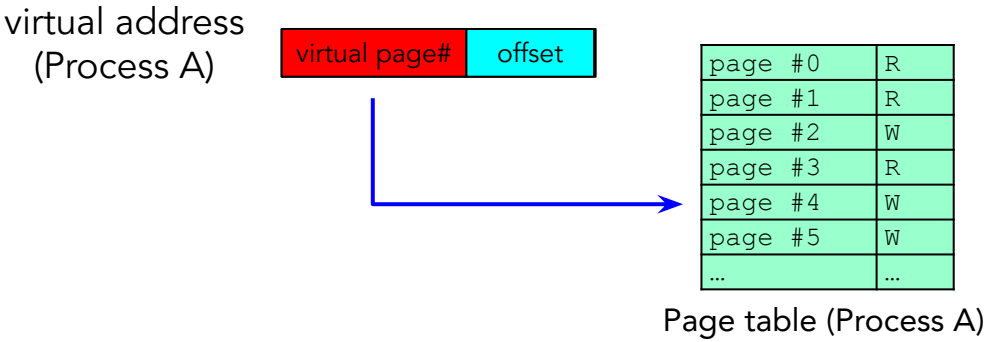


Version 4.0: Inverted page table

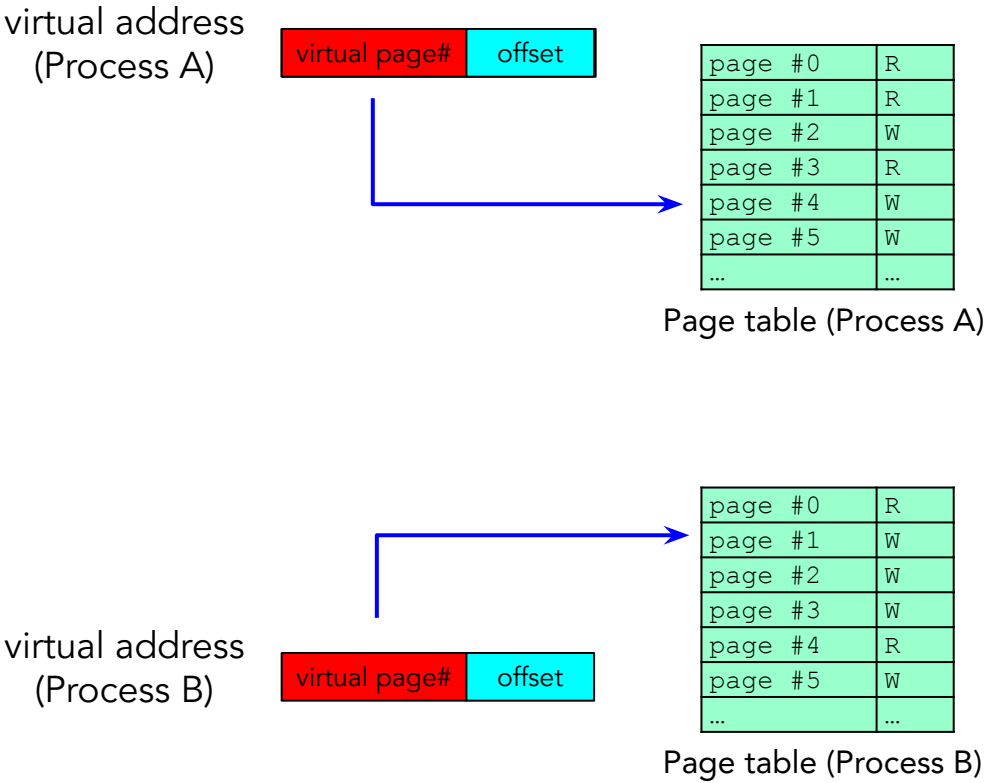


Version 4.0: How about sharing?

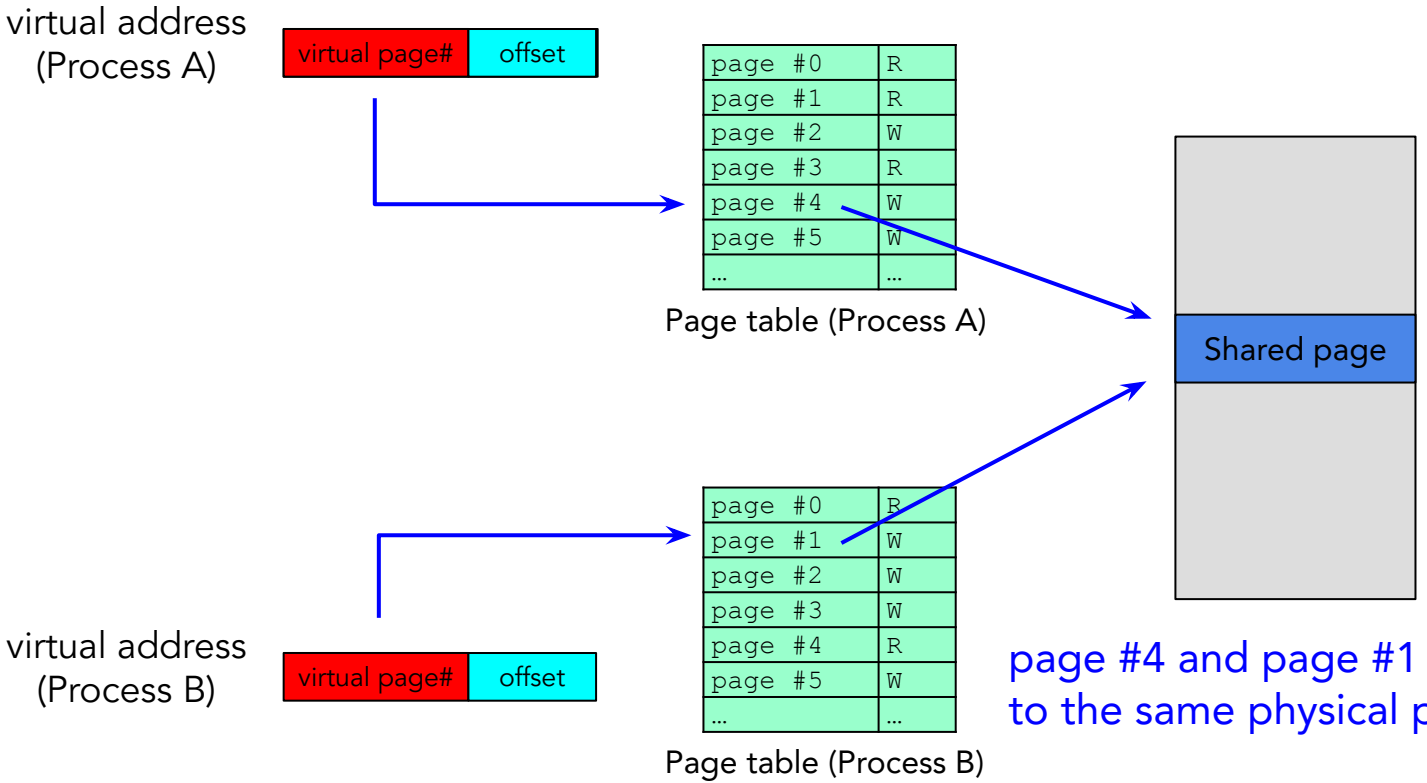
Version 4.0: How about sharing?



Version 4.0: How about sharing?



Version 4.0: How about sharing?



page #4 and page #1 can point to the same physical page

Version 4.0: Paging

- Allocate physical memory in fixed size chunks (“pages”)

Version 4.0: Paging

- Allocate physical memory in fixed size chunks ("pages")

Isolation	*?*
Sharing	*?*
Virtualization	*?*
Expandable	*?*
Relative addresses	*?*
No external fragmentation	*?*
No internal fragmentation	*?*

Version 4.0: Paging

- Allocate physical memory in fixed size chunks ("pages")

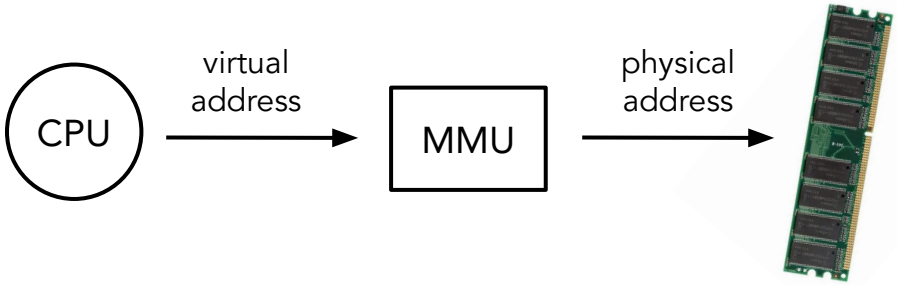
Isolation	✔
Sharing	✔
Virtualization	✗
Expandable	✔
Relative addresses	✔
No external fragmentation	✔
No internal fragmentation	✔

Version 4.0: Paging

➢ Allocate physical memory in fixed size chunks ("pages")

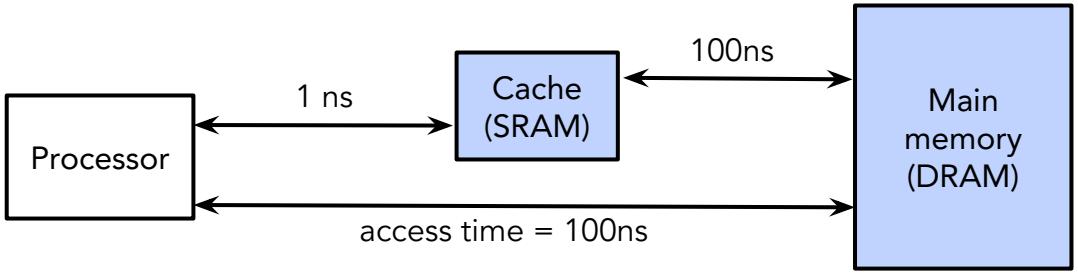
Isolation	✔
Sharing	✔
Virtualization	✗
Expandable	✔
Relative addresses	✔
No external fragmentation	✔
No internal fragmentation	✔

So far …



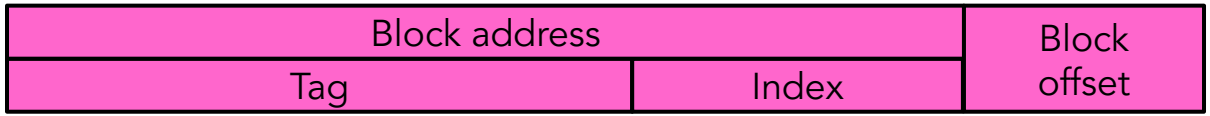
- The Principle of Locality
  - Program likely to access a relatively small portion of the address space at any instant of time.
  - Temporal Locality (Locality in Time):
    - Keep recently accessed data items closer to processor
  - Spatial Locality (Locality in Space):
    - Move contiguous blocks to the upper levels

Caching



- Cache
  - a repository for copies that can be accessed more quickly than the original
- Average Memory Access Time (AMAT) = (Hit Rate x HitTime) + (Miss Rate x MissTime)
- Examples:
  - HitRate = 50% => AMAT = ???
  - HitRate = 90% => AMAT = ???
  - HitRate = 99% => AMAT = ???

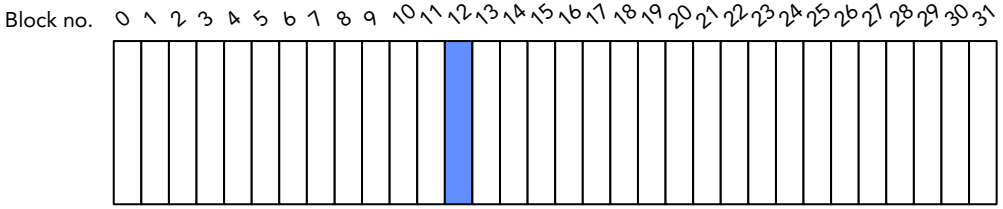
Cache block



- Index Used to Lookup Candidates in Cache
  - Index identifies the set
- Tag used to identify actual copy
  - If no candidates match, then declare cache miss

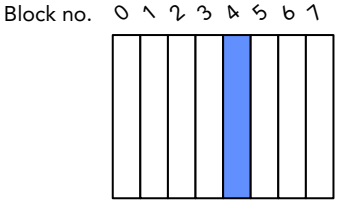
Where does a block get placed in a cache?

➢ Example



32-block address space

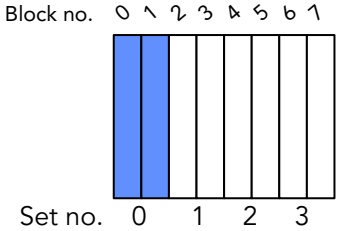
Where can we place the block 12 into a 8-block cache?



Direct mapped

block 12 can only go to specific cache block

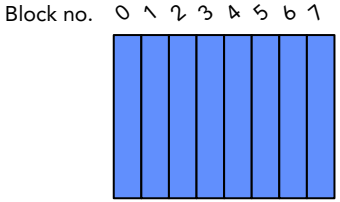
e.g., cache block = memory block mod 12



Set associative

block 12 can only go to a set of cache blocks

e.g., cache block set = memory block mod 4

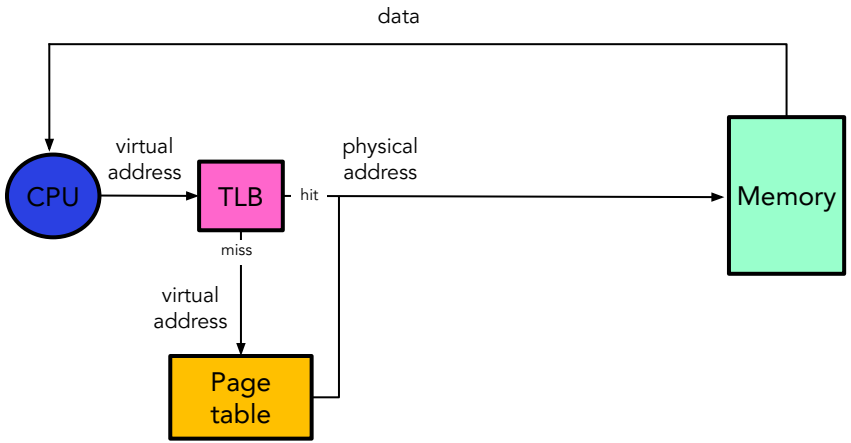


Fully associative

block 12 can go anywhere

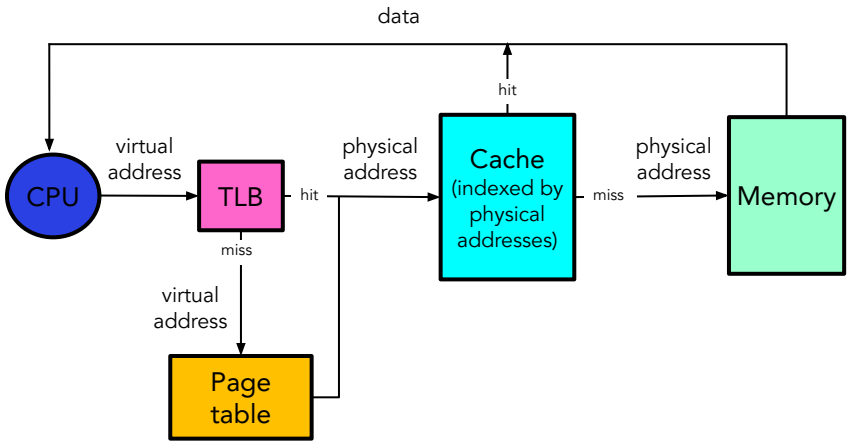
Translation look-aside buffer (TLB)

- Cache of virtual to physical page translations
  - Cache results of recent translations



Translation look-aside buffer (TLB)

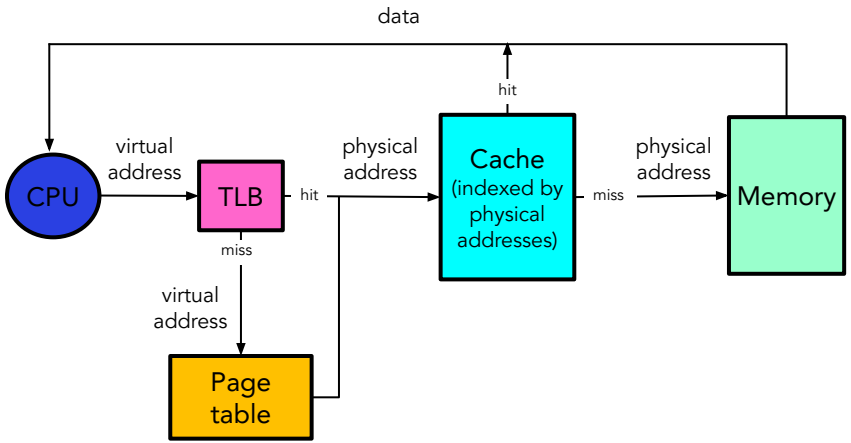
- Cache of virtual to physical page translations



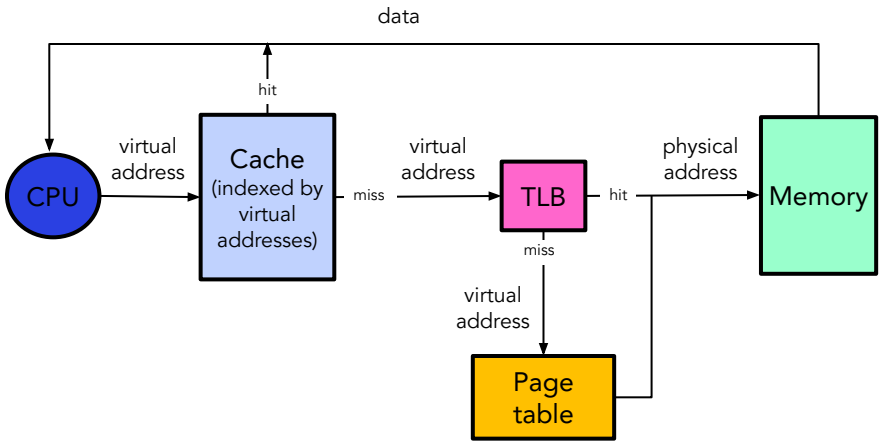
Physically-indexed caches

Translation look-aside buffer (TLB)

- Cache of virtual to physical page translations



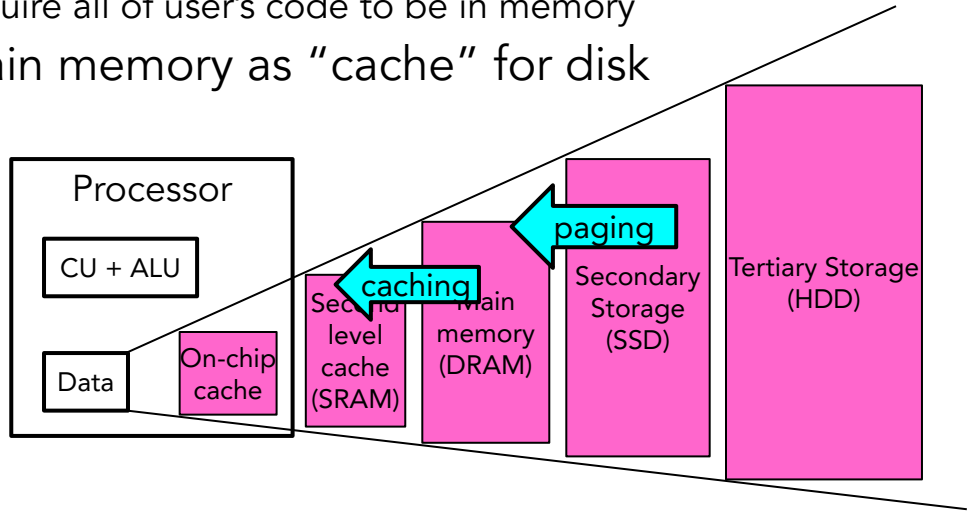
Physically-indexed caches



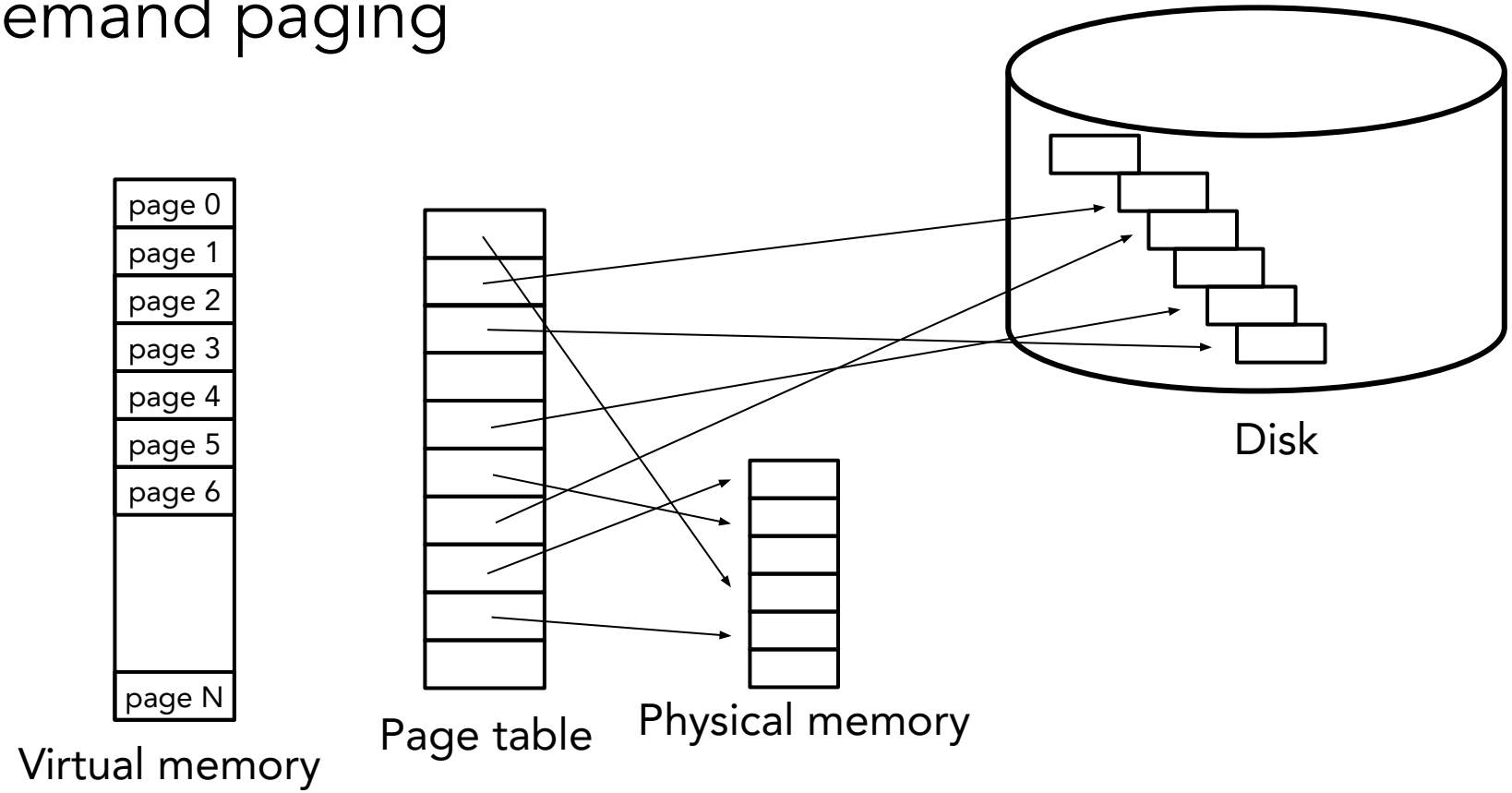
Virtually-indexed caches

Demand paging

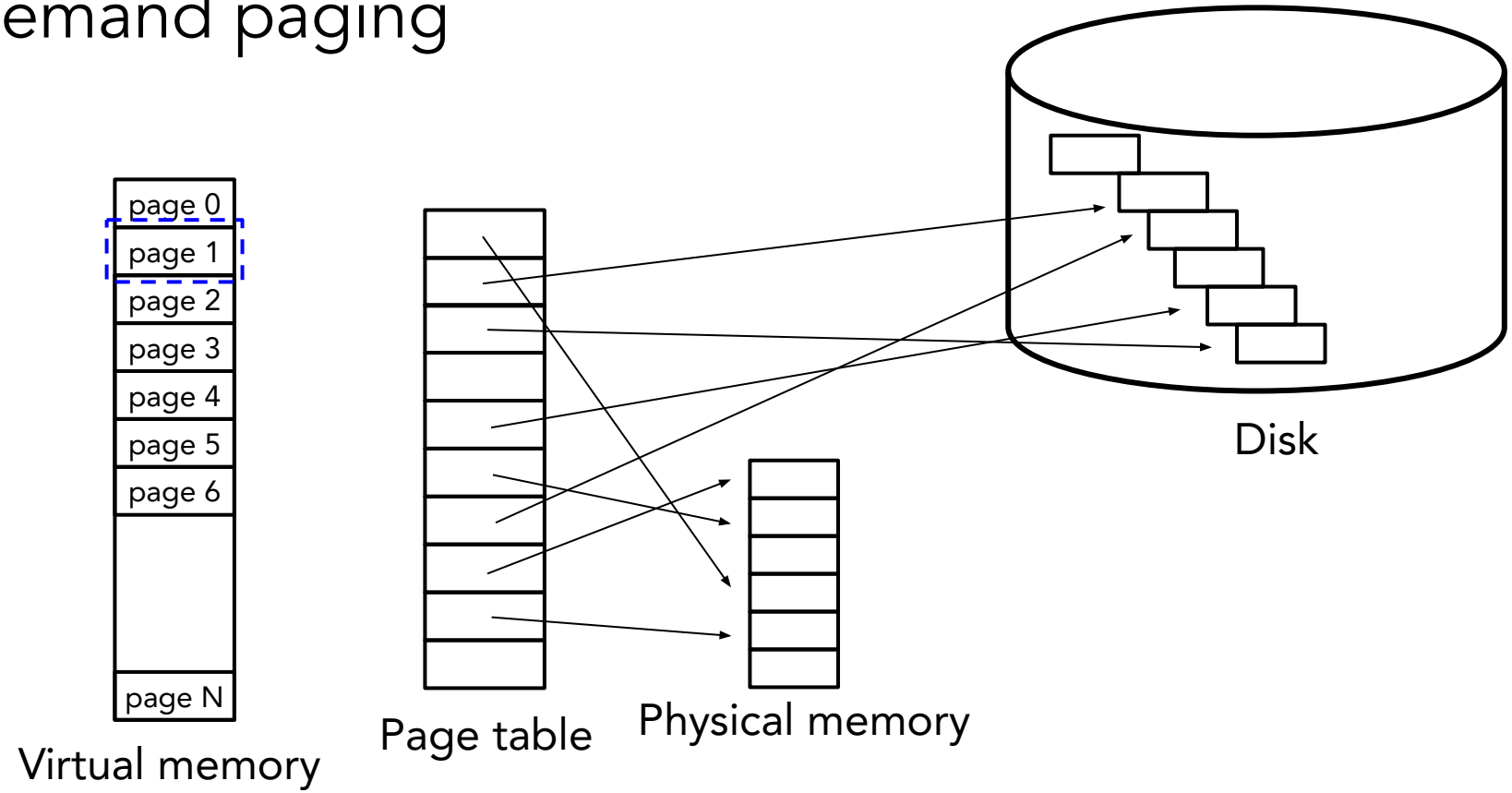
- Modern programs require a lot of physical memory
  - Memory per system growing faster than 25%-30%/year
- But they don't use all their memory all of the time
  - 90-10 rule: programs spend 90% of their time in 10% of their code
  - Wasteful to require all of user's code to be in memory
- Solution: use main memory as "cache" for disk



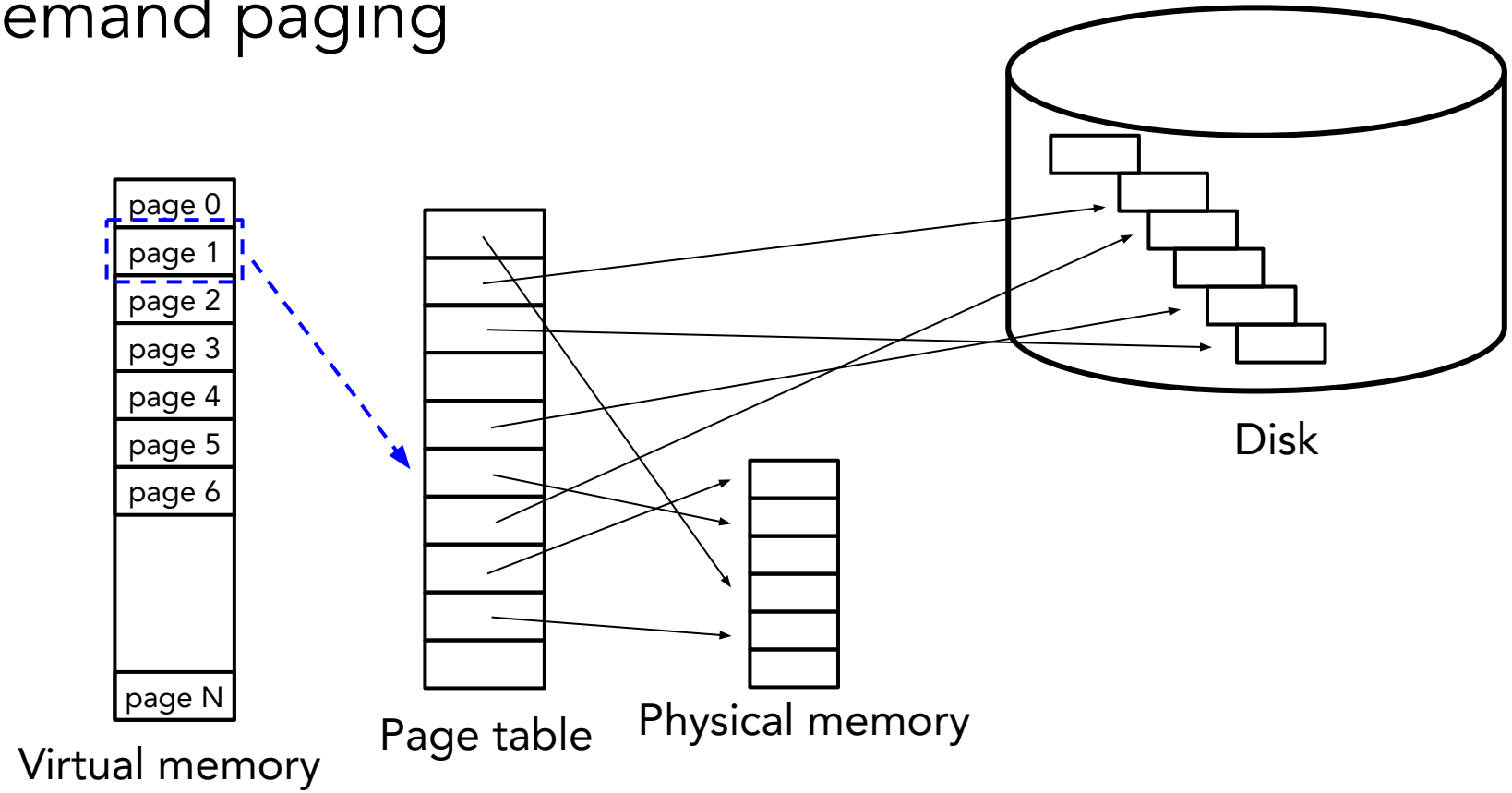
Demand paging



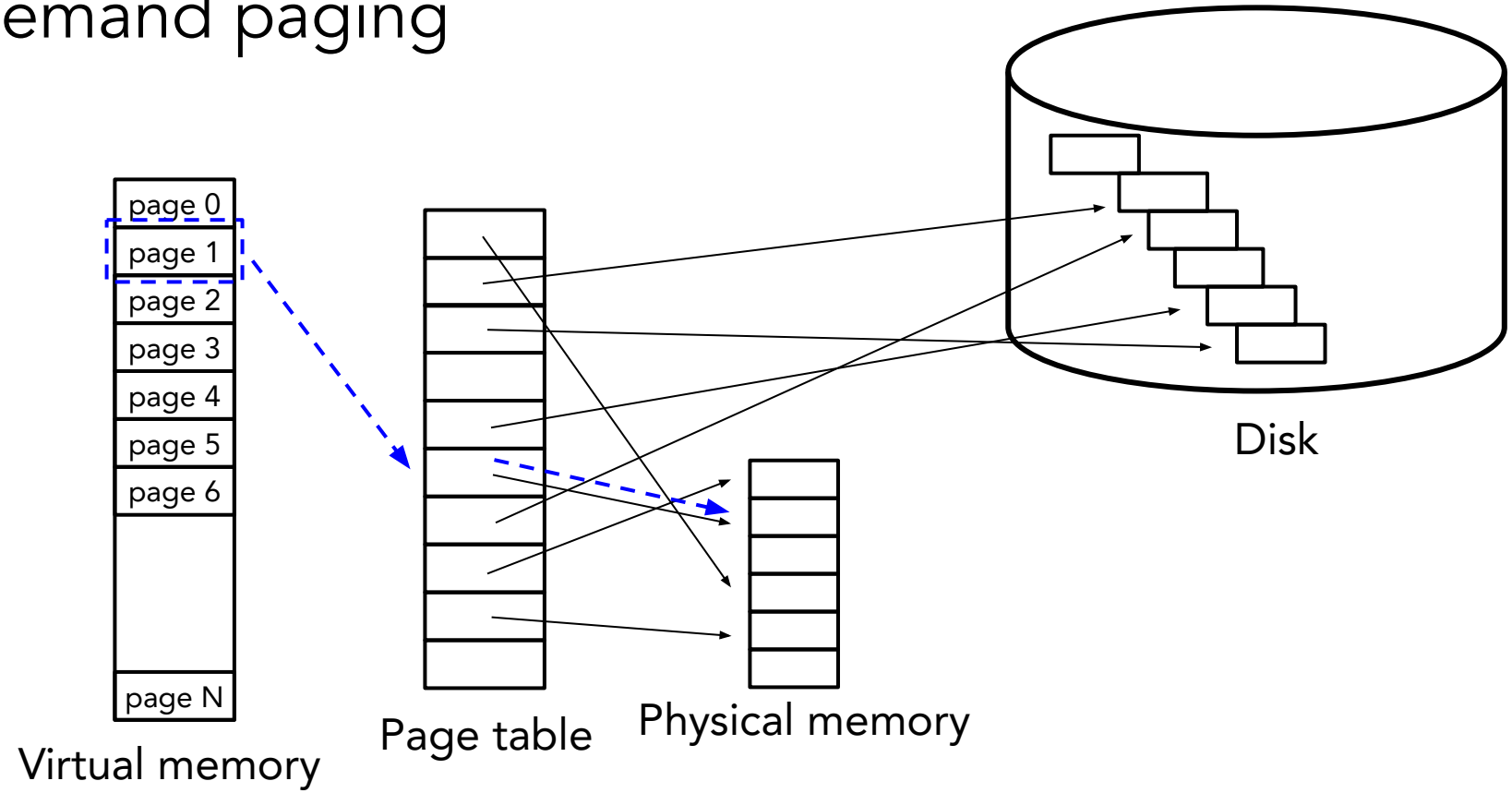
Demand paging



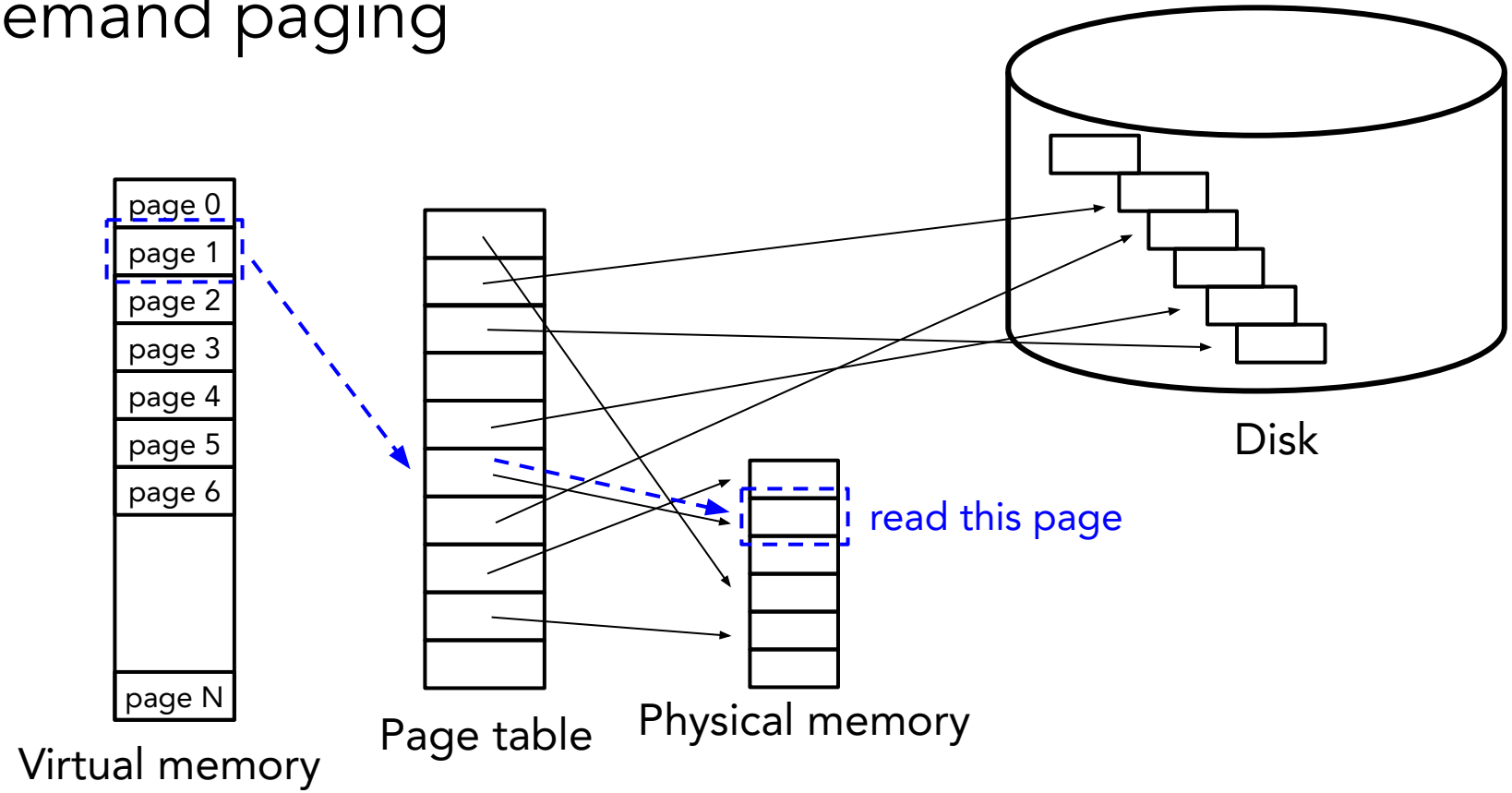
Demand paging



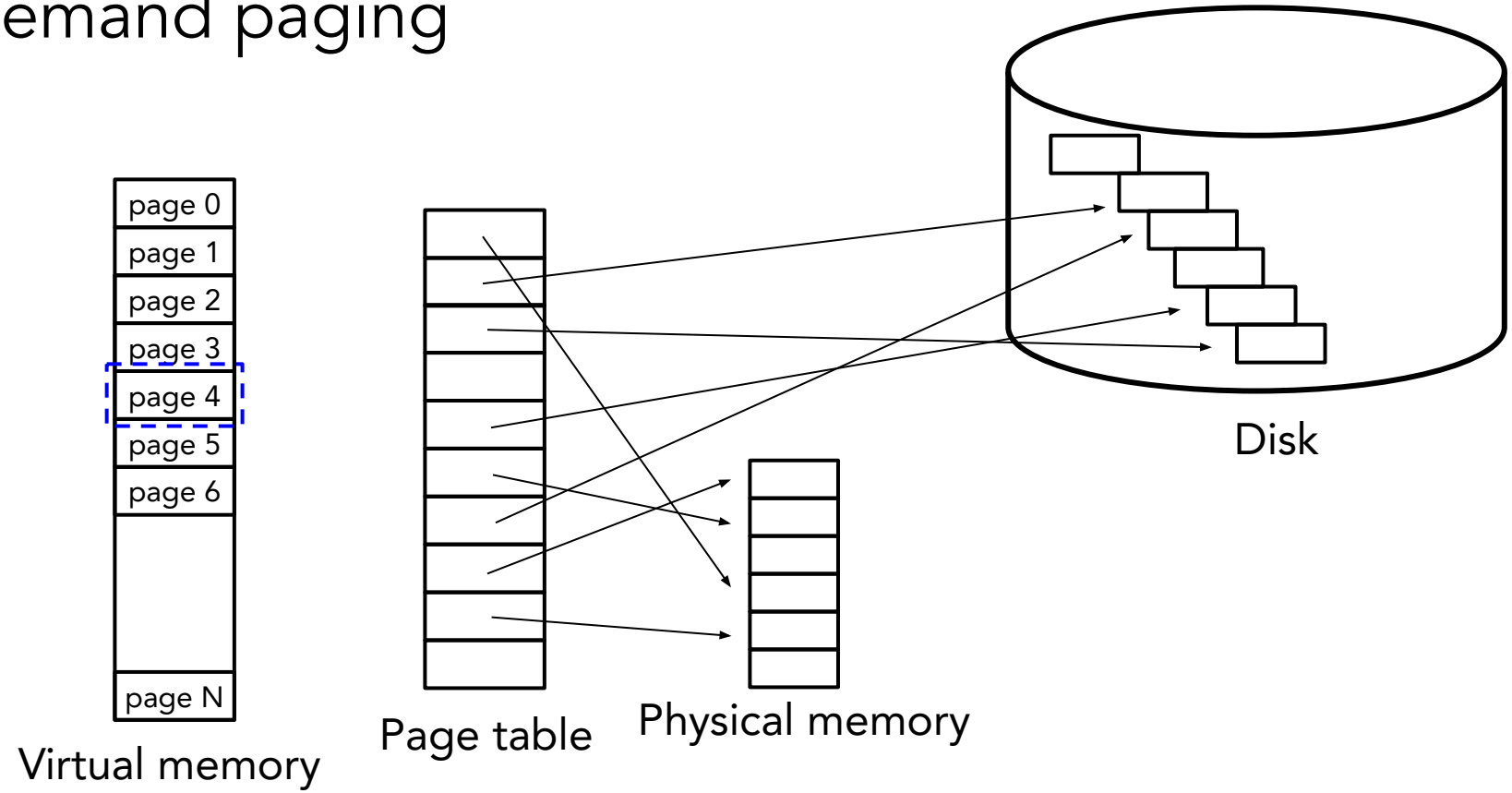
Demand paging



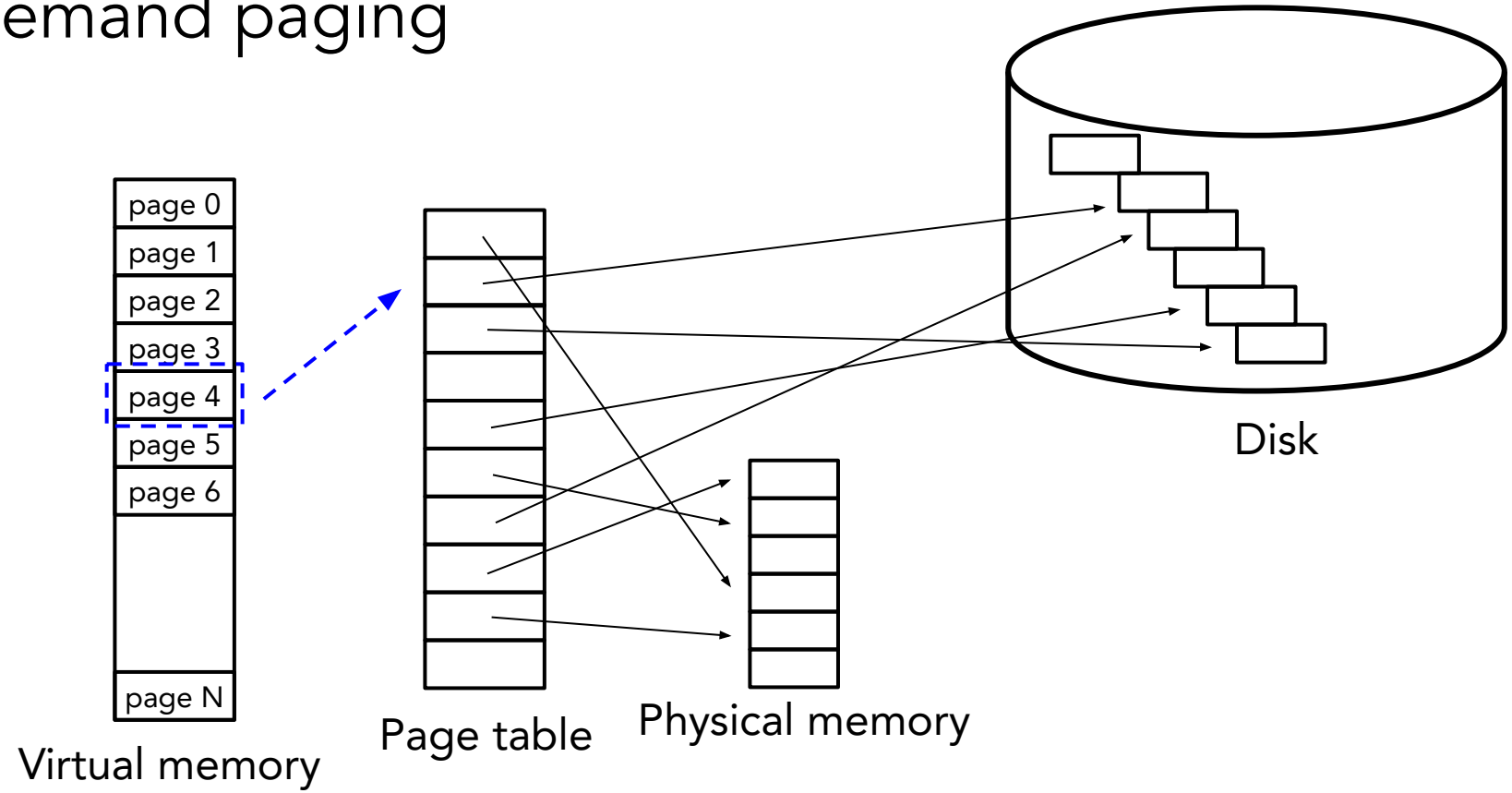
Demand paging



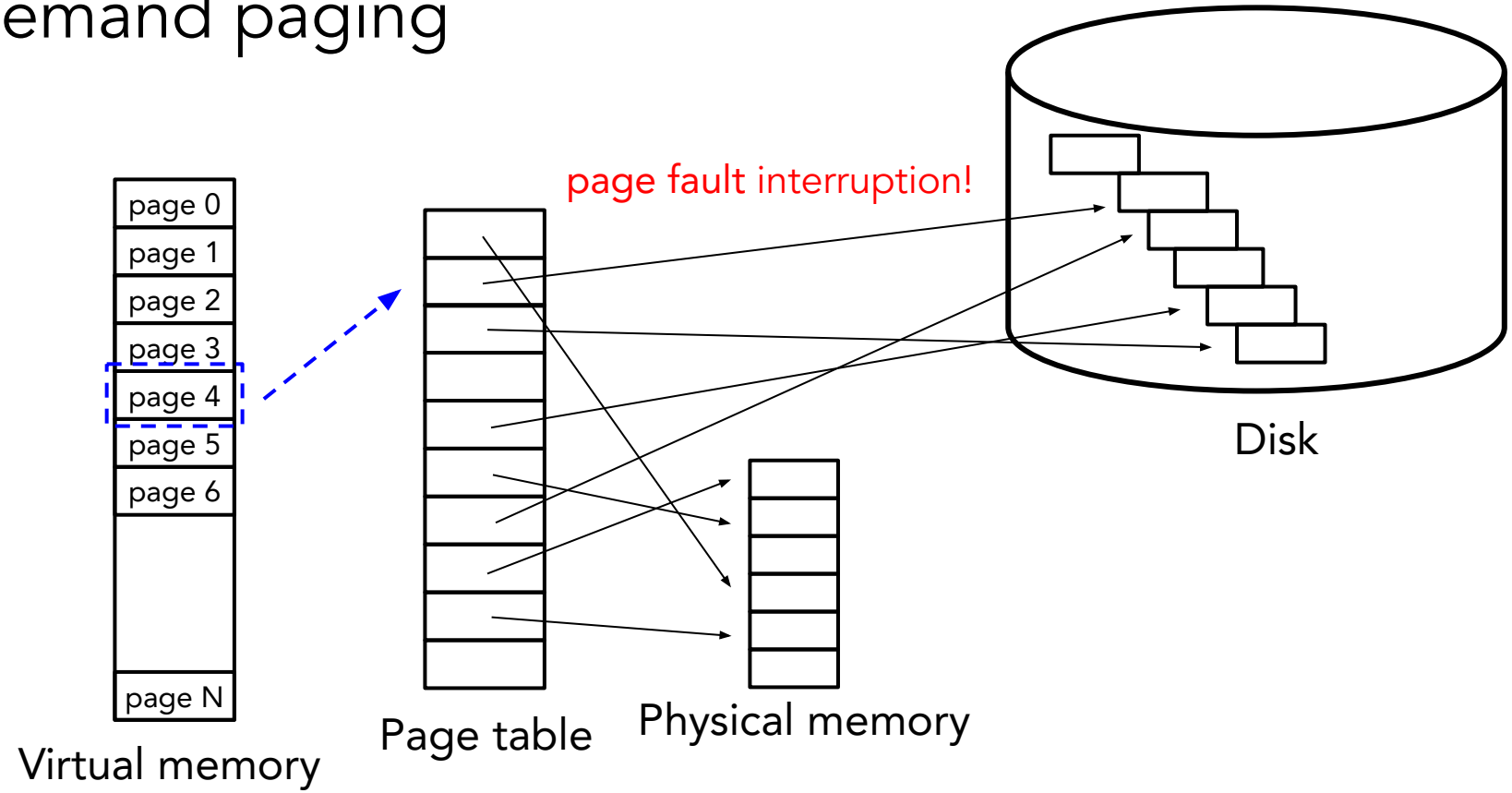
Demand paging



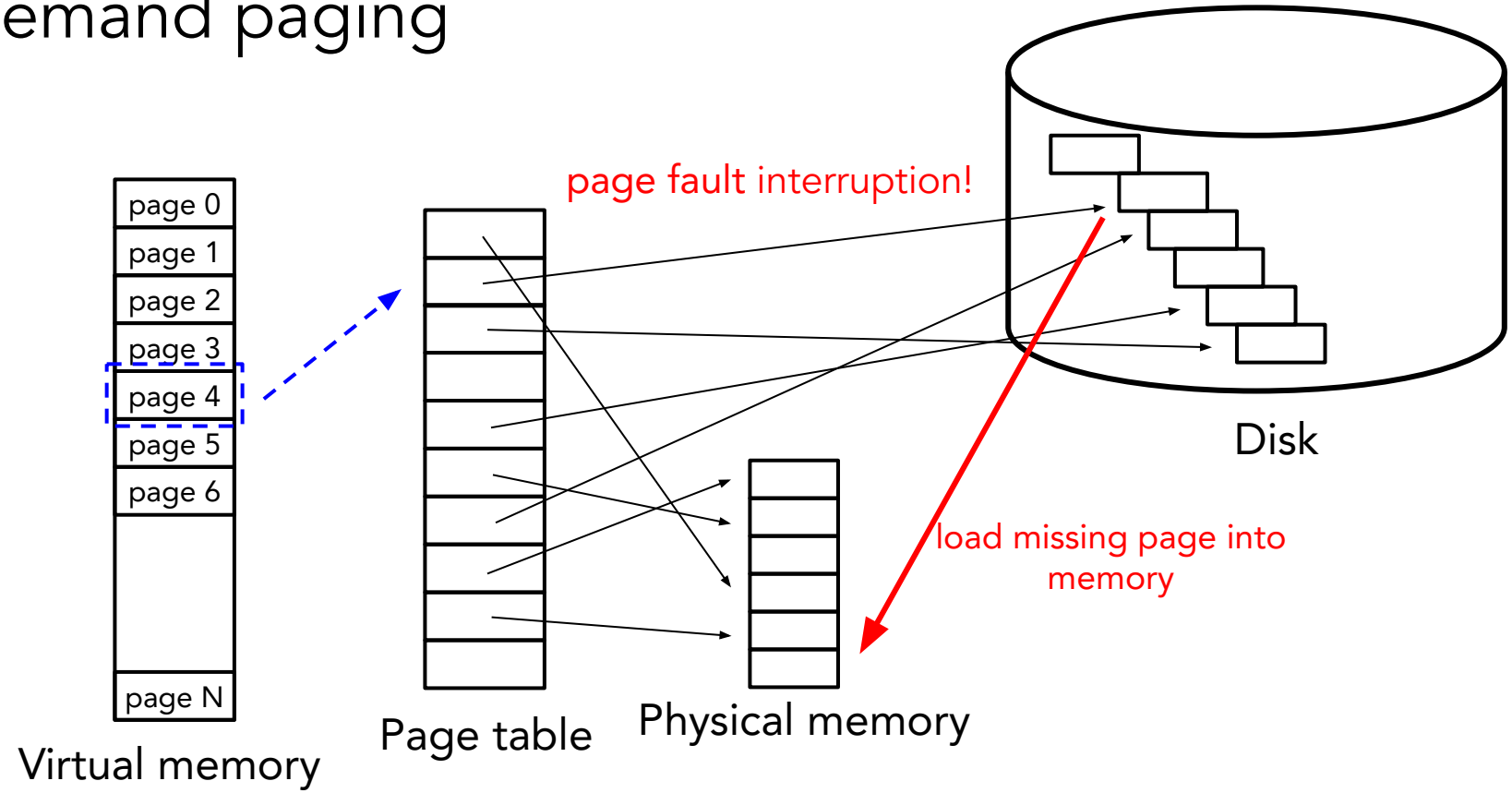
Demand paging



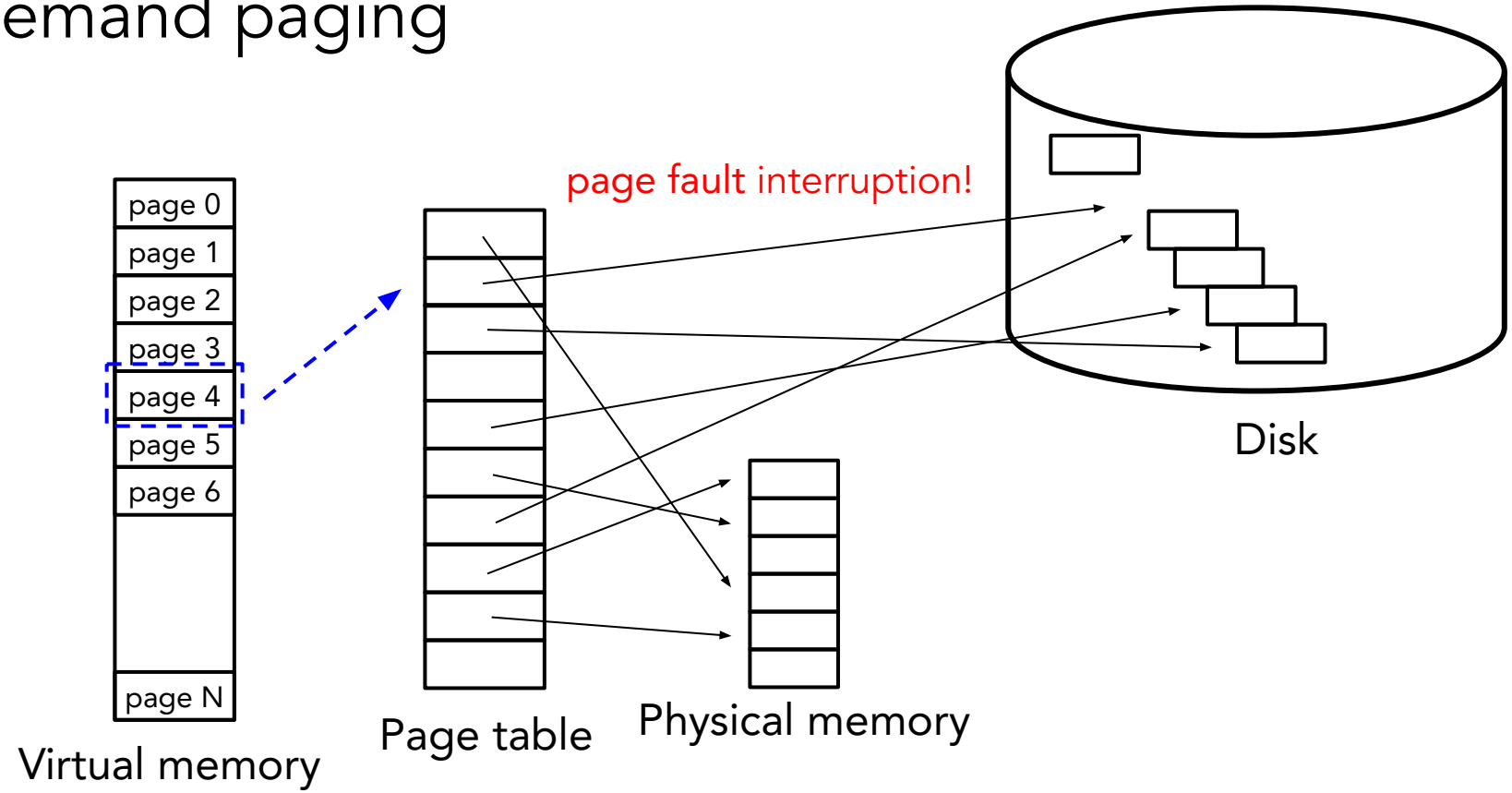
Demand paging



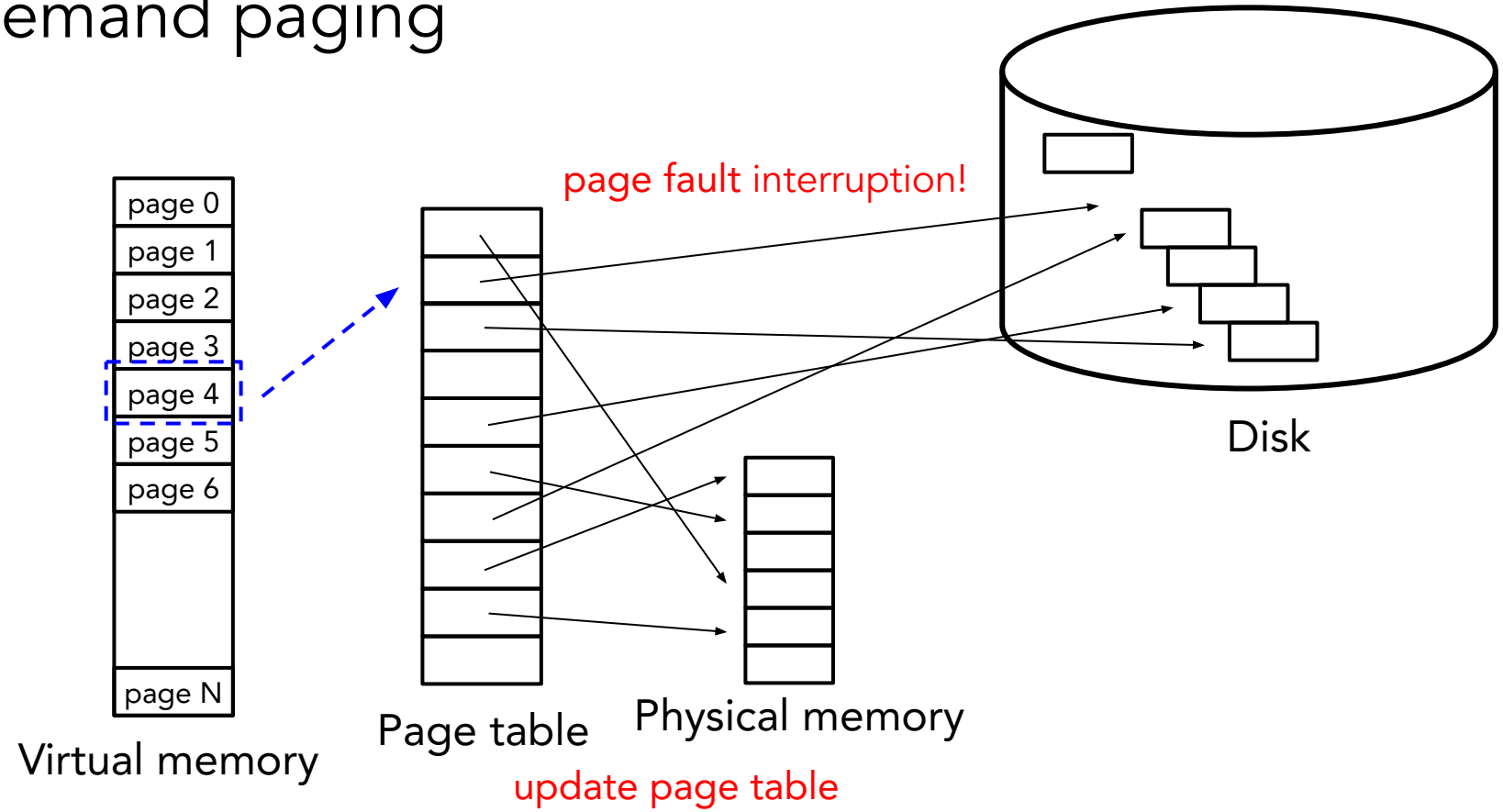
Demand paging



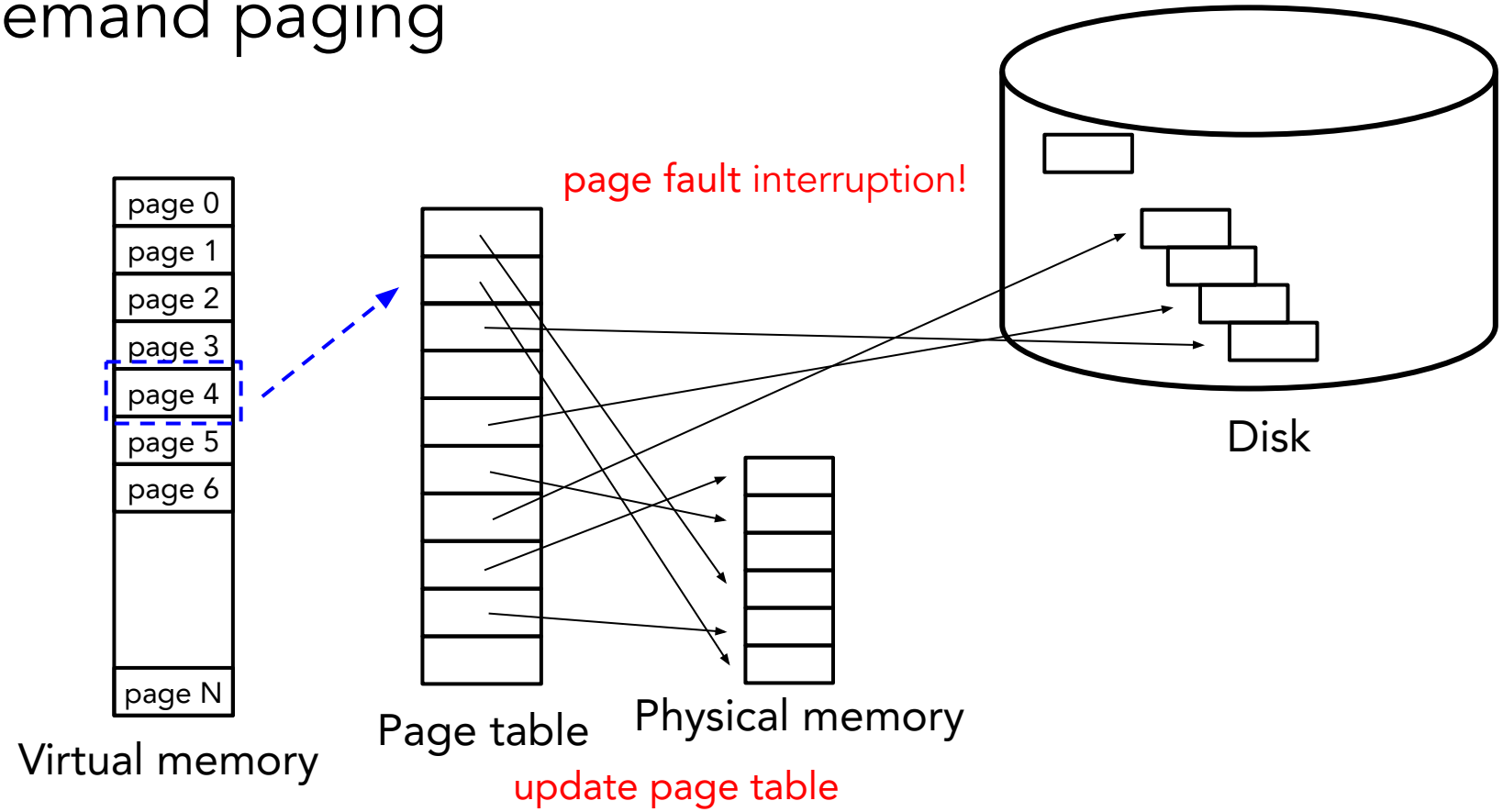
Demand paging



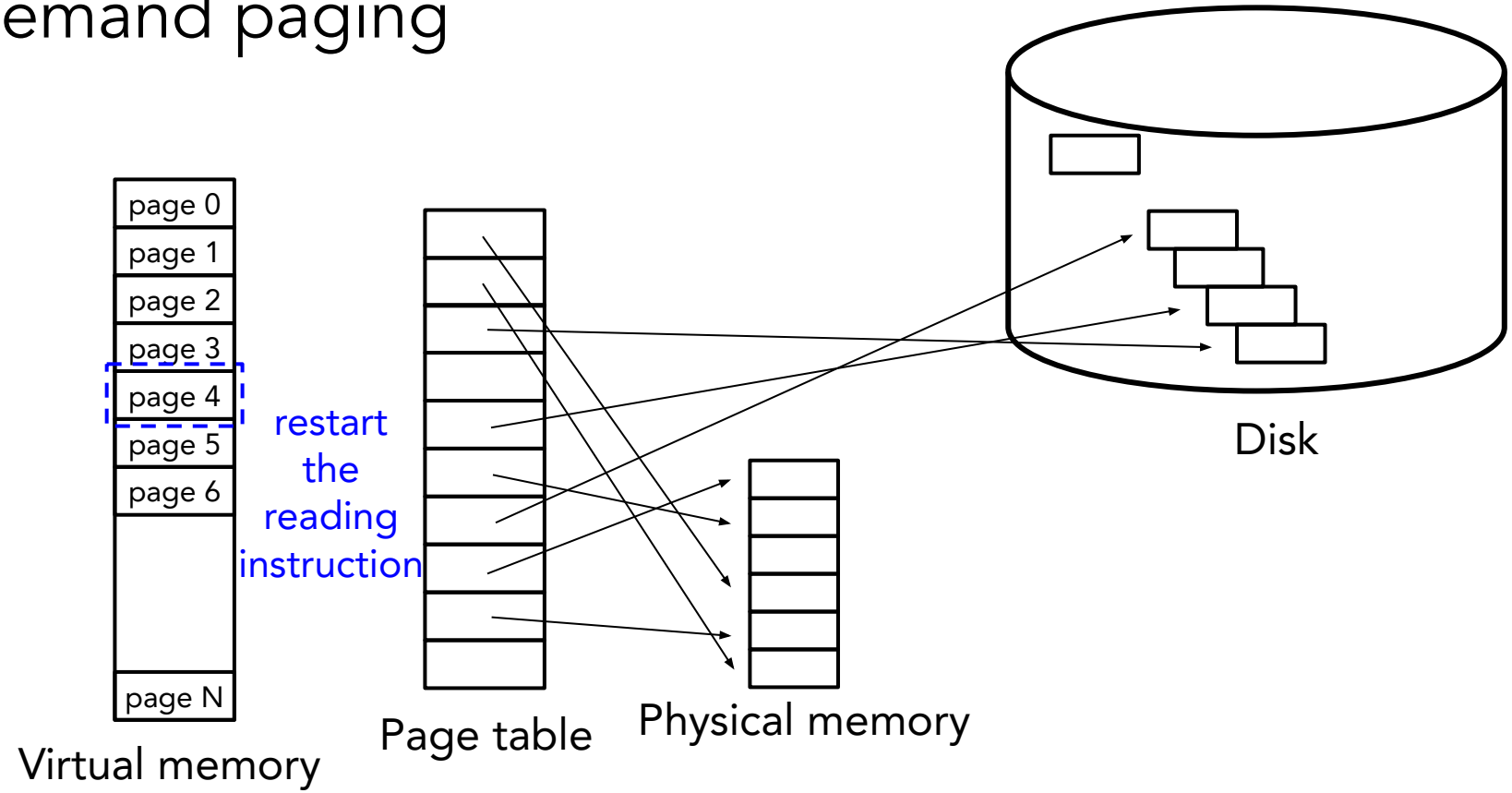
Demand paging



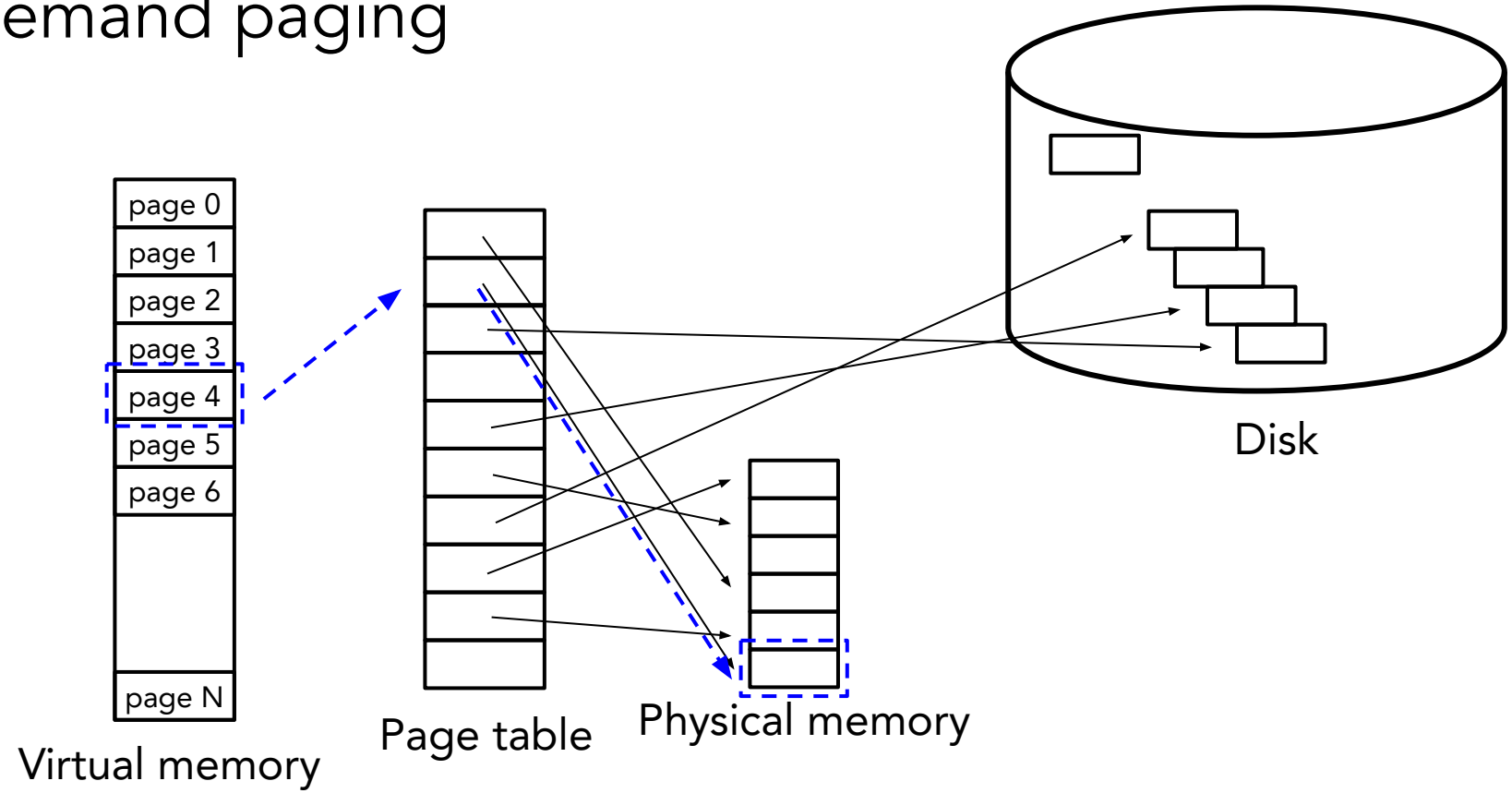
Demand paging



Demand paging



Demand paging



What factors lead to misses in page cache?

- Compulsory misses
  - Pages that have never been placed into memory before
- Capacity misses
  - Not enough memory
- Conflict misses
  - Several addresses map to the same set and evict blocks that are still needed
- Policy misses
  - Pages were in memory before but kicked out because of the replacement policy
  - How to fix?
    - Better replacement policy

Page replacement algorithms

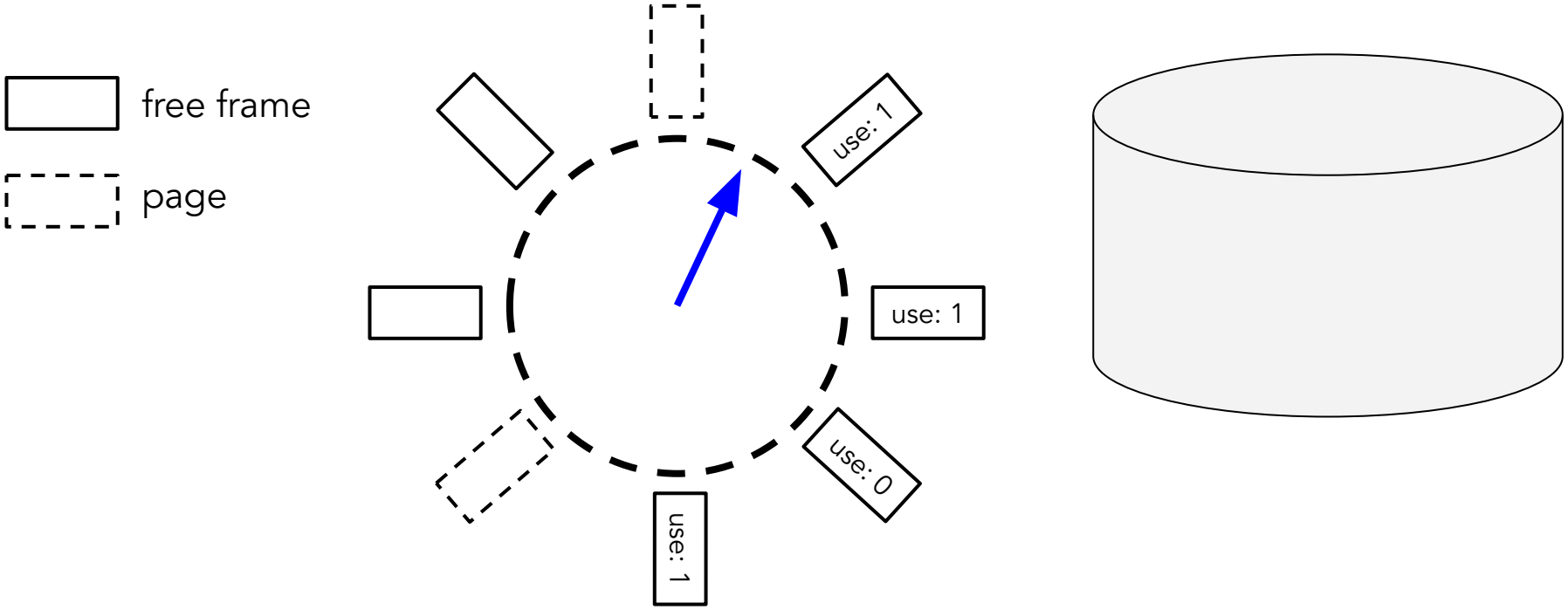
- Random: Pick any page to eject at random
  - Used mainly for comparison
- FIFO: The page brought in earliest is evicted
  - Ignores usage
- MIN (Minimum)
  - Replace page that won't be used for the longest time
- LRU (Least Recently Used):
  - Replace page that hasn't been used for the longest time

Clock algorithm

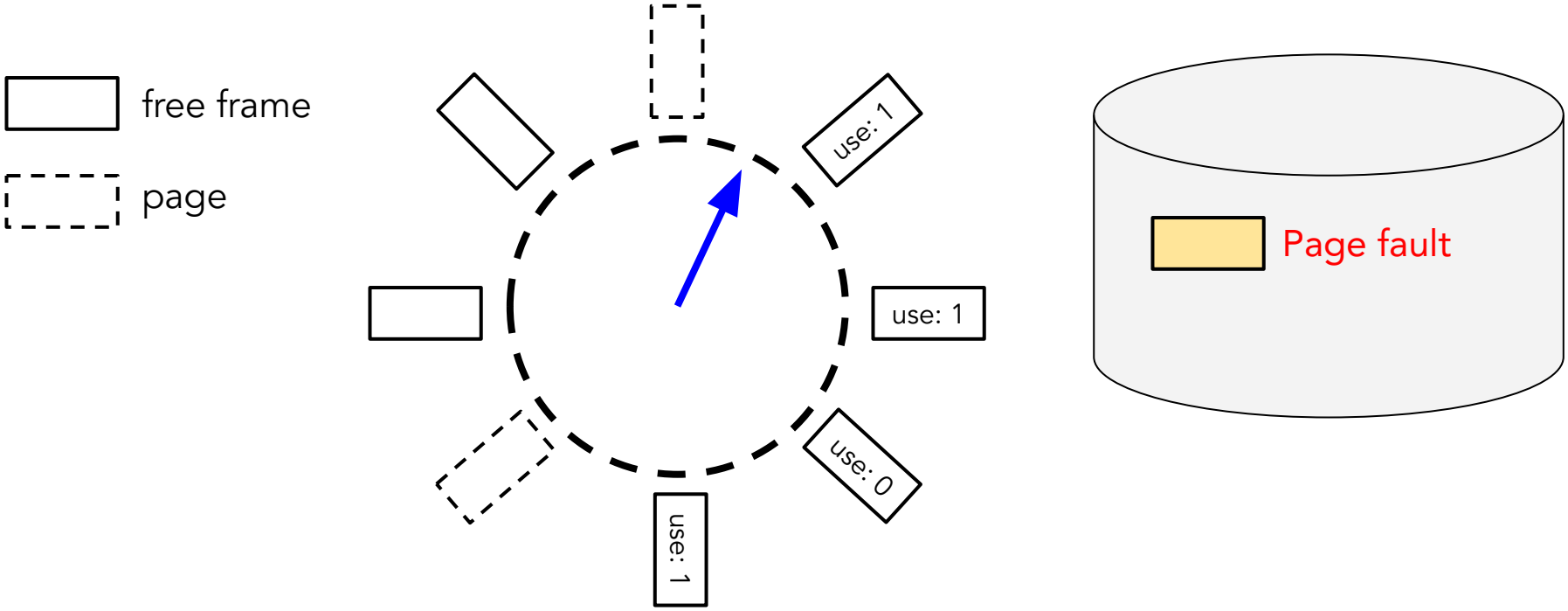
Clock algorithm

- Arrange physical pages in circle with single clock hand
  - Approximate LRU
  - Replace an old page, not the oldest page
- On page fault
  - Advance clock hand (not real time)
  - Check use bit
    - 1→ used recently; clear and leave alone
    - 0→ selected candidate for replacement

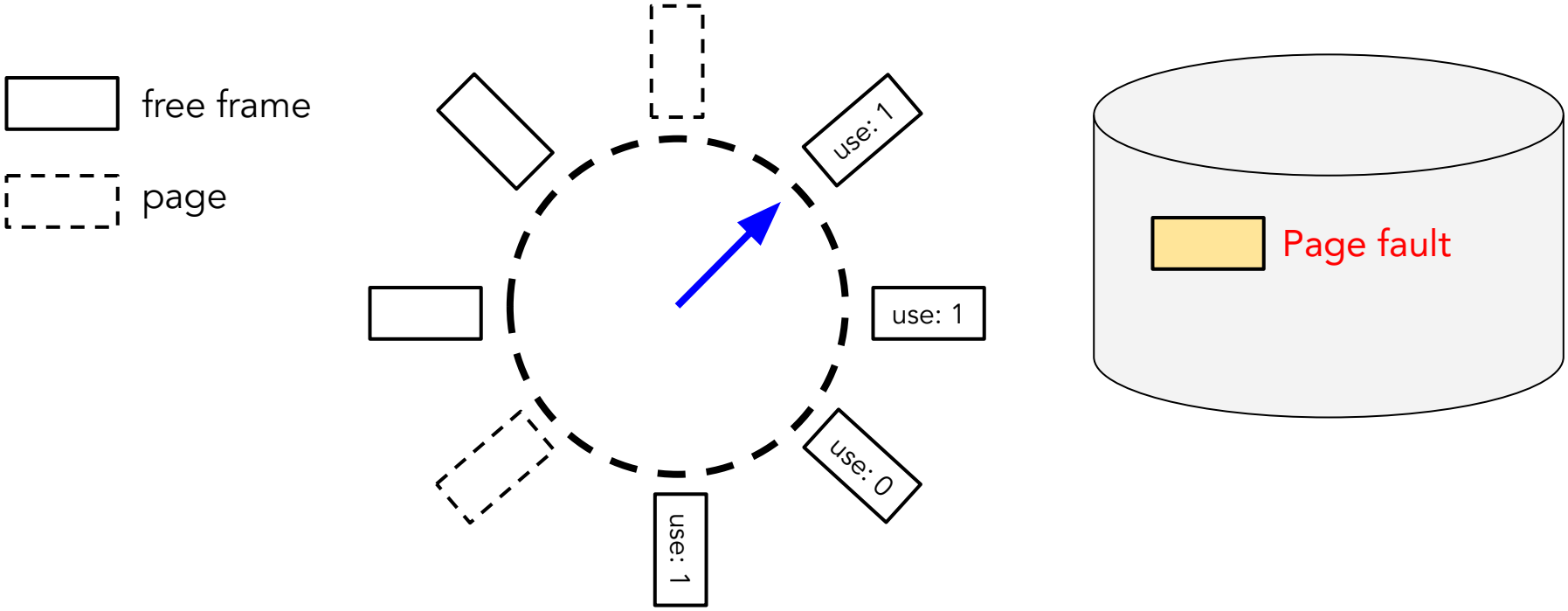
Clock algorithm: Example



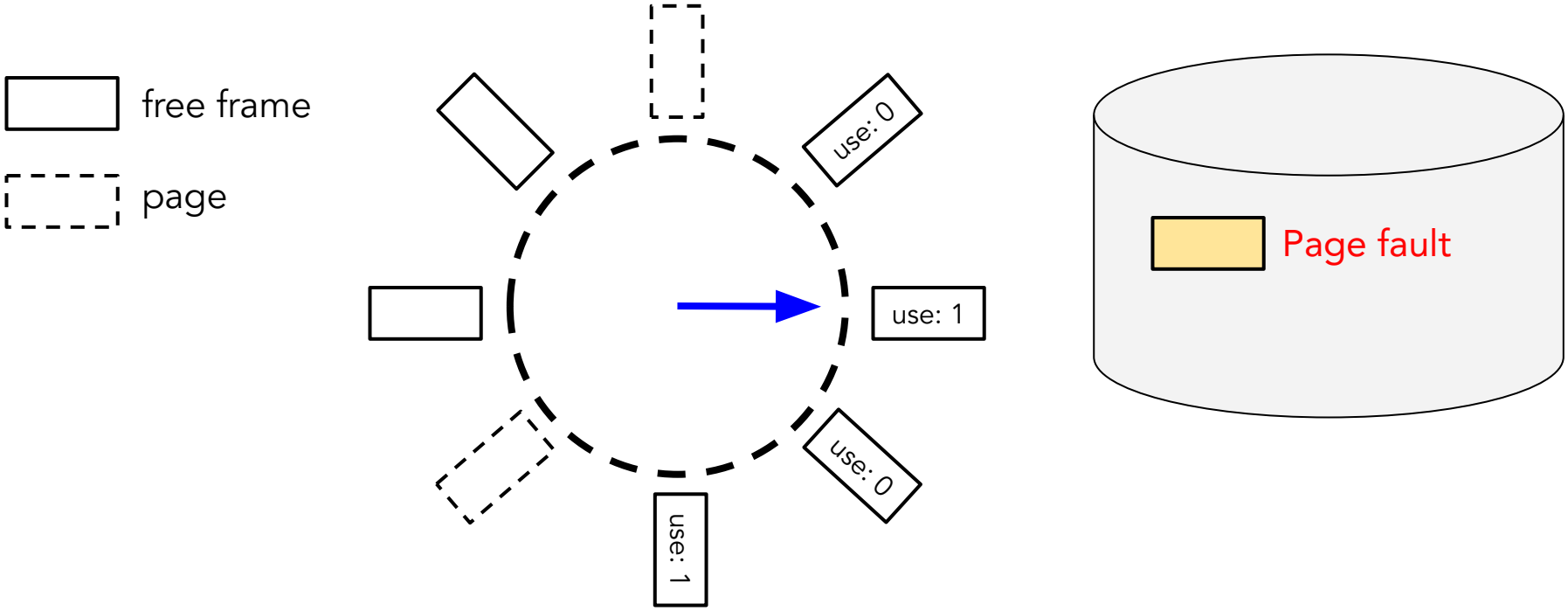
Clock algorithm: Example



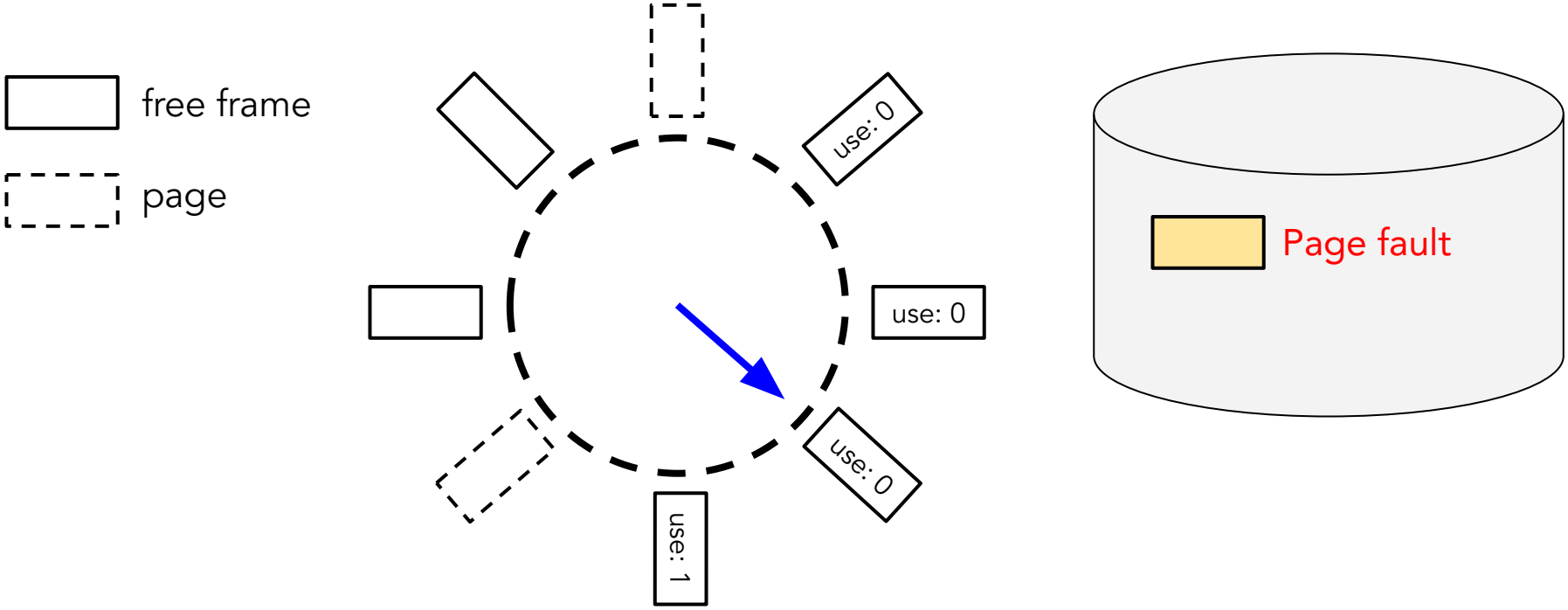
Clock algorithm: Example



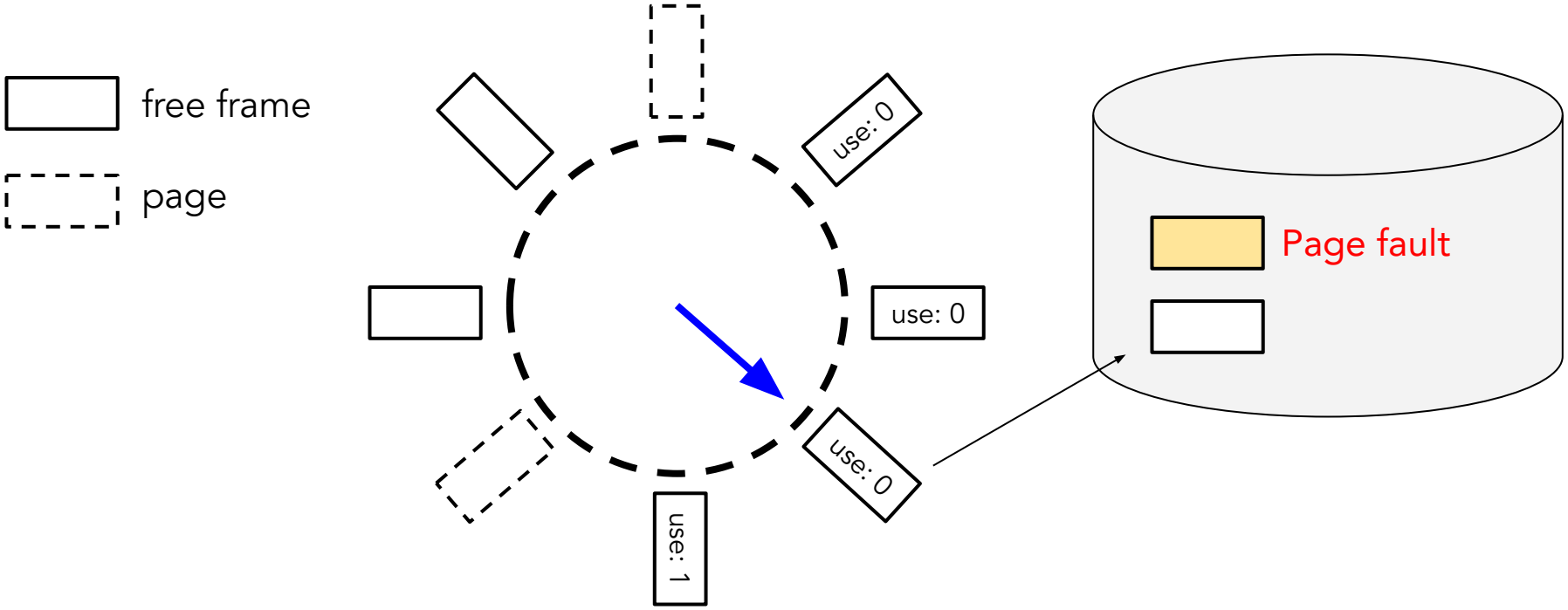
Clock algorithm: Example



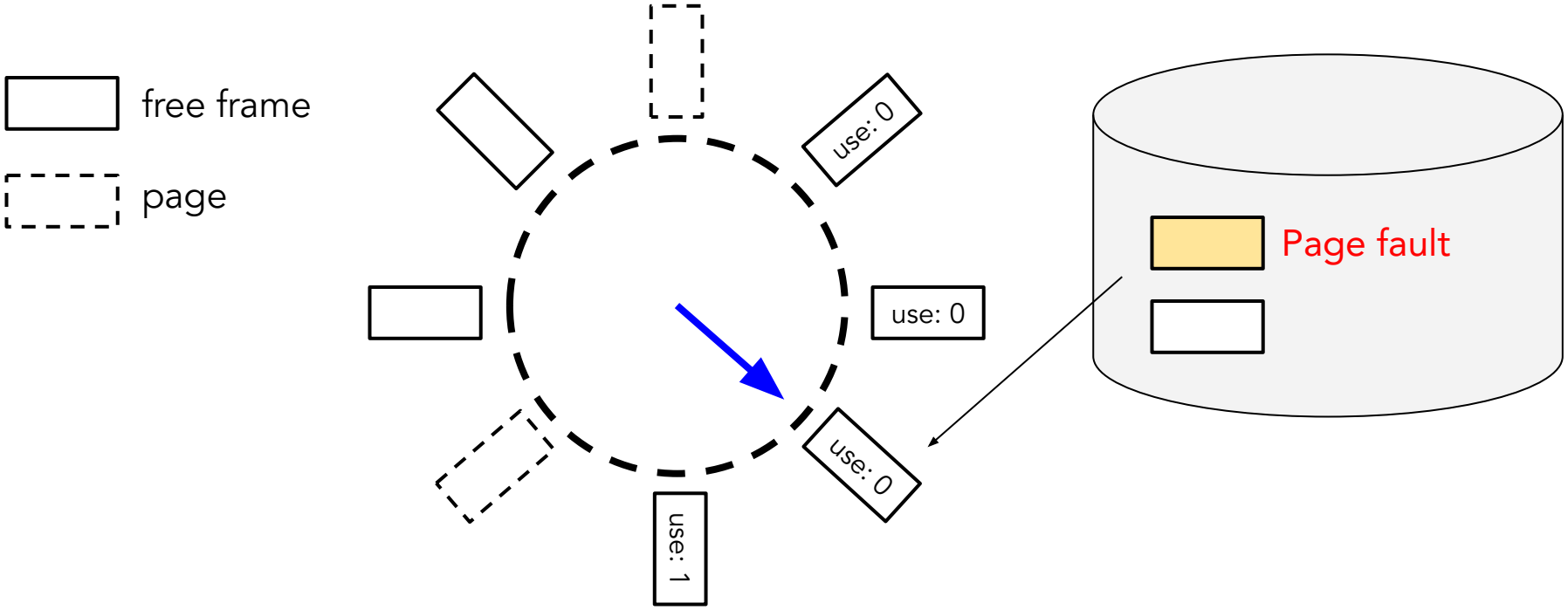
Clock algorithm: Example



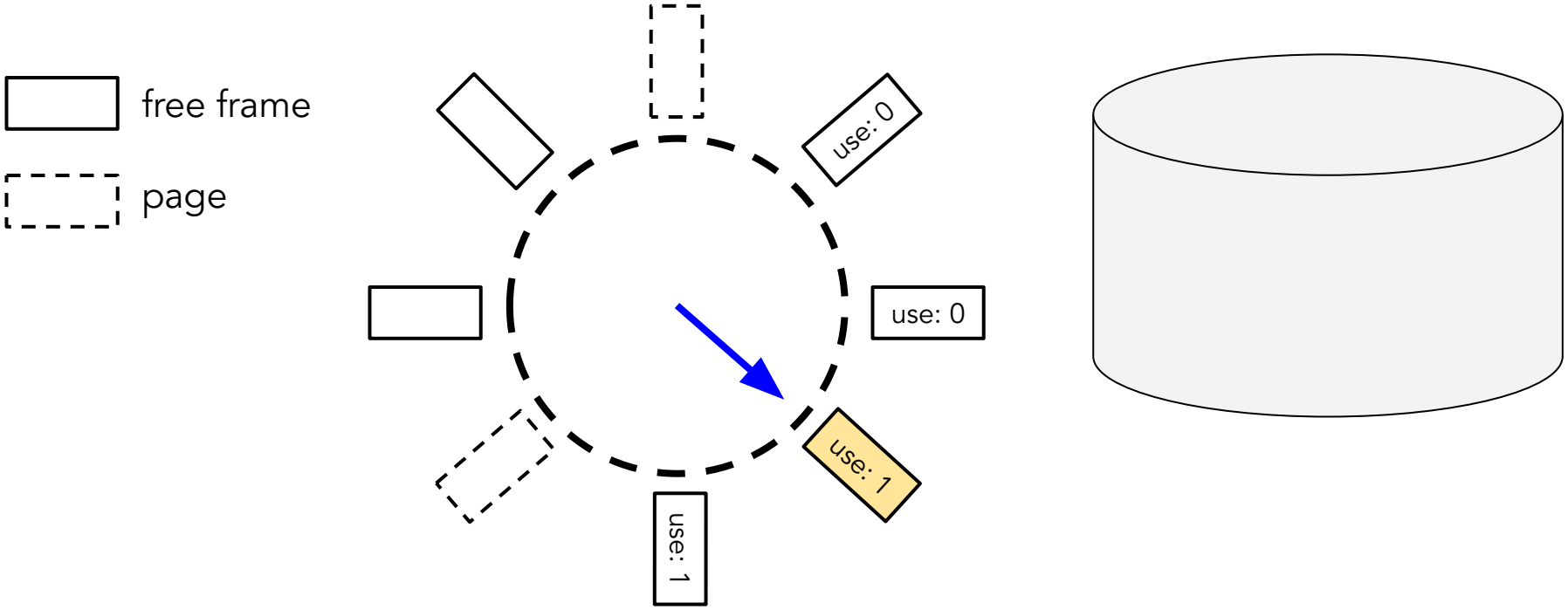
Clock algorithm: Example



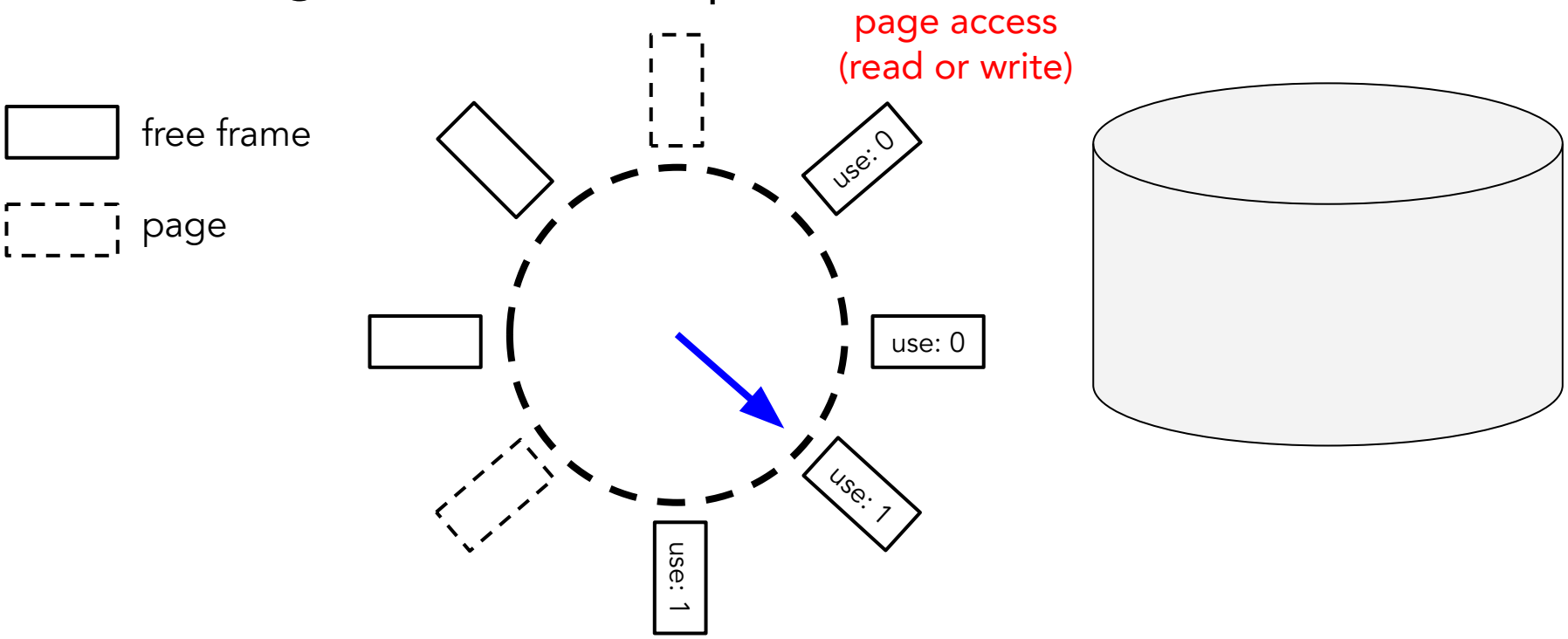
Clock algorithm: Example



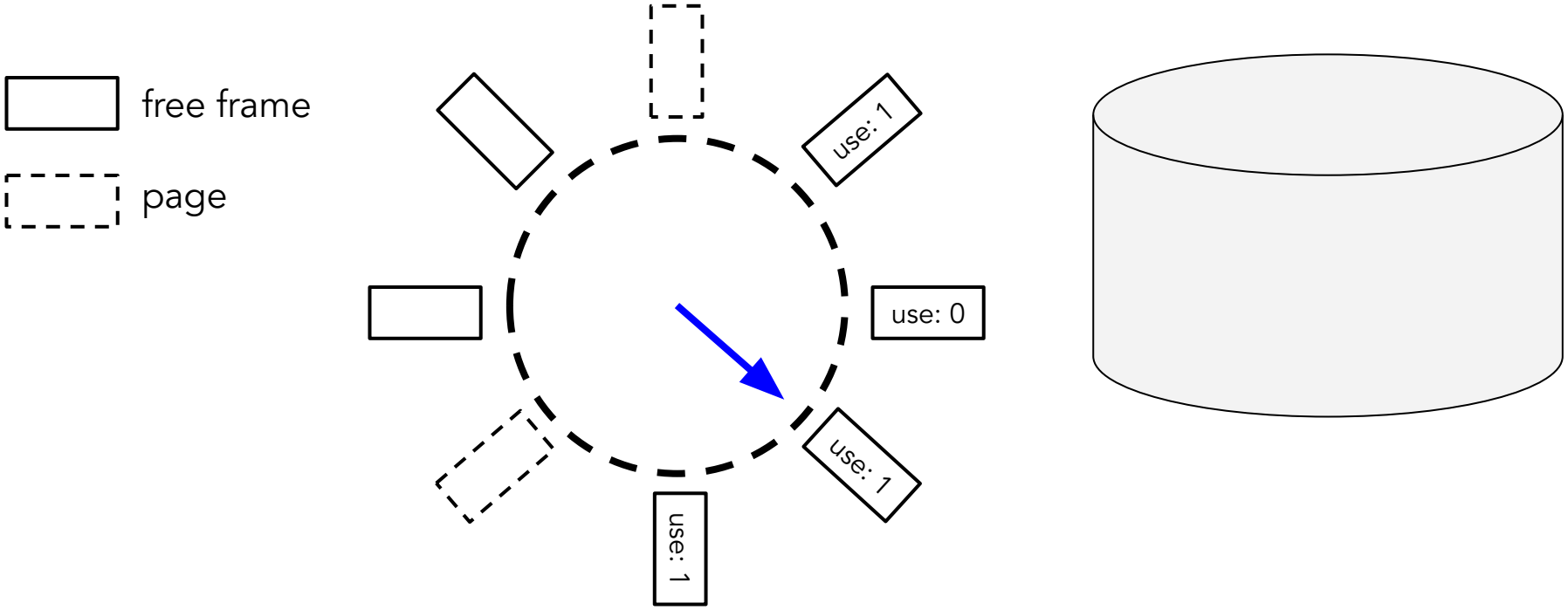
Clock algorithm: Example



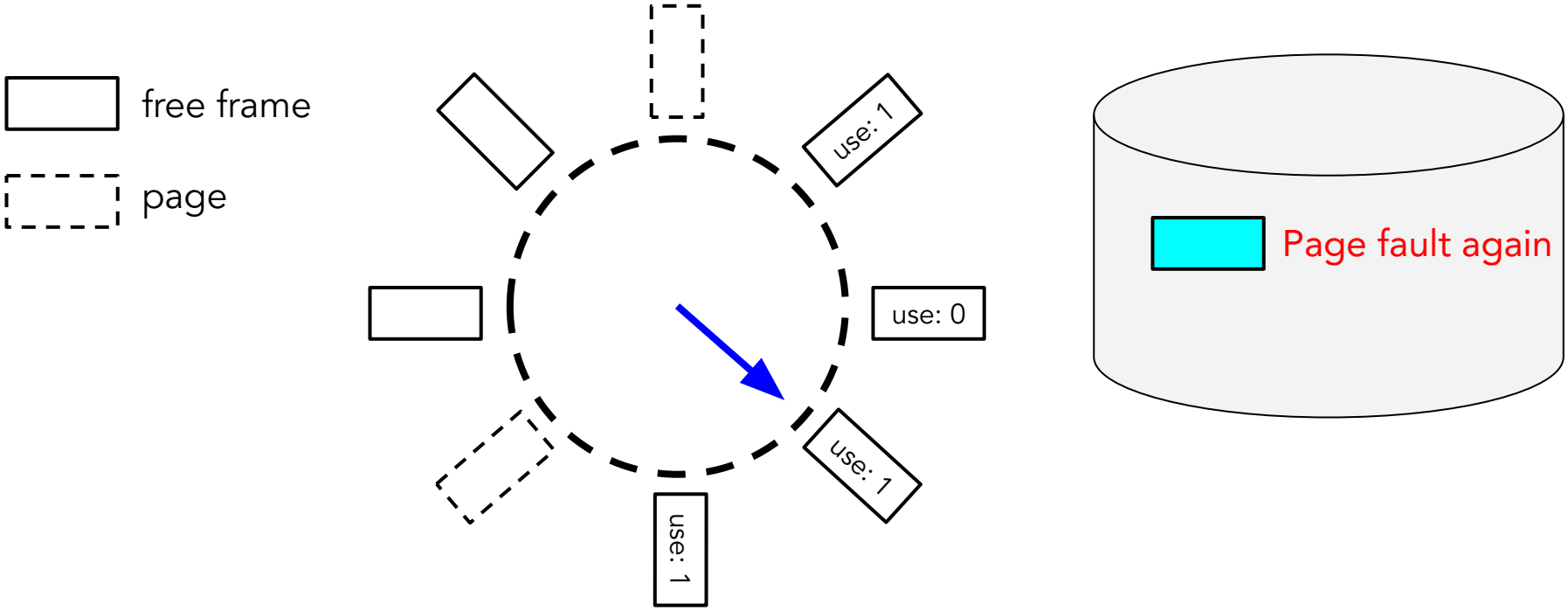
Clock algorithm: Example



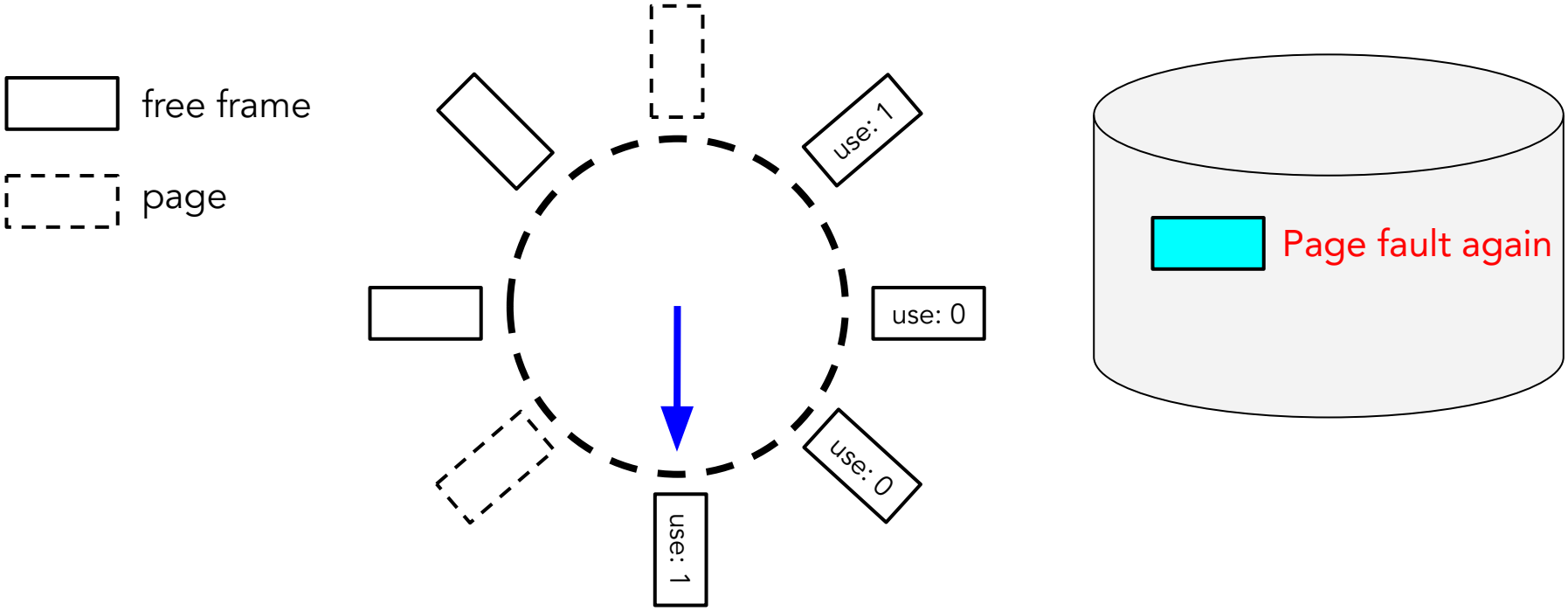
Clock algorithm: Example



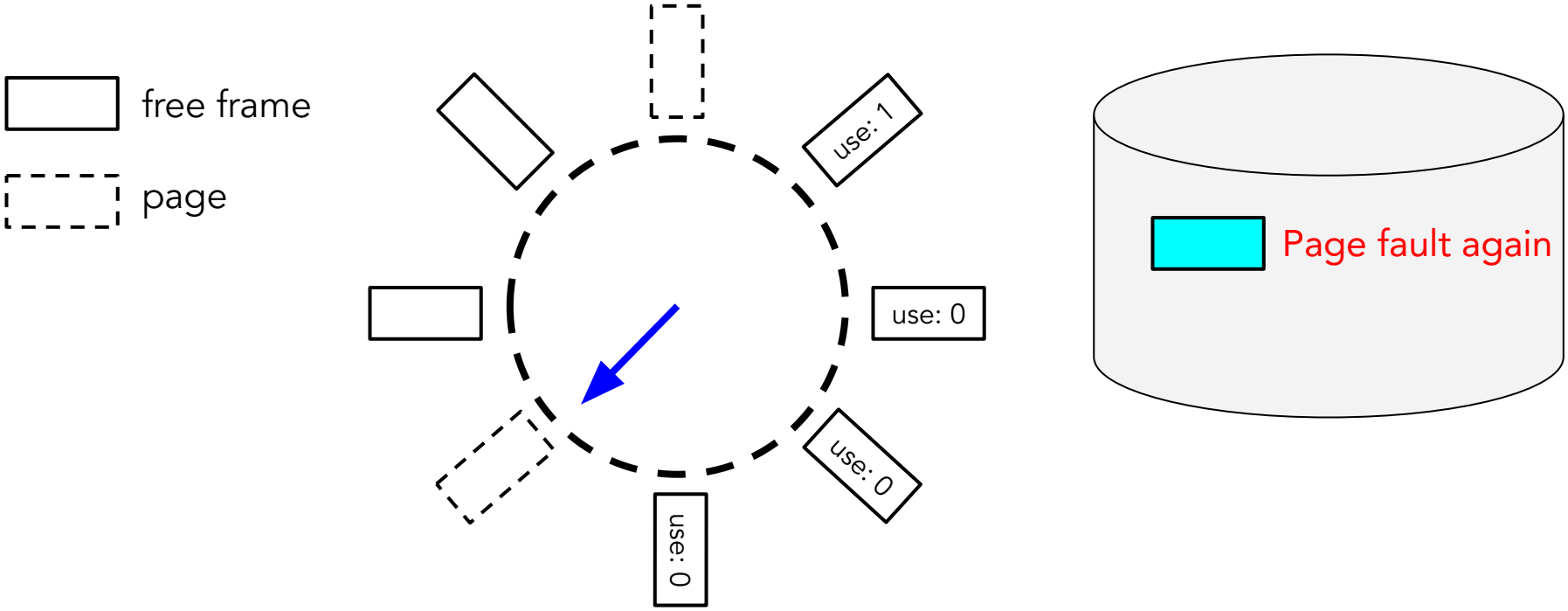
Clock algorithm: Example



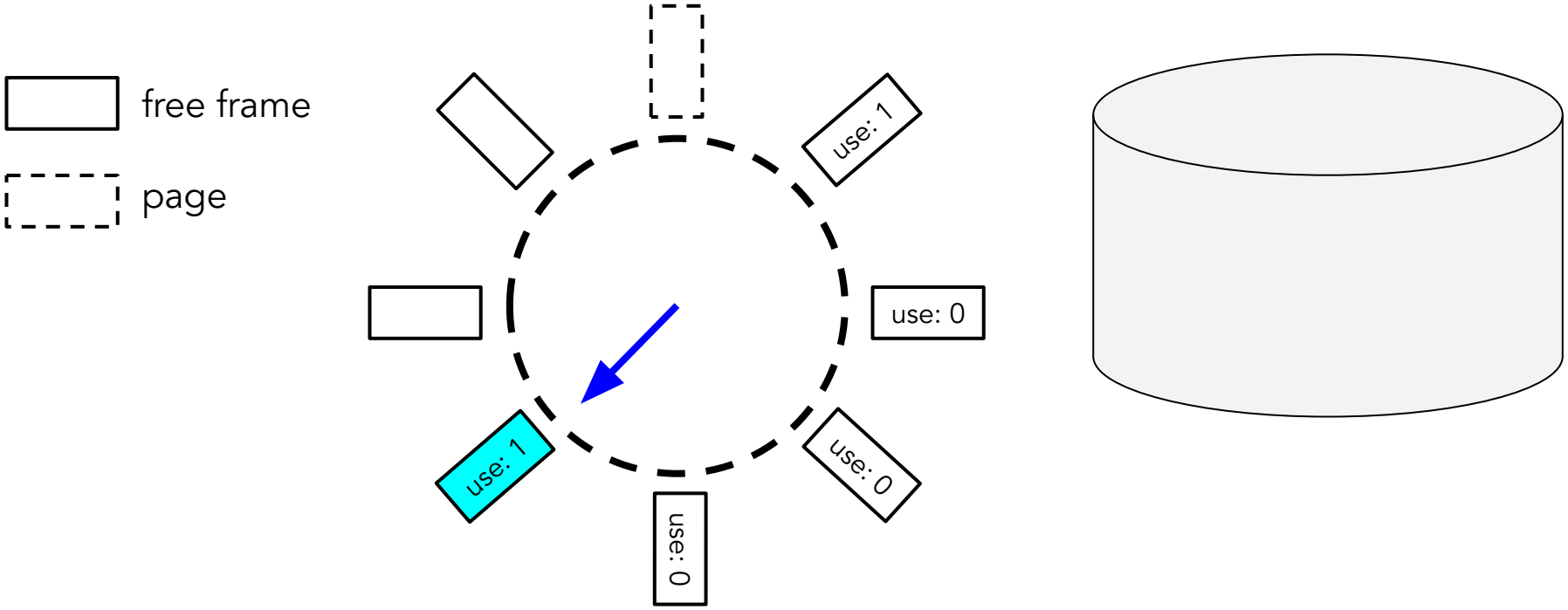
Clock algorithm: Example



Clock algorithm: Example



Clock algorithm: Example



Summary

- Memory management
  - No isolation
  - Base & bound
  - Segmentation
  - Paging
- Cache
  - TLB
  - Demand paging
  - Replacement policy